

18. Bölüm

Tasarım Desenleri

18.1 Desen Nedir?

Nesne yönelimli programlamanın ortaya çıkışıyla beraber, daha önceden yazılmış hazır bileşenlerin (**component**) yeniden kullanımı (**reusing**) konusunda oldukça önemli ilerlemeler sağlamıştır. Bunun paralelinde tecrübelerin (**experience**) yeniden kullanımı konusunda da çeşitli çalışmalar yapılmış ve **desen** (**pattern**) kavramı ortaya çıkmıştır. Desenlerin aksine **anti-desen** (**anti-pattern**) ise **başlangıçta çekici görünen** ancak uygulama sürecinde **verimsiz olduğu kanıtlanmış yaygın hatalı çözümlerdir**.

Yazılım geliştirme öncesi, geliştirme aşması ve sonrasında daha önce yaşanmış çeşitli problemlere karşı birçok kimse tarafından çeşitli çözümler getirilmiştir. **Desenler, daha önce geliştirme yapan programcıların yanlış yaparak doğru yapmayı öğrendikleri başarılı tecrübelerin anlatılması için bir araçtır**. Gerçekleştirilen bu çözümlerin, daha sonradan yaşanacak benzer nitelikli problemlerde kullanılması amacıyla desenler kullanılır. Desen kavramının temelleri Mimar *Christopher Alexander*'ın 1970 sonlarında başlattığı çalışmalara dayanmaktadır¹.

1987 yılında uluslararası “*Nesne Yönelimli Programlama, Sistemler, Diller ve Uygulamalar*” konferansına kadar desenlerle ilgili bir çalışma ortaya çıkmamıştır². Bu tarihten sonra ise başta *Grady Booch*, *Richard Helm*, *Erich Gamma* ve *Kent Beck* olmak üzere pek çok bilim adamı desenlerle ilgili makale ve sunumlar yayınlamışlardır. 1994 yılında *Erich Gamma*, *Richard Helm*, *Ralph Johnson* ve *John Vlissides* tarafından yayınlanan “*Tasarım Desenleri: Tekrar kullanılabilir Nesne Yönelimli Yazılımın Temelleri*” kitabı, tasarım desenlerin yazılımda kullanılmasında dönüm noktası olmuştur³. Yazılımlarda kullanılan desenler yedi ana başlıkta toplanırlar. Bunlar;

1. Mimari desenler (**architectural pattern**)
2. Analiz desenleri (**analysis pattern**)
3. Tasarım desenleri (**design pattern**)
4. Kodlama desenleri (**implementation pattern**)
5. Test desenleri (**test pattern**)
6. Çözüm desenleri (**solution pattern**)
7. Veri desenleri (**data pattern**)

Bu desenlerin en önemlisi ve ilk gelişenlerden birisi, **tasarım desenleridir** (**design pattern**). Tasarım desenleri aslında, Nesne Yönelimli Tasarım Prensiplerini, yazılımlara uygulamanın bir başka ifadesidir. Bu nedenle çok önemlidir. Tasarım desenleri ise üç ana başlıkta toplanmaktadır.

1. Nesne imali ile ilgili desenler (**creational patterns**)
2. Yapısal desenler (**structural patterns**)
3. Davranışlarla ilgili desenler (**behavioral patterns**)

Desenlerin çok sık kullanıldığı programlama yöntemine günümüzde **desen yönelimli programlama** (**pattern oriented programming**) adı verilmektedir⁴. İzleyen sayfalarda verilen desen UML diyagramlarının birçoğu *Erich Gamma*'nın **Design Pattern CD** yayınından alınmıştır⁵. Bölümün devamında verilecek tasarım desenlerinin UML diyagramları visual-paradigm adresinden alınmıştır⁶.

18.2 Nesne İmalatı ile İlgili Desenler

Nesne imalatı ile ilgili desenler (**creational patterns**), nesne (**object**) örnekleme ile ilgili ortaya çıkabilecek çeşitli problemlere çözüm getirmektedirler. Bu desenlerde nesneler dinamik olarak imal edilirler, çünkü değişim yönetiminde statik nesneler kullanılmazlar.

¹<http://gee.cs.oswego.edu/dl/ca/ca/ca.html>

²OOPSLA, Object Oriented Programming, Systems, Languages, and Applications

³Design Patterns: Elements of Reusable Object-Oriented Software, ISBN 0-201-63361-2

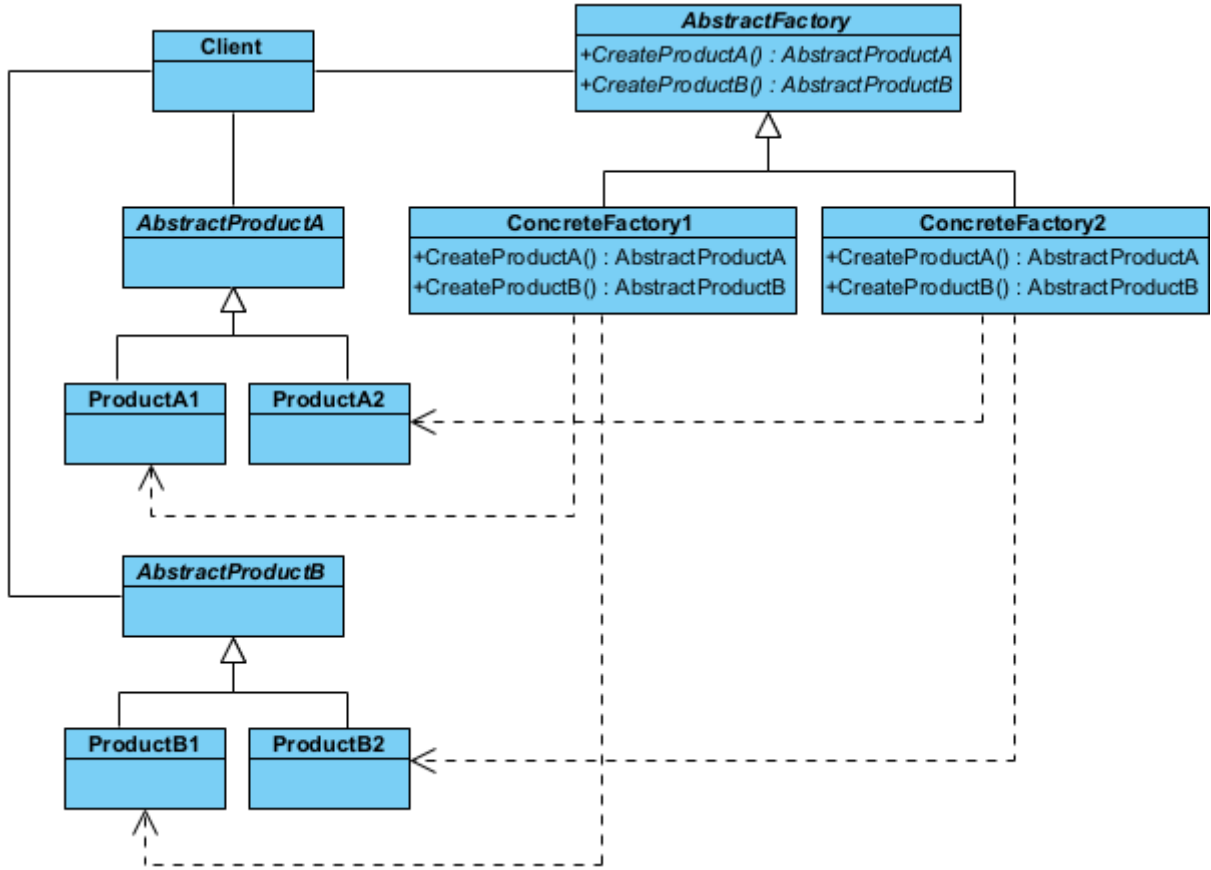
⁴www.mathcs.sjsu.edu/faculty/pearce/patterns.html

⁵Design Pattern CD, Addison-Wesley, 1998

⁶<https://circle.visual-paradigm.com/category/uml-diagrams/gof-design/>

§18.2.1 Soyut Fabrika

Soyut fabrika deseninde (**abstract factory pattern**) amaç, nesnelerin imal edilmesi ve kullanılması ile ilgili olan kısımlarının birbirinden ayrılmasıdır. Bu amaca, birbiriyle ilgili ya da birbirine bağımlı olabilecek nesnelerin imal edilmesinde, nesnelerin somut (**concrete**) sınıflarını değil soyut (**abstract**) sınıfları kullanılarak ulaşılır. Somut sınıflar istemciden izole edilir ve programcıya somut sınıfları daha kolay değiştirme için olanak sağlar.



Şekil 18.1: Soyut Fabrika Deseni UML Diyagramı

Bu tasarım deseninde, birbirleriyle ilişkili veya bağımlı nesne ailelerini, somut sınıflarını belirtmeden oluşturmak için kullanılan imalatla ilgili (**creational**) bir desendir. "Fabrikaların fabrikası" olarak da adlandırılır. Eğer sisteminizin farklı ürün aileleriyle (örneğin; Windows uyumlu buton/pencere ailesi ve Mac uyumlu buton/pencere ailesi) çalışması gerekiyorsa ve bu ailelerin birbirine karışmasını istemiyorsanız bu deseni kullanırsınız.

- ◊ Soyut Fabrika Arayüzü (**AbstractFactory**), ilgili tüm ürünleri oluşturacak yöntemlerin imzalarını (şablonunu) tanımlar. "Bir buton oluştur" ve "Bir pencere oluştur" gibi soyut görevleri belirler.
- ◊ Somut Fabrikalar (**ConcreteFactory**), belirli bir ürün ailesine ait nesneleri fiilen oluşturur. Örneğin **ModernMobilyaFabrikasi** sadece modern tarzdaki sandalyeleri ve masaları üretir.
- ◊ Soyut Ürünler (**AbstractProduct**), ürün ailesindeki her bir temel ürün türü için bir arayüz tanımlar. Tüm sandalyelerin sahip olması gereken ortak özellikleri (Örnek olarak **Otur()**) belirler.
- ◊ Somut Ürünler (**ConcreteProduct**), soyut ürünlerin farklı varyasyonlarını (ailelerini) temsil eden gerçek sınıflardır. **ModernSandalye** veya **AntikaSandalye** gibi sınıflar burada yer alır.

Bu deseni kodlayalım;

```

// 1. Products (Ürünler)
// --- Abstract Products ---
public abstract class AbstractProductA
{
    public string ProductName { get; }
    protected AbstractProductA(string pAdi) => ProductName = pAdi;
    public virtual string GetUrunAdi() => ProductName;
}
public class ConcreteProductA1 : AbstractProductA

```

```

{
    public ConcreteProductA1(string pAdi) : base(pAdi) { }
}
public class ConcreteProductA2 : AbstractProductA
{
    public ConcreteProductA2(string pAdi) : base(pAdi) { }
}
// --- Abstract Product B ---
public abstract class AbstractProductB
{
    public string ProductName { get; }
    protected AbstractProductB(string pAdi) => ProductName = pAdi;
    public virtual string GetUrunAdi() => ProductName;

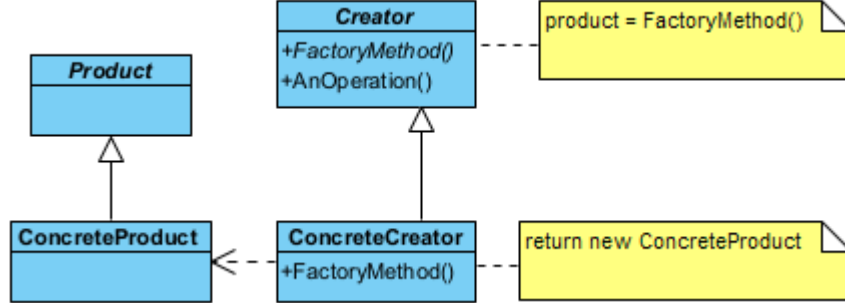
    public void Interact(AbstractProductA a)
    {
        Console.WriteLine($"{this.ProductName} ürünü {a.GetUrunAdi()} ile etkileşim halinde...");
    }
}
public class ConcreteProductB1 : AbstractProductB
{
    public ConcreteProductB1(string pAdi) : base(pAdi) { }
}
public class ConcreteProductB2 : AbstractProductB
{
    public ConcreteProductB2(string pAdi) : base(pAdi) { }
}
// 2. Factory (Fabrika)
// C#'ta soyut fabrikalar için genellikle interface (arayüz) tercih edilir. Çünkü özellik ve alan
→ tanımlaması içermez.
public interface IAbstractFactory
{
    AbstractProductA CreateProductA();
    AbstractProductB CreateProductB();
}
public class ConcreteFactory1 : IAbstractFactory
{
    public AbstractProductA CreateProductA() => new ConcreteProductA1("A1 ürünü");
    public AbstractProductB CreateProductB() => new ConcreteProductB1("B1 ürünü");
}
public class ConcreteFactory2 : IAbstractFactory
{
    public AbstractProductA CreateProductA() => new ConcreteProductA2("A2 ürünü");
    public AbstractProductB CreateProductB() => new ConcreteProductB2("B2 ürünü");
}
// 3. Client (İstemci)
class Program
{
    static void Main(string[] args)
    {
        IAbstractFactory factory1 = new ConcreteFactory1();
        IAbstractFactory factory2 = new ConcreteFactory2();
        var productA = factory1.CreateProductA();
        Console.WriteLine($"Fabrika 1 ürünü: {productA.GetUrunAdi()}");
        var productB = factory2.CreateProductB();
        Console.WriteLine($"Fabrika 2 ürünü: {productB.GetUrunAdi()}");
        productB.Interact(productA);
    }
}
/*Program Çıktısı:
Fabrika 1 ürünü: A1 ürünü
Fabrika 2 ürünü: B2 ürünü

```

B2 Ürünü ürünü A1 ürünü ile etkileşim halinde...
*/

§18.2.2 Fabrika Yöntemi

Fabrika yöntemi deseninde (**factory method pattern**) amaç, imal edilecek nesnenin sınıfını kesin olarak belirtmeden nesne yaratma işleminin gerçekleştirilmesidir. Bu desenin görevi, nesne üretim işini alt sınıflara



Şekil 18.2: Fabrika Yöntemi Deseni UML Diyagramı

bırakır. Hangi nesnenin üretileceğine çalışma zamanında karar verilir.

- ◊ Üretici (**Creator**), fabrika yöntemini tanımlayan soyut sınıftır.
- ◊ Ürün (**Product**), üretilecek nesnelerin ortak arayüzüdür.
- ◊ Somut Üretici (**ConcreteCreator**), ihtiyaca göre "A Ürünü" veya "B Ürünü" üreten gerçek fabrikadır.

Bu deseni kodlayalım;

```

// 1. Products (Ürünler)
// --- Ürün Sınıfları ---
public abstract class Product
{
    public string ProductName { get; }
    protected Product(string pAdi)
    {
        ProductName = pAdi;
    }
    public virtual string GetUrunAdi() => ProductName;
}
public class ConcreteProduct : Product
{
    public ConcreteProduct(string pAdi) : base(pAdi) { }
}
// 2. Creators (İmalatçılar)
// --- Creator (İmalatçı) Sınıfları ---
public abstract class Creator
{
    // Fabrika Yöntemi: Alt sınıflar tarafından geçersiz kılınacak olan metot.
    public abstract Product FactoryMethod();
    public abstract void AnOperation();
}
public class ConcreteCreator : Creator
{
    public override Product FactoryMethod()
    {
        return new ConcreteProduct("Fabrika Yöntemi Ürünü");
    }
    public override void AnOperation()
    {
        Console.WriteLine("Fabrika Yöntemini Kullanan Sınıf Başka İşlemler de yapabilir...");
    }
}
// 3. Client (İstemci)
class Program
{

```

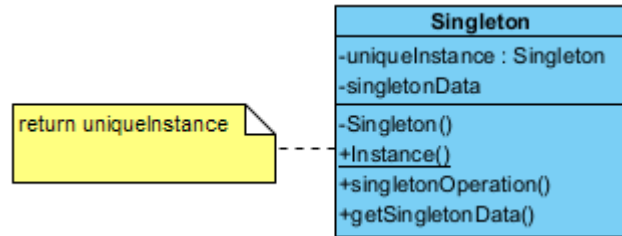
```

static void Main(string[] args)
{
    // 1. Creator (İmalatçı) nesnesini oluşturuyoruz.
    Creator imalatci = new ConcreteCreator();
    // 2. Ürünü oluşturuyoruz. Bellek yönetimi tamamen .NET tarafından yapılır.
    Product urun = imalatci.FactoryMethod();
    Console.WriteLine($"İmalatçı tarafından imal edilen ürün: {urun.GetUrunAdi()}");
    imalatci.AnOperation();
}
}
/*Program Çıktısı:
İmalatçı tarafından imal edilen ürün: Fabrika Yöntemi Ürünü
Fabrika Yöntemini Kullanan Sınıf Başka İşlemler de yapabilir...
*/

```

§18.2.3 Tekil Nesne

Tekil Nesne deseninde (singleton pattern) amaç, bir sınıftan yalnızca bir örnek (instance) imal etmektir. Bir sınıfın yalnızca bir nesnesinin olmasına izin verilir, birden fazla olmasına izin verilmez.



Şekil 18.3: Tekil Deseni UML Diyagramı

Şimdi Singleton sınıfını kodlayalım;

```

//1. Singleton sınıfı: Öyle bir sınıftır ki yalnızca bir örneği(instance) vardır.
class Singleton
{
    // Sınıftan yaratılan örneklerle değil, sınıfa ait üye değişken (class scope member variable)
    // tanımlanıyor.
    private static Singleton? _uniqueInstance=null;
    private int _singletonData;
    // Birden fazla örneğe(instance) izin vermemek için yapıcı(constructor) işlevin
    // görünürlüğü(visibility) özel(private) olmalıdır.
    private Singleton()
    {
        _singletonData=0;
    }
    //Tekil Nesneyi verecek Yöntem:
    public static Singleton Instance()
    {
        if (_uniqueInstance == null)
            _uniqueInstance = new Singleton();
        return _uniqueInstance;
    }
    //Tekil nesnenin yöntem ve özellikleri:
    public void SingletonOperation()
    {
        Console.WriteLine("Singleton İşlevi Çağrıldı.");
    }
    public int SingletonData {
        get
        {
            return _singletonData;
        }
        set{
            _singletonData=value;
        }
    }
}

```

```

    }
}
//2. İstemci
class Client
{
    static void Main(string[] args)
    {
        Singleton object1 = Singleton.Instance();
        Singleton object2 = Singleton.Instance();
        if (object1 == object2)
            Console.WriteLine
            ("object1 ve object2 AYNI nesnedir.");
        else
            Console.WriteLine
            ("object1 ve object2 FARKLI nesnedir.");
        object1.SingletonOperation();
    }
}
/* Program Çıktısı:
object1 ve object2 AYNI nesnedir.
Singleton İşlevi Çağrıldı.
*/

```

Bu desende `if (instance == null)` kontrolüyle nesne oluşturulduğu görülüyor. Bu yöntem çoklu iş parçacığı (**multi-threading**) ortamlarında aynı anda iki nesnenin oluşmasına neden olabilir. Bunun için çift kilitleme (**double-check locking**) iş parçacığı güvenli tekil nesne (**thread-safe singleton**) desnei veya .NET'in **Lazy<T>** sınıfını kullanmalıyız;

```

//1. Singleton Sınıfı
public sealed class VeritabaniBaglantisisi
{
    // Lazy<T> kullanımı, nesnenin sadece .Value özelliğine erişildiğinde
    // ve thread-safe bir şekilde oluşturulacağını garanti eder.
    private static readonly Lazy<VeritabaniBaglantisisi> _instance = new
    ↪ Lazy<VeritabaniBaglantisisi>(() => new VeritabaniBaglantisisi());

    // Dışarıdan erişim için tek nokta
    public static VeritabaniBaglantisisi Instance => _instance.Value;

    // Yapıcı metodun private olması, dışarıdan new ile nesne oluşturulmasını engeller.
    private VeritabaniBaglantisisi()
    {
        // Ağır yük getiren başlangıç işlemleri burada yapılabilir.
        Console.WriteLine("Veritabanı bağlantı nesnesi oluşturuldu.");
    }

    public void Baglan()
    {
        Console.WriteLine("Veritabanına bağlanıldı.");
    }
}

//2. İstemci
class Client
{
    static void Main(string[] args)
    {
        // Programın herhangi bir yerinde:
        // Bu satır çağrılana kadar nesne bellekte oluşturulmaz.
        VeritabaniBaglantisisi.Instance.Baglan();
    }
}

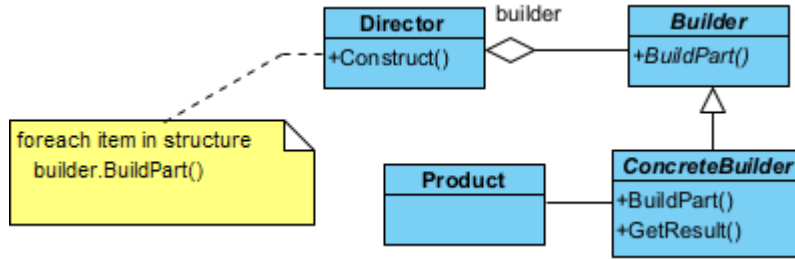
```

Neden `Lazy<T>` Kullanmalıyız?

- ◊ **İş Parçacığı-Güvenli (thread-safe) Olması:** `Lazy<T>` varsayılan olarak **thread-safe** çalışır. `lock` anahtar kelimesini manuel olarak kullanmanıza (**double-check locking**) gerek kalmaz.
- ◊ **Performans:** Nesne uygulama başladığında değil, gerçekten `Instance` özelliğine ilk kez erişildiğinde oluşturulur.
- ◊ **Okunabilirlik:** Karmaşık kilit mekanizmalarına kıyasla çok daha temiz ve anlaşılır bir söz dizimi sunar.
- ◊ **Hata Yönetimi:** Eğer nesne oluşturulurken bir hata (**exception**) oluşursa, `Lazy<T>` bu durumu yönetebilir.

§18.2.4 Kurucu

Kurucu deseninde (builder pattern) amaç, çok karmaşık bir nesnenin imal edilmesiyle ilgili işlemleri bir başka sınıfa bırakmaktır. Böylece nesnenin gösterimi ve kullanılması ilişkin kodlar ile imalatına ilişkin kodlar birbirinden ayrılmış olur.



Şekil 18.4: Kurucu Deseni UML Diyagramı

Bu deseni bir fast-food restoranındaki "Menü Hazırlama" sürecine benzetebilirsin. Hamburgerin içine turşu konulup konulmayacağı, içeceğin boyutu veya yan ürünün ne olacağı gibi adımlar tek tek belirlenir, ancak en sonunda ortaya tek bir "Menü" nesnesi çıkar.

- ◊ **Ürün (Product)**, inşa edilmeye çalışılan karmaşık nesnedir. Genellikle çok sayıda parçadan veya konfigürasyondan oluşur.
- ◊ **Soyut Kurucu (Builder)**, ürünün parçalarını oluşturmak için gerekli olan tüm adımları (yöntemleri) arayüz olarak tanımlar. "Ekmek koy", "Köfte ekle", "Sos dök" gibi soyut inşa adımlarını belirler.
- ◊ **Somut Kurucu (ConcreteBuilder)**, soyut kurucudaki adımları fiilen gerçekleştirir ve ürünün bir örneğini saklar. "Vejetaryen Burger" veya "Double Burger" gibi farklı ürün varyasyonlarını nasıl inşa edeceğini bilir.
- ◊ **Yönetici (Director)**, inşa adımlarının hangi sırayla çağrılacağını kontrol eder. İstemciden (**Client**) aldığı kurucu nesnesini kullanarak, doğru sırayla (Örn: Önce temel, sonra yan ürünler) inşa sürecini yürütür.

```

using System;
using System.Collections.Generic; // List<T> için gerekli
// 1. Product (Ürün)
class Product
{
    private readonly List<string> _parts = new List<string>();

    public void Add(string part)
    {
        _parts.Add(part);
    }

    public void Show()
    {
        Console.WriteLine("\nÜrün Parçaları:");
        foreach (string part in _parts)
        {
            Console.WriteLine($"- {part}");
        }
    }
}
// 2. Builder (Soyut Kurucu)
abstract class Builder
{

```

```

    public abstract void BuildPartA();
    public abstract void BuildPartB();
    public abstract Product GetResult();
}
// 3. Concrete Builders (Somut Kurucular)
class ConcreteBuilder1 : Builder
{
    private Product _product = new Product();
    public override void BuildPartA() => _product.Add("A Parçası");
    public override void BuildPartB() => _product.Add("B Parçası");
    public override Product GetResult()
    {
        Product finishedProduct = _product;
        _product = new Product(); // Yeni bir inşa için builder'ı sıfırlıyoruz.
        return finishedProduct;
    }
}
class ConcreteBuilder2 : Builder
{
    private Product _product = new Product();
    public override void BuildPartA() => _product.Add("X Parçası");
    public override void BuildPartB() => _product.Add("Y Parçası");

    public override Product GetResult()
    {
        Product finishedProduct = _product;
        _product = new Product();
        return finishedProduct;
    }
}
// 4. Director (Yönetici)
class Director
{
    // İnşa sürecinin algoritmasını Director belirler.
    public void Construct(Builder builder)
    {
        builder.BuildPartA();
        builder.BuildPartB();
    }
}
// 5. Client (İstemci)
class Program
{
    static void Main(string[] args)
    {
        Director director = new Director();
        // Builder 1 ile inşa
        Builder b1 = new ConcreteBuilder1();
        director.Construct(b1);
        Product p1 = b1.GetResult();
        p1.Show();
        // Builder 2 ile inşa
        Builder b2 = new ConcreteBuilder2();
        director.Construct(b2);
        Product p2 = b2.GetResult();
        p2.Show();
    }
}
/*Program Çıktısı:
Ürün Parçaları:
- A Parçası
- B Parçası

```

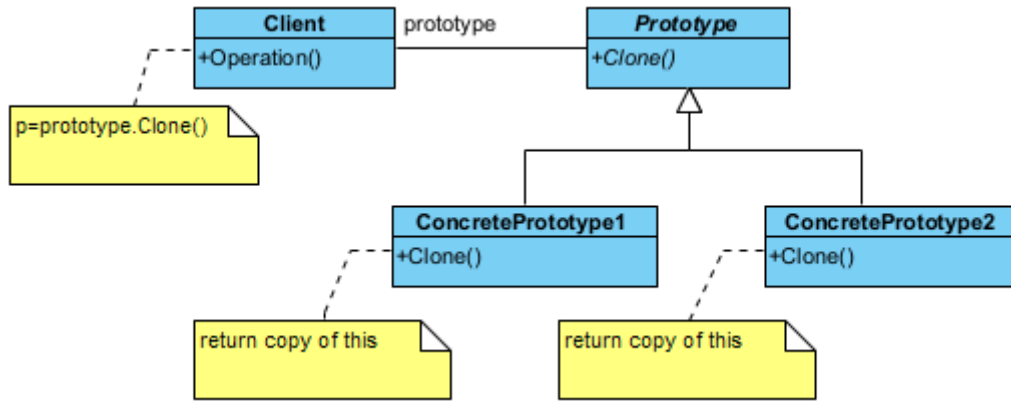

Ürün Parçaları:

- X Parçası
 - Y Parçası
- */

§18.2.5 Prototip

Prototip deseni (**prototype pattern**), mevcut bir nesnenin kopyasını (klonunu) oluşturmak için kullanılan imalatla ilgili bir desendir. Bu desenin temel amacı, bir nesneyi oluşturmanın maliyeti (zaman, bellek veya işlem gücü açısından) çok yüksek olduğunda, sıfırdan **new** işleci ile üretmek yerine mevcut bir örneği klonlayarak yeni nesneler elde etmektir.

- ◊ Soyut Prototip (**Prototype**), nesnenin kendisini kopyalayabilmesi için gerekli olan arayüzü tanımlar. Genellikle sadece bir **clone()** yönteminin imzasını barındırır.
- ◊ Somut Prototip (**ConcretePrototype**), soyut prototip arayüzünü uygular ve asıl kopyalama işlemini gerçekleştirir. Kendi verilerini yeni bir nesneye aktarır.
- ◊ İstemci (**Client**), yeni bir nesneye ihtiyaç duyduğunda, sınıfa gidip "yeni bir tane üret" demek yerine, elindeki prototip nesnesine "kendini klonla" der.



Şekil 18.5: Prototip Deseni UML Diyagramı

Şimdi bu deseni kodlayalım;

```

// 1. Prototip Arayüzü
public abstract class Prototype
{
    protected string _data;
    protected Prototype(string data) => _data = data;
    public abstract Prototype Clone();
    public void ShowData()
    {
        Console.WriteLine($"Nesne Verisi: {_data}");
    }
}

// 2. Somut Prototip (Concrete Prototype)
public class ConcretePrototype : Prototype
{
    public ConcretePrototype(string data) : base(data) { }
    // Bu metot "Shallow Copy" (Sığ Kopyalama) yapar.
    public override Prototype Clone()
    {
        // MemberwiseClone, nesnenin tüm alanlarını yeni bir nesneye kopyalar.
        return (Prototype)this.MemberwiseClone();
    }
}

// 3. İstemci (Client)
class Program
{
    static void Main(string[] args)
    {

```

```
// 1. Orijinal nesneleri oluşturalım
Prototype object1 = new ConcretePrototype("40.30");
object1.ShowData();
// 2. Klonlama işlemi
Prototype clone1 = object1.Clone();
clone1.ShowData();
Prototype object2 = new ConcretePrototype("Mehmet");
object2.ShowData();
Prototype clone2 = object2.Clone();
clone2.ShowData();
// Nesnelerin aynı olup olmadığını kontrol edelim (Adres kontrolü)
Console.WriteLine($"Nesneler aynı mı?: {ReferenceEquals(object1, clone1)}");
}
}
/*Program Çıktısı:
Nesne Verisi: 40.30
Nesne Verisi: 40.30
Nesne Verisi: Mehmet
Nesne Verisi: Mehmet
Nesneler aynı mı?: False
*/
```

Nesneleri kopyalama söz konusu olduğunda, genel olarak üç tür kavramdan bahsedilir;

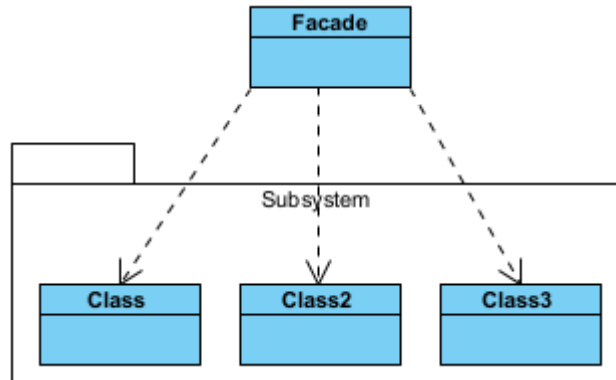
1. **Üstünkörü kopyalamada (shallow copy)** yeni nesne eskisini referans gösterir. Bu durumda birinci nesnenin bir alanı (**field**) değiştiğinde ikincisinin ki de değişmiş olur. Yani birinde olan değişiklik diğerini etkiler. Bu kopyalamanın diğer adı da bit düzeyi kopyalamadır (**bitwise copy**).
2. **Derinlemesine kopyalamada (deep copy)** ise birinci nesnenin aynısı bir başka bellek bölgesinde oluşturulur ve ikinci nesne yeni bellek bölgesini referans gösterir. Bu kopyalamanın diğer adı da üye düzeyi kopyalamadır (**memberwise copy**). Kopyalama sonrasında bir nesne üzerinde olan değişiklik yeni nesneyi etkilemez.
3. Üçüncü tip bir kopyalama ise **tembel kopyalamadır (lazy copy)**. Yukarıda bahsi geçen iki kopyalamanın iç içe girmiş şeklidir

18.3 Yapısal Desenler

Yapısal desenler (structural pattern), sınıflar ve nesnelerin çok olduğu daha karmaşık programlarda getirilen yapısal çözüm yöntemleridir.

§18.3.1 Önyüz

Önyüz deseninde (facade pattern) karmaşık bir alt sistemdeki (**subsystem**) çok sayıda sınıfı ve arayüzü, kullanımı kolay tek bir üst düzey arayüz arkasında gizlemek için kullanılan yapısal bir desendir. Bu deseni bir restoranın "Garsonu" gibi düşünebilirsin. Sen sadece sipariş verirsın; garson ise mutfak, bulaşıkhanenin ve kasanın karmaşıklığıyla senin adına ilgilenir.



Şekil 18.6: Önyüz Deseni UML Diyagramı

- ◊ Önyüz (**Facade**), karmaşık alt sistemleri bilen ve onları yöneten "ana giriş kapısı"dır. İstemciden (**Client**) gelen basit isteği, alt sistemdeki doğru nesnelere yönlendirir ve onları uygun sırayla çalıştırır.
- ◊ Alt Sistem Sınıfları (**SubSystem**), işin asıl detaylarını yapan çok sayıda sınıftır (Örn: SesSistemi, Işık-Denetleyici, Projeksiyon). **Facade** nesnesinden tamamen habersizdirler. Kendi işlerini yaparlar. **Facade**

sadece bunları koordine eder.

- ◊ İstemci (**Client**), sistemi kullanan son kullanıcı veya başka bir kod parçasıdır. Alt sistemin karmaşık detaylarıyla (hangi sınıf hangi sırayla çağrılacak gibi) uğraşmaz; sadece **Facade**'in sunduğu basit yöntemleri çağırır.

Şimdi desenimizi kodlayalım;

```
// 1. Alt Sistemler (Subsystems)
// Bu sınıflar karmaşık iş mantığını barındırır.
public class SubSystem1
{
    public void Operation11() => Console.WriteLine("Subsystem1: Yöntem 1 çalıştırıldı.");
}
public class SubSystem2
{
    public void Operation22() => Console.WriteLine("Subsystem2: Yöntem 2 çalıştırıldı.");
}
public class SubSystem3
{
    public void Operation31() => Console.WriteLine("Subsystem3: Yöntem 1 çalıştırıldı.");
    public void Operation32() => Console.WriteLine("Subsystem3: Yöntem 2 çalıştırıldı.");
    public void Operation33() => Console.WriteLine("Subsystem3: Yöntem 3 çalıştırıldı.");
}
// 2. Facade (Önyüz) Sınıfı
// İstemciye (Client) basit bir arayüz sunar, karmaşayı yönetir.
public class Facade
{
    // C#'ta nesneler referans tipli olduğu için 'new' ile doğrudan oluşturulur.
    // Bellek yönetimi Garbage Collector tarafından yapılır.
    private readonly SubSystem1 _subsystem1;
    private readonly SubSystem2 _subsystem2;
    private readonly SubSystem3 _subsystem3;
    public Facade()
    {
        _subsystem1 = new SubSystem1();
        _subsystem2 = new SubSystem2();
        _subsystem3 = new SubSystem3();
    }
    public void ExecuteComplexOperation()
    {
        Console.WriteLine("--- Facade Karmaşık İşlemi Başlatıyor ---");
        _subsystem1.Operation11();
        _subsystem2.Operation22();
        _subsystem3.Operation31();
        _subsystem3.Operation33();
        _subsystem3.Operation32();
        Console.WriteLine("--- İşlem Tamamlandı ---");
    }
}
// 3. Client (İstemci)
class Program
{
    static void Main(string[] args)
    {
        // İstemci alt sistemlerin detaylarını bilmek zorunda değil.
        // Sadece Facade ile konuşur.
        Facade facade = new Facade();
        facade.ExecuteComplexOperation();
    }
}
/*Program Çıktısı:
--- Facade Karmaşık İşlemi Başlatıyor ---
Subsystem1: Yöntem 1 çalıştırıldı.
```

```
Subsystem2: Yöntem 2 çalıştırıldı.
Subsystem3: Yöntem 1 çalıştırıldı.
Subsystem3: Yöntem 3 çalıştırıldı.
Subsystem3: Yöntem 2 çalıştırıldı.
--- İşlem Tamamlandı ---
*/
```

Aşağıda bu desene gerçek dünya örneği olarak alt sistemleri olan bir bilgisayarın başlatılması modellenmiştir;

```
// --- Sabit Tanımlamalar ---
public static class BootConstants
{
    public const long Address = 0x7C00;
    public const int Sector = 0;
    public const int SectorSize = 512;
}
// 1. --- Alt Sistemler (Subsystems) ---
public class Cpu
{
    public void Freeze() => Console.WriteLine("CPU: Durduruldu (Freeze).");
    public void Jump(long position) => Console.WriteLine($"CPU: Adrese atlandı:
    ↳ 0x{position:X}");
    public void Execute() => Console.WriteLine("CPU: Komutlar çalıştırılıyor...");
}
public class Memory
{
    public void Load(long position, string data) => Console.WriteLine($"Memory: 0x{position:X}
    ↳ adresine veri yüklendi: [{data}]");
}
public class HardDrive
{
    public string Read(long lba, int size)
    {
        Console.WriteLine($"HardDrive: Sector {lba} üzerinden {size} byte veri okundu.");
        return "OS_KERNEL_DATA";
    }
}
// 2. --- Facade (Önyüz) ---
public class ComputerFacade
{
    private readonly Cpu _cpu;
    private readonly Memory _memory;
    private readonly HardDrive _hardDrive;
    public ComputerFacade()
    {
        _cpu = new Cpu();
        _memory = new Memory();
        _hardDrive = new HardDrive();
    }
    public void Start()
    {
        Console.WriteLine("Bilgisayar başlatılıyor...");
        Console.WriteLine(new string('-', 35));
        // Facade karmaşık boot algoritmasını yönetir:
        _cpu.Freeze();
        string data = _hardDrive.Read(BootConstants.Sector, BootConstants.SectorSize);
        _memory.Load(BootConstants.Address, data);
        _cpu.Jump(BootConstants.Address);
        _cpu.Execute();
        Console.WriteLine(new string('-', 35));
        Console.WriteLine("Sistem hazır!");
    }
}
```

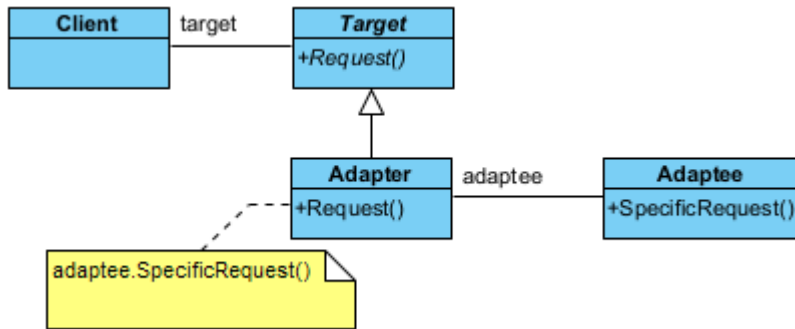
```

}
// 3. --- Client (İstemci) ---
class Program
{
    static void Main()
    {
        // İstemci alt sistemlerle (CPU, RAM, HDD) hiç ilgilenmez.
        var computer = new ComputerFacade();
        computer.Start();
    }
}
/*Program Çıktısı:
Bilgisayar başlatılıyor...
-----
CPU: Durduruldu (Freeze).
HardDrive: Sector 0 üzerinden 512 byte veri okundu.
Memory: 0x7C00 adresine veri yüklendi: [OS_KERNEL_DATA]
CPU: Adrese atlandı: 0x7C00
CPU: Komutlar çalıştırılıyor...
-----
Sistem hazır!
*/

```

§18.3.2 Adaptör

Adaptör deseni (**adapter pattern**), yabancı bir ara yüzü istemcinin anlayacağı bir ara yüze dönüştürür. Yani farklı ara yüzlere sahip sınıfların, iletişim ve etkileşim kurabilecekleri ortak bir nesne oluşturarak birlikte çalışmalarına izin verir. Bu desenin amacı, uyumsuz iki arayüzün (**interface**) birlikte çalışmasını sağlar. Burada



Şekil 18.7: Adaptör Deseni UML Diyagramı

yabancı ara yüze yaptırılacak işin bir şekilde yapıldığı ve zaten yapılan bu işlemin istemcinin (**Client**) isteğine uygun hale getirildiği düşünülmelidir. Bir nevi elektrik prizi dönüştürücüsü görevi görür.

- ◊ İstemci (**Client**), mevcut sistemle çalışan ana kod.
- ◊ Uyumsuz Nesne (**Adaptee**), sisteme dahil edilmek istenen ama arayüzü farklı olan nesne.
- ◊ Hedef (**Target**), istemcinin (**Client**) beklediği arayüz.
- ◊ Uyarlayıcı (**Adapter**), uyumsuz nesneyi sarmalayarak, istemcinin beklediği biçime çeviren aracı sınıf.

Şimdi de desenimizi kodlayalım;

```

// 1. Adaptee (Uyarlanan): Mevcut olan ama arayüzü uyumsuz olan sınıf.
// Bu sınıfın koduna dokunamadığımızı varsayıyoruz (Örn: Bir DLL içindedir).
public class Adaptee
{
    public void SpecificRequest()
    {
        Console.WriteLine("Adaptee: Özel istek (SpecificRequest) yöntemi çalıştırıldı.");
    }
}

// 2. Target (Hedef): İstemcinin beklediği standart arayüz.
public interface ITarget
{

```

```

    void Request();
}
// 3. Adapter (Uyarlayıcı): ITarget arayüzünü Adaptee'ye bağlar.
public class Adapter : ITarget
{
    private readonly Adaptee _adaptee;
    public Adapter()
    {
        _adaptee = new Adaptee();
    }
    public void Request()
    {
        Console.WriteLine("Adapter: Standart istek alındı, Adaptee'ye yönlendiriliyor...");
        // Adaptörün asıl görevi: Standart çağrıyı özel çağrıya dönüştürmek.
        _adaptee.SpecificRequest();
    }
}
// 4. Client (İstemci)
class Program
{
    static void Main(string[] args)
    {
        // İstemci ITarget arayüzünü bilir, arka planda Adaptee olduğunu bilmez.
        ITarget target = new Adapter();
        target.Request();
    }
}
/*Program Çıktısı:
Adapter: Standart istek alındı, Adaptee'ye yönlendiriliyor...
Adaptee: Özel istek (SpecificRequest) yöntemi çalıştırıldı.
*/

```

Bu desen gerçek dünya örneği olarak aşağıdaki iz kaydı (log) örneği verilebilir;

```

// 1. --- Adaptee (Uyarlanan Sınıf) ---
// Değiştiremediğimiz veya eski (Legacy) bir kütüphaneden gelen sınıf
public class OldLogger
{
    public void LogMessage(string msg)
    {
        Console.WriteLine($"[Eski Kayıt Sistemi]: {msg}");
    }
}
// 2. --- Target Interface (Hedef Arayüz) ---
// Yeni sistemin beklediği modern ve standart arayüz
public interface ILogger
{
    void Log(string msg);
}
// 3. --- Adapter (Adaptör Sınıfı) ---
public class LoggerAdapter : ILogger
{
    private readonly OldLogger _oldLogger;
    // C#'ta bağımlılıkları yapıcı metot üzerinden (Dependency Injection) almak en iyi pratiktir.
    public LoggerAdapter(OldLogger oldLogger)
    {
        _oldLogger = oldLogger;
    }
    public void Log(string msg)
    {
        // Adaptörün asıl görevi: Metot ismini ve gerekiyorsa parametre yapısını dönüştürmek
        _oldLogger.LogMessage(msg);
    }
}

```

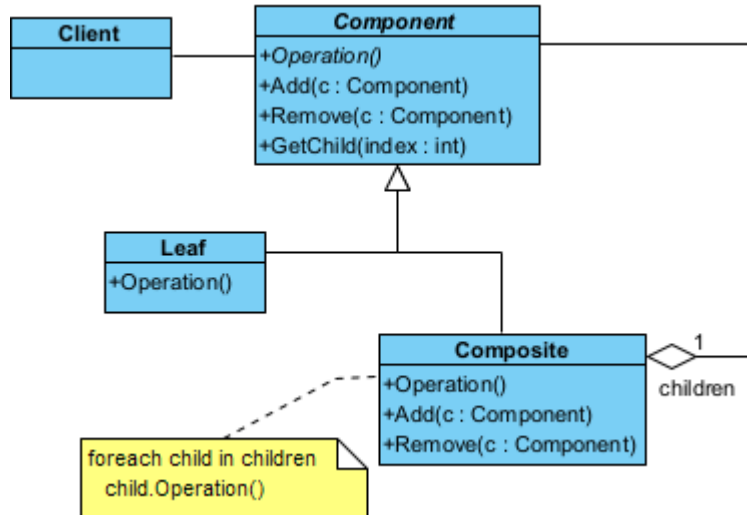
```

    }
}
// 4. --- Client (İstemci) ---
class Program
{
    // İstemci sadece ILogger arayüzünü tanır, arka plandaki implementasyondan bağımsızdır.
    static void AppLog(ILogger logger)
    {
        logger.Log("Sistem başarıyla çalıştı.");
    }
    static void Main()
    {
        Console.WriteLine("Modern uygulama log gönderiyor...");
        // 1. Eski sistemden bir nesne oluşturuyoruz.
        OldLogger oldSys = new OldLogger();
        // 2. Eski nesneyi adaptör içine "paketliyoruz".
        // C#'ta sahiplik devri (move) gerekmez, referans aktarılır.
        ILogger modernLogger = new LoggerAdapter(oldSys);
        // 3. Uygulama artık eski sistemi yeni arayüz üzerinden kullanabilir.
        AppLog(modernLogger);
    }
}
/*Program Çıktısı:
Modern uygulama log gönderiyor...
[Eski Kayıt Sistemi]: Sistem başarıyla çalıştı.
*/

```

§18.3.3 Bileşik

Bileşik deseni (composite pattern), özyinelemeli (**recursive**) ya da hiyerarşik yapıya sahip nesne oluşturmak gerektiğinde yapısal olarak nasıl bir kodlama yöntemi izleneceğini söyler. Bu desen, her nesnenin bağımsız olarak veya aynı ara yüz üzerinden iç içe geçmiş nesneler kümesi olarak ele alınabileceği nesne hiyerarşilerinin oluşturulmasını kolaylaştırır.



Şekil 18.8: Bileşik Deseni UML Diyagramı

Bu desenin görevi, nesneleri "parça-bütün" hiyerarşisinde (ağaç yapısı) düzenler. Tekil nesne ile nesne gruplarına aynı şekilde davranılmasını sağlar. Bir klasör örneği üzerinden ilerlersek;

- ◊ Bileşen (**Component**), hem dosyaların hem klasörlerin ortak özelliklerini tanımlayan arayüz.
- ◊ Yaprak (**Leaf**), En küçük birim (Örnek olarak Dosya).
- ◊ Bileşik (**Composite**), içinde başka birimler barındırabilen yapı (Örnek olarak Dosya içerebilen Klasör).

Şimdi de desenimizi kodlayalım;

```

using System.Collections.Generic;
using System.Linq;

```

```
// 1. --- Abstract Component ---
public abstract class Component
{
    public Component? Parent { get; set; }
    public virtual bool IsComposite() => false;
    // Varsayılan boş gövdeler (Yapraklarda hata vermemesi için)
    public virtual void AddChild(Component component) { }
    public virtual void RemoveChild(Component component) { }
    public abstract string DisplayOperation();
}

// 2. --- Leaf (Yaprak) ---
public class Leaf : Component
{
    public override string DisplayOperation() => "Yaprak";
}

// 3. --- Composite (Dal / Bileşen Grubu) ---
public class Composite : Component
{
    // Çocukların listesi. C#'ta bellek yönetimi otomatik olduğu için
    // pointer yönetimine gerek yoktur.
    private readonly List<Component> _children = new List<Component>();
    public override void AddChild(Component component)
    {
        component.Parent = this;
        _children.Add(component);
    }
    public override void RemoveChild(Component component)
    {
        _children.Remove(component);
        component.Parent = null;
    }
    public override bool IsComposite() => true;
    public override string DisplayOperation()
    {
        // LINQ kullanarak çocukların operasyonlarını birleştiriyoruz.
        var results = _children.Select(c => c.DisplayOperation());
        string combinedResult = string.Join("+", results);
        return $"Dal({combinedResult})";
    }
}

// 4. --- Client (İstemci) ---
class Program
{
    static void Main()
    {
        // 1. Sadece yaprak örneği
        var yaprak = new Leaf();
        Console.WriteLine("Sadece Yapraktan Oluşan Nesne: " + yaprak.DisplayOperation());
        // 2. Karmaşık ağaç yapısı
        var kok = new Composite();
        var dal1 = new Composite();
        dal1.AddChild(new Leaf()); // Yaprak 1
        var yaprak2 = new Leaf(); // İleride silmek istersek referansını tutarız
        dal1.AddChild(yaprak2);
        kok.AddChild(dal1);
        Console.WriteLine("Yapraktan ve Dallardan Oluşan Nesne: " + kok.DisplayOperation());
        // 3. Bir dal daha ekleme
        var dal2 = new Composite();
        dal2.AddChild(new Leaf());
        kok.AddChild(dal2);
        Console.WriteLine("Bir dal daha eklenmiş Nesne: " + kok.DisplayOperation());
    }
}
```



```

        // C#'ta 'delete' yoktur; 'kok' referansı bittiğinde tüm ağaç GC tarafından temizlenir.
    }
}
/*Program Çıktısı:
Sadece Yapraktan Oluşan Nesne: Yaprak
Yapraktan ve Dallardan Oluşan Nesne: Dal(Dal(Yaprak+Yaprak))
Bir dal daha eklenmiş Nesne: Dal(Dal(Yaprak+Yaprak)+Dal(Yaprak))
*/

```

Bu desene gerçek dünya örneği olarak aşağıdaki dosya sistemini gösteren örnek verilebilir;

```

using System;
using System.Collections.Generic;
using System.Linq;

// 1. --- Component (Bileşen) ---
public abstract class FileSystemNode
{
    public abstract void ShowDetails(int indent = 0);
    public abstract long GetSize();
}

// 2. --- Leaf (Yaprak) ---
public class File : FileSystemNode
{
    private string _name;
    private long _size;
    public File(string name, long size)
    {
        _name = name;
        _size = size;
    }
    public override void ShowDetails(int indent)
    {
        Console.WriteLine($"{new string(' ', indent)}- Dosya: {_name} ({_size} KB)");
    }
    public override long GetSize() => _size;
}

// 3. --- Composite (Bileşik) ---
public class Directory : FileSystemNode
{
    private string _name;
    private List<FileSystemNode> _children = new List<FileSystemNode>();
    public Directory(string name)
    {
        _name = name;
    }
    public void Add(FileSystemNode node)
    {
        _children.Add(node);
    }
    public override void ShowDetails(int indent=0)
    {
        // LINQ kullanarak toplam boyutu o anki çocuklardan hesaplıyoruz
        Console.WriteLine($"{new string(' ', indent)}+ Klasör: {_name} [Toplam: {GetSize()}
        ↳ KB]");
        foreach (var child in _children)
        {
            // Özyinelemeli (Recursive) çağrı
            child.ShowDetails(indent + 4);
        }
    }
    public override long GetSize()
    {

```

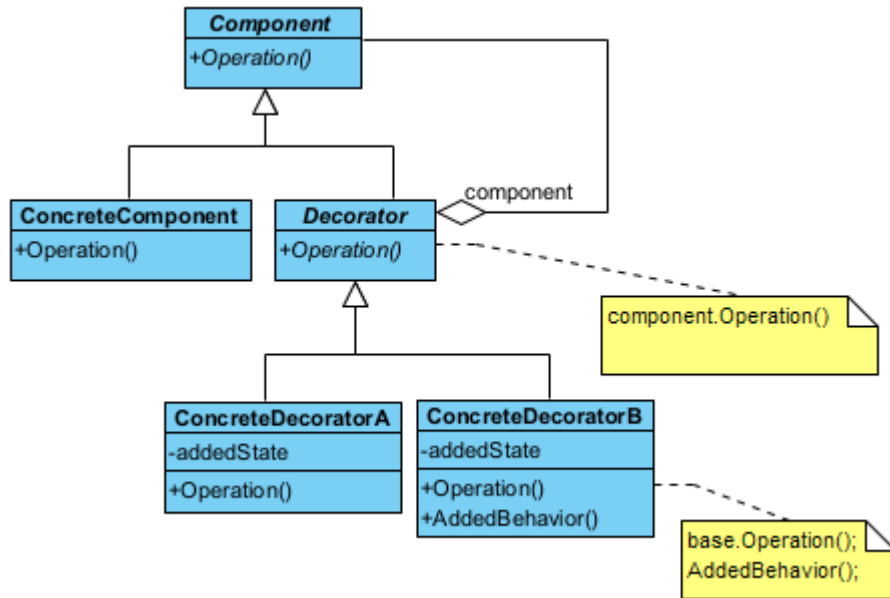
```

        // LINQ'nun Sum metodu ile çocukların boyutlarını topluyoruz
        return _children.Sum(child => child.GetSize());
    }
}
// 4. --- Client (İstemci) ---
class Program
{
    static void Main()
    {
        // Klasörleri oluşturalım
        var root = new Directory("C_Surucusu");
        var docs = new Directory("Belgelerim");
        // Dosyaları ekleyelim
        docs.Add(new File("Notlar.txt", 10));
        root.Add(docs);
        root.Add(new File("Sistem.log", 500));
        root.ShowDetails();
    }
}
/*Program Çıktısı:
+ Klasör: C_Surucusu [Toplam: 510 KB]
+ Klasör: Belgelerim [Toplam: 10 KB]
- Dosya: Notlar.txt (10 KB)
- Dosya: Sistem.log (500 KB)
*/

```

§18.3.4 Süsleyici

Süsleyici deseninde (**decorator pattern**), bir nesneye dinamik olarak yeni durum ve davranışları eklemek mümkündür. Yani çalıştırma anında, bir nesnenin sahip olduğu yeteneklere, yeni yeteneklerin eklenmesini sağlar.



Şekil 18.9: Süsleyici Deseni UML Diyagramı

Bu tasarım deseni, bir nesneye dinamik olarak (çalışma zamanında) yeni sorumluluklar veya özellikler eklemek için kullanılan yapısal bir desendir. Kalıtımın (**inheritance**) aksine, özellikleri sınıf hiyerarşisini karmaşıktırmadan, nesneleri birbirinin içine "sarmalayarak" (**wrapping**) ekler. Bunu bir Rus matruşka bebeği veya bir kahve siparişine eklenen ekstralar (süt, şeker, karamel) gibi düşünebilirsin.

- ◆ Bileşen (**Component**), hem temel nesnenin hem de süsleyicilerin uyması gereken ortak soyut arayüzü tanımlar. Temel işlevin (Örnek olarak `ucretHesapla()`) imzasını tanımlar.
- ◆ Somut Bileşen (**ConcreteComponent**), üzerine eklemeye yapılacak olan "temel" nesnedir. Fonksiyonun en yalın halini gerçekleştirir (Örnek olarak Sade Kahve).
- ◆ Soyut Süsleyici (**Decorator**), süsleyicilerin soyut temel yapısını kurar. İçinde bir **Component** referansı

tutar. Bu referans sayesinde hem temel nesneyi hem de başka bir süsleyiciyi sarmalayabilir.

- ◊ Somut Süsleyiciler (**ConcreteDecorator**), gerçek özellikleri ekleyen sınıflardır. Temel nesnenin metodunu çağırır ve üzerine kendi eklemesini yapar (Örnek olarak Kahve ücretine +5 TL süt bedeli eklemek).

Şimdi desenimizi kodlayalım;

```
using System;

// 1. Component (Bileşen): Ortak arayüz
public abstract class Component
{
    public abstract void Operation();
}

// 2. ConcreteComponent (Somut Bileşen): Temel nesne
public class ConcreteComponent : Component
{
    public override void Operation()
    {
        Console.WriteLine("Temel Ürün İşlemi");
    }
}

// 3. Decorator (Süsleyici): Sarmalayıcı temel sınıf
public abstract class Decorator : Component
{
    protected Component _component;
    protected Decorator(Component component)
    {
        _component = component;
    }
    public override void Operation()
    {
        // Eğer bir bileşen sarmalanmışsa onun işlemini çağır
        _component?.Operation();
    }
}

// 4. ConcreteDecorator1: Yeni Durum Ekleyen Süsleyici
public class ConcreteDecorator1 : Decorator
{
    private int _addedState;
    public ConcreteDecorator1(Component component, int state) : base(component)
    {
        _addedState = state * 100;
    }
    public override void Operation()
    {
        base.Operation(); // Önceki nesnenin (zincirin) davranışını çağır
        Console.WriteLine($"-> Decorator 1: Eklendi (State: {_addedState})");
    }
}

// 5. ConcreteDecorator2: Yeni Davranış Ekleyen Süsleyici
public class ConcreteDecorator2 : Decorator
{
    public ConcreteDecorator2(Component component) : base(component) { }
    private string AddedBehavior()
    {
        return "Bu metin Yeni Davranıştan Geliyor...";
    }
    public override void Operation()
    {
        base.Operation(); // Önceki nesnenin davranışını çağır
        Console.WriteLine($"-> Decorator 2: Yeni Davranış Tetiklendi: {AddedBehavior()}");
    }
}
```

```

}
// 6. Client (İstemci)
class Program
{
    static void Main()
    {
        Console.WriteLine("--- Dekoratör Zinciri Oluşturuluyor ---");
        // 1. Temel ürün oluştur
        Component myProduct = new ConcreteComponent();
        // 2. Ürünü Decorator 1 ile süsle
        // C#'ta 'new' kullanarak nesneleri iç içe geçirmek çok doğaldır.
        var decorated1 = new ConcreteDecorator1(myProduct, 2);
        // 3. Ortaya çıkan nesneyi Decorator 2 ile tekrar süsle
        var decorated2 = new ConcreteDecorator2(decorated1);
        // Tek bir çağrı tüm zinciri (içten dışa) tetikler
        decorated2.Operation();
        // C#'ta bellek yönetimi otomatik olduğu için manuel temizliğe gerek yoktur.
    }
}
/*Program Çıktısı:
--- Dekoratör Zinciri Oluşturuluyor ---
Temel Ürün İşlemi
-> Decorator 1: Eklendi (State: 200)
-> Decorator 2: Yeni Davranış Tetiklendi: Bu metin Yeni Davranıştan Geliyor...
*/

```

Aşağıda gerçek dünya örneği olarak sipariş edilen kahveyi süsleyen bir örnek verilmiştir;

```

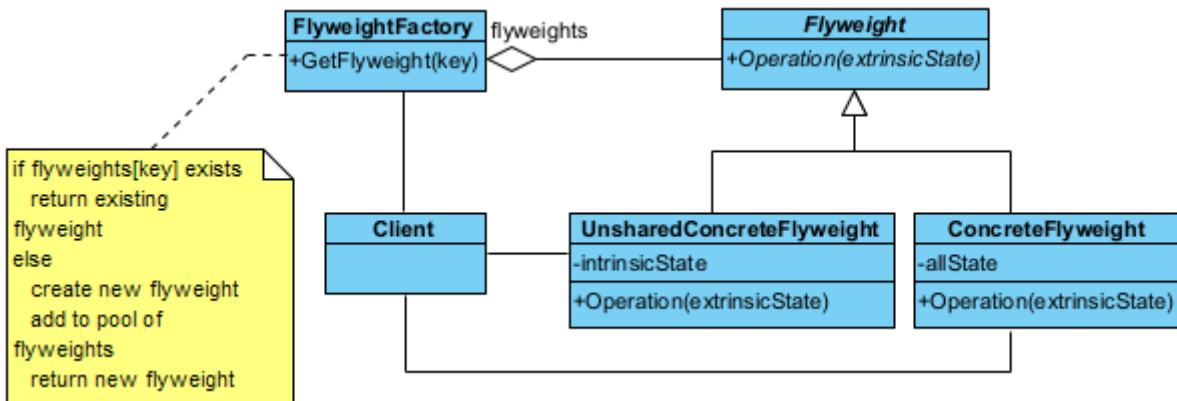
// 1. --- Base Component (Temel Bileşen) ---
public abstract class Coffee
{
    public abstract double GetCost();
    public abstract string GetDescription();
}
// 2. --- Concrete Component (Somut Bileşen) ---
public class SimpleCoffee : Coffee
{
    public override double GetCost() => 5.0;
    public override string GetDescription() => "Sade Kahve";
}
// 3. --- Base Decorator (Temel Süsleyici) ---
public abstract class CoffeeDecorator : Coffee
{
    protected Coffee _coffee;
    // C#'ta dekoratör sarmaladığı nesneyi referans olarak tutar
    protected CoffeeDecorator(Coffee coffee)
    {
        _coffee = coffee;
    }
}
// 4. --- Concrete Decorators (Somut Süsleyiciler) ---
public class MilkDecorator : CoffeeDecorator
{
    public MilkDecorator(Coffee coffee) : base(coffee) { }
    public override double GetCost() => _coffee.GetCost() + 1.5;
    public override string GetDescription() => _coffee.GetDescription() + ", süt";
}
public class SugarDecorator : CoffeeDecorator
{
    public SugarDecorator(Coffee coffee) : base(coffee) { }
    public override double GetCost() => _coffee.GetCost() + 0.5;
    public override string GetDescription() => _coffee.GetDescription() + ", şeker";
}

```

```
// 5. --- Client (İstemci) ---
class Program
{
    static void Main()
    {
        // 1. Sade bir kahve oluşturuyoruz
        Coffee kahvem = new SimpleCoffee();
        // 2. Kahveyi süt ile sarmalıyoruz
        kahvem = new MilkDecorator(kahvem);
        // 3. Mevcut (sütlü) kahveyi şeker ile sarmalıyoruz
        kahvem = new SugarDecorator(kahvem);
        // Sonuçları yazdıralım
        Console.WriteLine($"Sipariş: {kahvem.GetDescription()}");
        Console.WriteLine($"Toplam Fiyat: {kahvem.GetCost()} TL");
    }
}
/*Program Çıktısı:
Sipariş: Sade Kahve, süt, şeker
Toplam Fiyat: 7 TL
*/
```

§18.3.5 Sinek Sıklet

Sinek Sıklet deseninde (flyweight pattern), çok sayıda benzer nesnenin bulunduğu durumlarda bellek kullanımını minimize etmek için kullanılan yapısal (structural) bir desendir.



Şekil 18.10: Sinek Sıklet Deseni UML Diyagramı

Bu desenin temel stratejisi, nesnelerin ortak verilerini (intrinsic state) paylaşarak bellekte tek bir kopyasını tutmak, her nesneye özgü değişen verileri (extrinsic state) ise dışarıdan sağlamaktır.

- ◇ Soyut Sinek Sıklet (Flyweight), paylaşılan nesnelerin uyması gereken arayüzü tanımlar. Nesneye özgü verilerin (extrinsic state) parametre olarak geçildiği yöntemin imzasını barındırır.
- ◇ Somut Sinek Sıklet (ConcreteFlyweight), paylaşılan veriyi (intrinsic state) kendi içinde saklayan nesnedir. Bu nesneler "değişmez" (immutable) olmalıdır. Örneğin; bir oyun için "Ağaç" modelinin 3D verisi burada bir kez tutulur.
- ◇ Sinek Sıklet Fabrikası (FlyweightFactory), Flyweight nesnelerini yönetir ve paylaşılmasını sağlar. İstendiğinde, eğer nesne daha önce oluşturulmuşsa mevcut olanı verir; yoksa yenisini oluşturup havuza ekler.
- ◇ Bağlam (Context), her nesneye özgü, sürekli değişen verileri (konum, renk gibi) tutar. Somut ConcreteFlyweight nesnesini bu verilerle birlikte kullanarak asıl işlemi gerçekleştirir.

Şimdi desenimizi kodlayalım;

```
using System;
using System.Collections.Generic;
// 1. Flyweight Arayüzü
public abstract class Flyweight
{
    // İçsel Durum (Intrinsic State): Nesneler arasında ortak olan ve değişmeyen veri
    protected string IntrinsicState;
    protected Flyweight(string state)
```

```

    {
        IntrinsicState = state;
    }
    public abstract void Operation(string extrinsicState);
}
// 2. Somut Paylaşılan Flyweight (Concrete Flyweight)
public class ConcreteFlyweight : Flyweight
{
    public ConcreteFlyweight(string state) : base(state) { }
    public override void Operation(string extrinsicState)
    {
        // Dışsal Durum (Extrinsic State): İşlem anında dışarıdan gelen, değişken veri (konum,
        // ↪ renk vb.)
        Console.WriteLine($"[Paylaşılan] İç Durum: {IntrinsicState} | Dış Durum:
        // ↪ {extrinsicState}");
    }
}
// 3. Flyweight Factory (Fabrika)
public class FlyweightFactory
{
    private readonly Dictionary<string, Flyweight> _flyweights = new Dictionary<string,
    // ↪ Flyweight>();
    public Flyweight GetFlyweight(string key)
    {
        // Eğer bu anahtara sahip nesne zaten varsa onu döndür
        if (_flyweights.ContainsKey(key))
        {
            Console.WriteLine($"-> Fabrika: {key} zaten var, mevcut olan döndürülüyor.");
            return _flyweights[key];
        }
        // Yoksa yeni bir tane oluştur ve depola
        Console.WriteLine($"-> Fabrika: {key} bulunamadı, yeni oluşturuluyor.");
        var flyweight = new ConcreteFlyweight(key);
        _flyweights.Add(key, flyweight);
        return flyweight;
    }
    public void ListFlyweights()
    {
        Console.WriteLine($"\\nFabrikadaki toplam nesne sayısı: {_flyweights.Count}");
        foreach (var key in _flyweights.Keys)
        {
            Console.WriteLine($"- {key}");
        }
    }
}
// 4. Client (İstemci)
class Program
{
    static void Main(string[] args)
    {
        FlyweightFactory factory = new FlyweightFactory();
        // Aynı nesneleri tekrar tekrar isteyelim
        Flyweight v1 = factory.GetFlyweight("Binek Araç");
        Flyweight v2 = factory.GetFlyweight("Kamyon");
        Flyweight v3 = factory.GetFlyweight("Binek Araç"); // Fabrikadan mevcut olan gelecek
        // Dışsal durumları (konum gibi) işlem sırasında gönderiyoruz
        v1.Operation("Konum: 41.00, 28.97");
        v2.Operation("Konum: 39.93, 32.85");
        v3.Operation("Konum: 40.71, -74.00");
        factory.ListFlyweights();
    }
}

```

```

/* Program Çıktısı:
-> Fabrika: Binek Araç bulunamadı, yeni oluşturuluyor.
-> Fabrika: Kamyon bulunamadı, yeni oluşturuluyor.
-> Fabrika: Binek Araç zaten var, mevcut olan döndürülüyor.
[Paylaşılan] İç Durum: Binek Araç | Dış Durum: Konum: 41.00, 28.97
[Paylaşılan] İç Durum: Kamyon | Dış Durum: Konum: 39.93, 32.85
[Paylaşılan] İç Durum: Binek Araç | Dış Durum: Konum: 40.71, -74.00

```

Fabrikadaki toplam nesne sayısı: 2

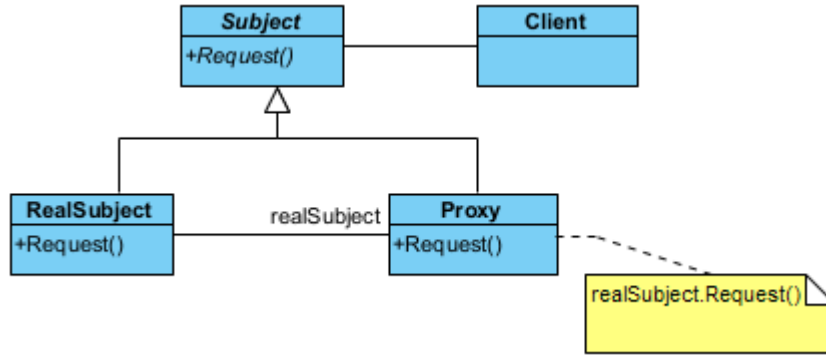
```

- Binek Araç
- Kamyon
*/

```

§18.3.6 Vekil

Vekil deseni (proxy pattern), kullanılacak ve erişilecek bir nesnenin yerine geçen bir başka nesneye ihtiyaç duyulduğunda kullanılır. Bu desen, başka bir nesneye erişimi kontrol etmek için kullanılan yapısal bir desendir. Bir nesnenin önüne "vekil" bir katman koyarak, asıl nesneye erişmeden önce güvenlik kontrolü yapılabilir, nesnenin yüklenmesini geciktirebilir (**lazy loading**) veya işlem sonuçlarını önbelleğe alabilirsiniz.



Şekil 18.11: Vekil Deseni UML Diyagramı

Bu deseni bir şirketteki "Yönetici Asistanı" gibi düşünebilirsin. Yöneticiyle doğrudan görüşemezsin; önce asistanla (**Proxy**) görüşürsün, asistan uygun görürse seni yöneticiye (**RealSubject**) yönlendirir.

- ◊ Bağlam (**Subject**), hem asıl nesnenin hem de vekilin (**Proxy**) uyması gereken ortak soyut arayüzü tanımlar. İstemcinin (**Client**) her iki nesneye de aynı şekilde davranabilmesini sağlar.
- ◊ Gerçek Bağlam (**RealSubject**), asıl işi yapan, asıl mantığı barındıran nesnedir. Genellikle oluşturulması maliyetli veya erişimi kısıtlı olan nesnedir.
- ◊ Vekil (**Proxy**), gerçek nesneye olan referansı kendi içinde saklar. İstekleri gerçek nesneye iletmeden önce araya girer. Erişim kontrolü yapabilir, gerçek nesne henüz oluşturulmamışsa onu oluşturabilir veya gelen isteği reddedebilir.
- ◊ İstemci (**Client**), nesnenin doğrudan kendisine değil, vekil (**Proxy**) nesnesine ulaşır ve bu nesneye isteklerini iletir.

Şimdi desenimizi kodlayalım;

```

using System;
// 1. Subject (Özne Arayüzü): Gerçek nesne ve vekilin ortak kontratı
public interface ISubject
{
    void Request();
}
// 2. RealSubject (Gerçek Özne): Asıl iş mantığını barındıran sınıf
public class RealSubject : ISubject
{
    private readonly int _state;
    public RealSubject(int state)
    {
        _state = state;
    }
    public void Request()
    {

```

```

        Console.WriteLine($"RealSubject: İstek işleniyor (Durum: {_state})");
    }
}
// 3. Proxy (Vekil): Gerçek nesneye erişimi denetleyen aracı
public class Proxy : ISubject
{
    private RealSubject? _realSubject;
    private readonly int _defaultState;
    // Senaryo 1: Tembel Yükleme (Lazy Initialization) için varsayılan yapıcı
    public Proxy()
    {
        _defaultState = 20;
    }
    // Senaryo 2: Mevcut bir nesne üzerinden vekillik yapmak
    public Proxy(RealSubject realSubject)
    {
        _realSubject = realSubject;
    }
    private bool CheckAccess()
    {
        Console.WriteLine("Proxy: Erişim yetkisi kontrol ediliyor...");
        return true;
    }
    private void LogAccess()
    {
        Console.WriteLine("Proxy: İşlem günlüğü (log) kaydedildi.");
    }
    public void Request()
    {
        if (CheckAccess())
        {
            // Nesne henüz oluşturulmamışsa (Lazy Initialization) burada oluşturulur
            if (_realSubject == null)
            {
                _realSubject = new RealSubject(_defaultState);
            }
            _realSubject.Request();
            LogAccess();
        }
    }
}
// 4. Client (İstemci)
class Program
{
    static void Main()
    {
        // 1. Doğrudan erişim
        Console.WriteLine("--- Doğrudan RealSubject Kullanımı ---");
        ISubject gercekOzne = new RealSubject(10);
        gercekOzne.Request();
        // 2. Kendi nesnesini yöneten vekil (Lazy Proxy)
        Console.WriteLine("\n--- Proxy (Yeni Nesne İle / Lazy) ---");
        ISubject vekil1 = new Proxy();
        vekil1.Request();
        // 3. Mevcut nesne üzerinden vekillik yapan vekil
        Console.WriteLine("\n--- Proxy (Mevcut Nesne Referansı İle) ---");
        ISubject vekil2 = new Proxy((RealSubject)gercekOzne);
        vekil2.Request();
    }
}
/* Program Çıktısı:

```



```

--- Doğrudan RealSubject Kullanımı ---
RealSubject: İstek işleniyor (Durum: 10)

--- Proxy (Yeni Nesne İle / Lazy) ---
Proxy: Erişim yetkisi kontrol ediliyor...
RealSubject: İstek işleniyor (Durum: 20)
Proxy: İşlem günlüğü (log) kaydedildi.

--- Proxy (Mevcut Nesne Referansı İle) ---
Proxy: Erişim yetkisi kontrol ediliyor...
RealSubject: İstek işleniyor (Durum: 10)
Proxy: İşlem günlüğü (log) kaydedildi.
*/

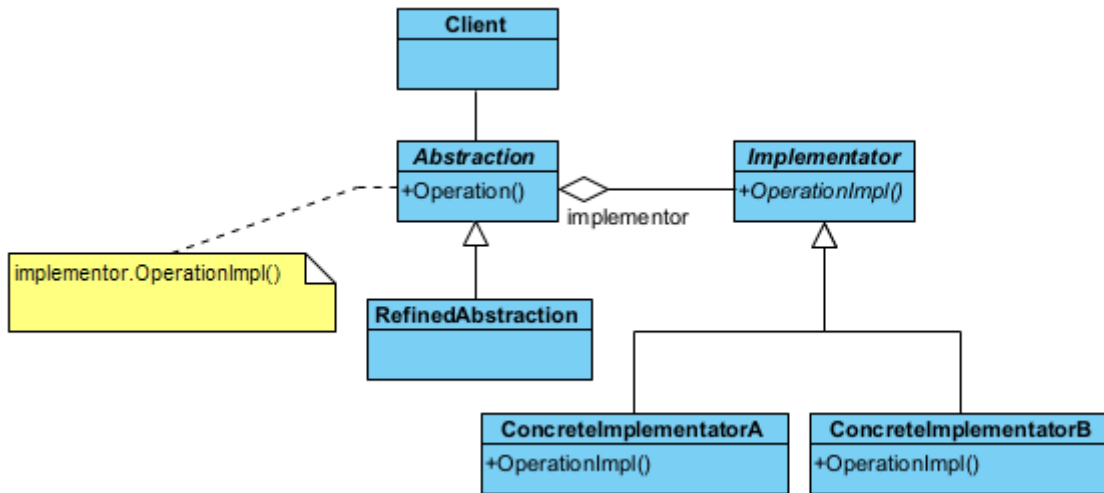
```

Bu desenle ilgili şu kavramlardan bahsetmek gerekir;

- ◊ Bir nesne, bulunduğu yerden uzaktaki bir nesneyi kullanacaksa uzak vekil (**remote proxy**) kullanılır. Gerçek nesne çok ağırsa (örneğin büyük bir resim), nesneyi sadece gerçekten ihtiyaç duyulduğunda (**lazy loading**) oluşturur.
- ◊ Bir nesneye başka nesneler tarafından erişim kısıtlanacaksa koruma vekili (**protection proxy**) olarak adlandırılır. Yetkilendirme (**authentication**) kontrolü yapar.
- ◊ Bir nesneye erişim sırasında başka işlemler yapılacaksa vekil, akıllı referans (**smart reference**) olarak adlandırılır. Böyle durumda vekil, nesneye bir erişim yapıldığında, diğer nesnelerin erişmemesi için kullanılacak nesneye kilit koyabilir. Akıllı referans, akıllı gösterici (**smart pointer**) olarak da adlandırılır. Nesne başka bir sunucudaysa ağ trafiğini yönetir.

§18.3.7 Köprü

Köprü deseni (bridge pattern), işi yapan alt yüklenici veya taşeron (**implementator**) nesne ile işi yaptıran nesne (**client**) arasındaki bağımlılığı soyutlama ile ortadan kaldırır. Yani birbiriyle oldukça iç içe olan kodlama ile soyutlamayı ortak ara yüz ile birbirinden uzaklaştırır. Sistemin genişlemesine izin verir. Bu desen, yazılımlarda oldukça çok kullanılan bir desendir. Çünkü taşeronlar değişse de yeni taşeron eklense de istemci tarafında bir şey değişmez.



Şekil 18.12: Köprü Deseni UML Diyagramı

Bu tasarım deseni, bir nesnenin soyutlamasını (**abstraction**), onun uygulamasından (**implementation**) ayırarak her ikisinin de birbirinden bağımsız olarak geliştirilmesini sağlayan yapısal bir desendir. Bu desen, özellikle **sınıf patlaması (class explosion)** dediğimiz, her yeni özellik için onlarca alt sınıf türetmek zorunda kaldığımız durumlarda imdadımıza yetişir. Kalıtım yerine bileşimi (**composition**) kullanır.

- ◊ Soyutlama (**Abstraction**), istemcinin (**Client**) kullandığı üst düzey kontrol arayüzüdür. İçinde bir **Implementor** referansı tutar. İstemciden gelen komutları bu referansa iletir.
- ◊ Geliştirilmiş Soyutlama (**RefinedAbstraction**),soyutlamayı genişleten alt sınıflardır. Temel arayüzü kullanarak daha spesifik kontrol mantıkları ekler (Örn: Standart Kumanda→Gelişmiş Akıllı Kumanda).
- ◊ Taşeron Arayüzü (**Implementor**), tüm somut uygulama sınıfları için ortak arayüzü tanımlar. Genellikle düşük seviyeli (ilkel) operasyonları barındırır (Örn: **cihazAc()**, **sesAyari()**).
- ◊ Somut Taşeronlar (**ConcreteImplementor**), **Implementor** arayüzünü gerçekleyen asıl cihazlardır. Plat-forma veya cihaza özgü gerçek kodları barındırır (Örn: Televizyon, Radyo).

Şimdi desenimizi kodlayalım;

```
using System;
// 1. --- Implementor (Uygulayıcı / Taşeron Arayüzü) ---
public interface IImplementor
{
    void OperationImpl();
}
// 2. --- Concrete Implementors ---
public class ConcreteImplementorA : IImplementor
{
    public void OperationImpl()
    {
        Console.WriteLine("A Taşeronu tarafından yapılan iş...");
    }
}
public class ConcreteImplementorB : IImplementor
{
    public void OperationImpl()
    {
        Console.WriteLine("B Taşeronu tarafından yapılan iş...");
    }
}
// 3. --- Abstraction (Soyut Yüklenici) ---
public abstract class Abstraction
{
    protected IImplementor _implementor;
    protected Abstraction(IImplementor implementor)
    {
        _implementor = implementor;
    }
    // Çalışma anında taşeronu değiştirme (Property veya Method ile yapılabilir)
    public void SetImplementor(IImplementor implementor)
    {
        _implementor = implementor;
    }
    public virtual void Operation()
    {
        if (_implementor != null)
        {
            Console.WriteLine("Yüklenici Taşerona iş yaptırıyor:");
            _implementor.OperationImpl();
        }
    }
}
// 4. --- Refined Abstraction (Geliştirilmiş Soyutlama) ---
public class RefinedAbstraction : Abstraction
{
    public RefinedAbstraction(IImplementor implementor) : base(implementor) { }
}
// 5. --- Client (İstemci) ---
class Program
{
    static void Main()
    {
        // 1. Yükleniciyi ilk taşeronla (A) başlatıyoruz
        var yuklenici = new RefinedAbstraction(new ConcreteImplementorA());
        yuklenici.Operation();
        Console.WriteLine("--- Taşeron Değiştiriliyor ---");
        // 2. Çalışma anında yeni bir taşerona (B) geçiyoruz
        yuklenici.SetImplementor(new ConcreteImplementorB());
        yuklenici.Operation();
    }
}
```

```

}
/* Program Çıktısı:
Yüklenici Taşeronu iş yaptırıyor:
A Taşeronu tarafından yapılan iş...
--- Taşeron Değiştiriliyor ---
Yüklenici Taşeronu iş yaptırıyor:
B Taşeronu tarafından yapılan iş...
*/

```

Köprü deseni aşağıdaki durumlarda kullanılır;

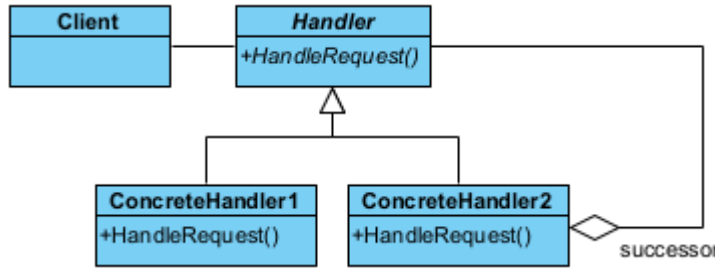
- ◊ Taşeron nesneye kalıcı bir bağımlılık istenmediği durumlar. Çalışma anında (**run-time**) taşeronlar arasında geçiş yapmayı sağlar.
- ◊ Birden çok taşeron kullanarak sistemin genişlemesine ihtiyaç olduğu durumlar.
- ◊ Taşeronlar üzerindeki değişiklikler, istemciyi etkilemez. İstemci tarafında derleme olmadan yazılımın değiştirilmesi sağlanır.
- ◊ Projede aynı anda farklı birden çok taşeron tasarımı yapılabildiğinden, proje parçalara ayrılarak kendi içinde hızla sınıf tasarımı yapılabilir. Bu durum *Rumbaugh* tarafından iç içe genelleştirme (**nested generalization**) olarak adlandırılmıştır.

18.4 Davranışla İlgili Desenler

Davranışla ilgili desenler (**behavioral patterns**) nesnelerin çeşitli olaylar karşısında gösterecekleri davranışlara çözüm getirmektedirler.

§18.4.1 Sorumluluk Zinciri

Bazı durumlarda bir nesne, kendinin yapmayacağı bir işlemi halefine (**successor**) devredebilir. Ya da nesnenin kendi sorumluluğunu aşan durumlarda ilgili diğer nesneye işlem yaptırılır. İşte bu durumda **sorumluluk zinciri deseni** (**chain of responsibility pattern**) kullanılır.



Şekil 18.13: Sorumluluk Zinciri Deseni UML Diyagramı

Bu desenin temel amacı, isteği gönderen ile isteği işleyen arasındaki bağı gevşetmektir (**decoupling**). İstek, zincir boyunca ilerler ve her bir işleyici ya isteği çözer ya da bir sonraki işleyiciye paslar.

- ◊ İşleyici (**Handler**), İsteklerin nasıl işleneceğine dair ortak bir arayüz tanımlar. İşleyici, bir sonraki işleyiciyi tutan bir referans barındırır ve zincirin halkalarını birbirine bağlar.
- ◊ Somut İşleyiciler (**ConcreteHandler**), kendisine gelen isteği kontrol eder. Eğer istek kendi sorumluluk alanındaysa işi bitirir. Eğer değilse (veya işin bir kısmını yapıp devam etmesi gerekiyorsa), isteği zincirdeki bir sonraki halkaya iletir.
- ◊ İstemci (**Client**), zinciri oluşturur (halkaları birbirine bağlar) ve ilk isteği zincirin başına gönderir. İsteğin kim tarafından, nasıl çözüleceğiyle ilgilenmez; sadece "çözülmesini" bekler.

Şimdi desenimizi kodlayalım;

```

using System;
// 1. Soyut İşleyici (Handler)
public abstract class Handler
{
    // C#'ta nesne ömrü referans bazlıdır, unique_ptr yerine basit bir alan yeterlidir.
    protected Handler? Successor;
    public void SetSuccessor(Handler next)
    {
        Successor = next;
    }
    public abstract void HandleRequest(int request);
}

```

```
// 2. Somut İşleyici 1: Küçük Talepler (Örn: Memur)
public class Clerk : Handler
{
    public override void HandleRequest(int request)
    {
        if (request < 100)
        {
            Console.WriteLine($"Memur: {request} birimlik talebi onayladı.");
        }
        else if (Successor != null)
        {
            Console.WriteLine("Memur: Talep limitimi aşıyor, Şefe devrediliyor...");
            Successor.HandleRequest(request);
        }
        else
        {
            Console.WriteLine("Hata: Talebi karşılayacak kimse bulunamadı!");
        }
    }
}

// 3. Somut İşleyici 2: Büyük Talepler (Örn: Şef)
public class Manager : Handler
{
    public override void HandleRequest(int request)
    {
        if (request >= 100 && request < 500)
        {
            Console.WriteLine($"Şef: {request} birimlik talebi onayladı.");
        }
        else if (Successor != null)
        {
            Console.WriteLine("Şef: Talep limitimi aşıyor, Müdüre devrediliyor...");
            Successor.HandleRequest(request);
        }
        else
        {
            Console.WriteLine("Şef: Bu kadar büyük bir yetkim yok ve başka amir tanımlı değil!");
        }
    }
}

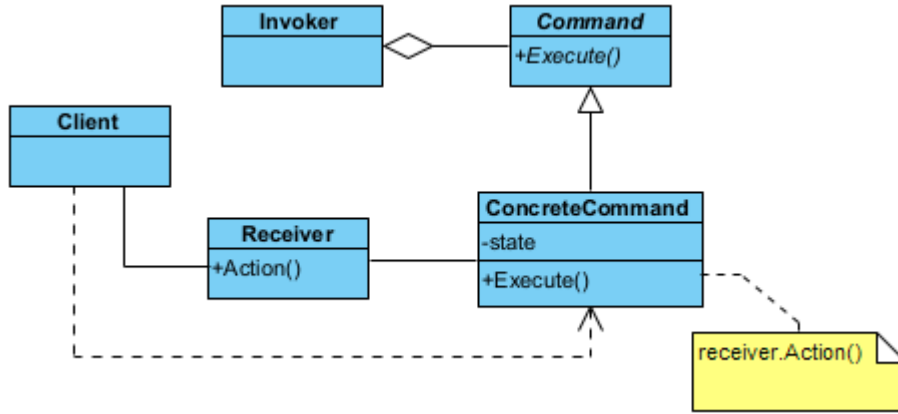
// 4. Client (İstemci)
class Program
{
    static void Main()
    {
        // Zinciri oluşturuyoruz
        var clerk = new Clerk();
        var manager = new Manager();
        // Memurun halefi Şef olsun
        clerk.SetSuccessor(manager);
        Console.WriteLine("--- Talep 1 (50) ---");
        clerk.HandleRequest(50);
        Console.WriteLine("--- Talep 2 (250) ---");
        clerk.HandleRequest(250);
        Console.WriteLine("--- Talep 3 (1000) ---");
        clerk.HandleRequest(1000);
    }
}

/* Program Çıktısı:
--- Talep 1 (50) ---
Memur: 50 birimlik talebi onayladı.
--- Talep 2 (250) ---
```

Memur: Talep limitimi aşıyor, Şefe devrediliyor...
 Şef: 250 birimlik talebi onayladı.
 --- Talep 3 (1000) ---
 Memur: Talep limitimi aşıyor, Şefe devrediliyor...
 Şef: Bu kadar büyük bir yetkim yok ve başka amir tanımlı değil!
 */

§18.4.2 Komut

Bazı durumlarda bir nesne diğer bir nesneye ileti (**message**) verip bir işlem başlattığında, bu işlem bitmeden başka işlemler yapmak isteyebilir. İşte bu tür durumlarda **komut deseni** (**command pattern**) kullanılır.



Şekil 18.14: Komut Deseni UML Diyagramı

Bu tasarım deseni, bir isteği veya eylemi kendi başına bir nesneye dönüştüren davranışsal bir desendir. Bu dönüşüm sayesinde eylemleri parametre olarak geçebilir, kuyruğa alabilir (**queue**), iz kaydını tutabilir (**log**) veya geri alabilirsiniz (**undo**). Kısacası; "ne yapılacağı" bilgisini, "kimin yapacağı" bilgisinden tamamen ayırır.

- ◊ Komut (**Command**), tüm komutların uyması gereken soyut arayüzü tanımlar. Genellikle tek bir yöntem barındırır (Örnek olarak **execute()**). Bazı durumlarda geri alma işlemi için **undo()** yöntemini de içerir.
- ◊ Somut Komut (**ConcreteCommand**), soyut komutu gerçek bir eylemle birleştirir. Alıcı (**Receiver**) nesnesini tanıır ve **execute()** çağrıldığında alıcının ilgili metodunu tetikler.
- ◊ Alıcı(**Receiver**), işin asıl mutfağıdır. Gerçek mantığı ve operasyonları barındırır. Komut nesnesi ona "çalış" dediğinde, o işi nasıl yapacağını bilir (Örnek olarak Işığın açmak, dosyayı kaydetmek).
- ◊ Çağırıcı(**Invoker**), komutun ne zaman çalışacağına karar verir. Komutu saklar ve kullanıcı bir düğmeye bastığında veya bir olay gerçekleştiğinde komutun **execute()** metodunu çağırır. Alıcıyı doğrudan tanımaz.

Şimdi desenimizi kodlayalım;

```

using System;
using System.Collections.Generic;
// 1. Receiver (Alıcı): Asıl iş mantığını barındıran sınıf
public class Receiver
{
    public void Action(string param) => Console.WriteLine($"Receiver: [{param}] komutu için temel
        ↳ işlemler yapılıyor.");
    public void ActionA(string param) => Console.WriteLine($"Receiver: [{param}] için A
        ↳ işlemi.");
    public void ActionB(string param) => Console.WriteLine($"Receiver: [{param}] için B
        ↳ işlemi.");
}
// 2. Command (Komut Arayüzü)
public interface ICommand
{
    void Execute();
}
// 3. Concrete Commands (Somut Komutlar)
public class SimpleCommand : ICommand
{
    private readonly string _state;

```

```

    private readonly Receiver _receiver;
    public SimpleCommand(string state, Receiver receiver)
    {
        _state = state;
        _receiver = receiver;
    }
    public void Execute()
    {
        _receiver.Action(_state);
    }
}
public class ComplexCommand : ICommand
{
    private readonly string _state;
    private readonly Receiver _receiver;
    public ComplexCommand(string state, Receiver receiver)
    {
        _state = state;
        _receiver = receiver;
    }
    public void Execute()
    {
        _receiver.ActionA(_state);
        _receiver.ActionB(_state);
    }
}
// 4. Invoker (Çağırıcı): Komutları saklar ve tetikler
public class Invoker
{
    private readonly List<ICommand> _history = new List<ICommand>();
    public void StoreAndExecute(ICommand cmd)
    {
        cmd.Execute();
        _history.Add(cmd); // Komutu sakla (Geri alma işlemleri için kullanılabilir)
    }
}
// 5. Client (İstemci)
class Program
{
    static void Main()
    {
        // Alıcı ve Çağırıcı nesneler oluşturulur
        Receiver receiver = new Receiver();
        Invoker invoker = new Invoker();

        Console.WriteLine("--- Komutlar Tetikleniyor ---");

        // Komutlar oluşturulur ve Invoker aracılığıyla çalıştırılır
        invoker.StoreAndExecute(new SimpleCommand("Yazdır", receiver));
        invoker.StoreAndExecute(new ComplexCommand("Logla ve Gönder", receiver));
    }
}
/* Program Çıktısı:
--- Komutlar Tetikleniyor ---
Receiver: [Yazdır] komutu için temel işlemler yapılıyor.
Receiver: [Logla ve Gönder] için A işlemi.
Receiver: [Logla ve Gönder] için B işlemi.
*/

```

Aşağıda komut desenine uygun bir ışığın kapatılıp açılması örneği verilmiştir;

```

using System;
using System.Collections.Generic;

```

```
// --- Receiver (Alıcı) ---
public class Light
{
    public void On() => Console.WriteLine("Işık AÇILDI.");
    public void Off() => Console.WriteLine("Işık KAPATILDI.");
}

// --- Command (Soyut Komut) ---
public interface ICommand
{
    void Execute();
    void Undo();
}

// --- Concrete Commands (Somut Komutlar) ---
public class LightOnCommand : ICommand
{
    private readonly Light _light;
    public LightOnCommand(Light light)
    {
        _light = light;
    }
    public void Execute() => _light.On();
    public void Undo() => _light.Off();
}

public class LightOffCommand : ICommand
{
    private readonly Light _light;
    public LightOffCommand(Light light)
    {
        _light = light;
    }
    public void Execute() => _light.Off();
    public void Undo() => _light.On();
}

// --- Invoker (Çağırıcı) ---
public class RemoteControl
{
    private readonly Stack<ICommand> _history = new Stack<ICommand>();
    public void ExecuteCommand(ICommand cmd)
    {
        cmd.Execute();
        _history.Push(cmd);
    }
    public void UndoLast()
    {
        if (_history.Count > 0)
        {
            Console.Write("[UNDO] ");
            // Pop() metodu hem en üstteki elemanı döndürür hem de yığından çıkarır.
            ICommand lastCommand = _history.Pop();
            lastCommand.Undo();
        }
        else
        {
            Console.WriteLine("[UYARI] Geri alınacak işlem yok!");
        }
    }
}

// --- Client (İstemci) ---
class Program
{
    static void Main()
    {

```

```

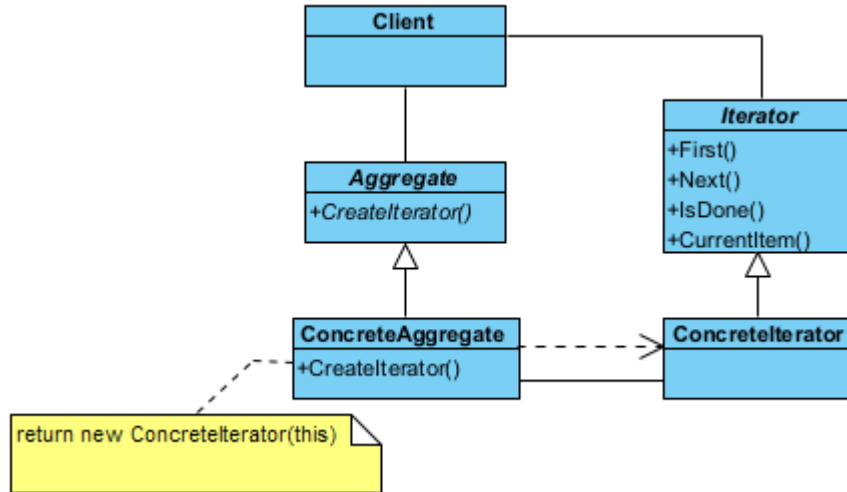
Light oturmaOdasiIsigi = new Light();
RemoteControl kumanda = new RemoteControl();
Console.WriteLine("--- İşlemler Başlıyor ---");
// Işığın açalım
kumanda.ExecuteCommand(new LightOnCommand(oturmaOdasiIsigi));
// Işığın kapatalım
kumanda.ExecuteCommand(new LightOffCommand(oturmaOdasiIsigi));
Console.WriteLine("\n--- Geri Alma (Undo) İşlemleri ---");
kumanda.UndoLast(); // Kapatmayı geri alır -> Işığın Açar
kumanda.UndoLast(); // Açmayı geri alır -> Işığın Kapatır
kumanda.UndoLast(); // Liste boş uyarısı verir
}
}
/* Program Çıktısı:
--- İşlemler Başlıyor ---
Işık AÇILDI.
Işık KAPATILDI.

--- Geri Alma (Undo) İşlemleri ---
[UNDO] Işık AÇILDI.
[UNDO] Işık KAPATILDI.
[UYARI] Geri alınacak işlem yok!
*/

```

§18.4.3 Yineleyici

Yineleyici deseninde (iterator pattern), birden fazla elemandan oluşan küme (**aggregate**) içindeki her bir elemana sırayla erişim (iteration) sağlar. Erişim işleminde, istemcinin kümenin nasıl yapılandırıldığı konusunda bilgi sahibi olması gerekmez. Bu desende, istemci tarafından veri kümesi soyut (**abstract**) olarak kullanılmış olur. Yani istemciye, veri yapısından bağımsız bir şekilde verilere erişim sağlayan bir ara yüz (**interface**) sunulur. Bu tasarım deseni, bir veri grubunun (liste, ağaç, yığın vb.) iç yapısını dışarıya açmadan, o



Şekil 18.15: Yineleyici Deseni UML Diyagramı

grubun elemanları üzerinde sırayla gezinebilmemizi sağlayan davranışsal bir desendir. Bu desen sayesinde, verilerin bellekte nasıl tutulduğunu (bir dizi mi yoksa birbirine bağlı halkalar mı?) bilmenize gerek kalmadan standart bir şekilde tüm elemanları dolaşabilirsiniz.

- ◇ Yineleyici (**Iterator**), elemanlar üzerinde gezinmek için gerekli olan standart arayüzü tanımlar. Genellikle `next()` (sonraki eleman), `hasNext()` (başka eleman var mı?) ve `current()` (mevcut eleman) gibi yöntemleri barındırır.
- ◇ Somut Yineleyici (**ConcreteIterator**), belirli bir veri yapısı (örneğin bir Vektör veya Liste) için gezinme mantığını uygular. O an hangi elemanda olduğunu (indis veya işaretçi bazlı) takip eder.
- ◇ Soyut Koleksiyon (**Aggregate**), koleksiyonun (**container**) kendisi için ortak bir arayüz sağlar. Kendi elemanlarını gezmek için kullanılacak olan uygun **Iterator** nesnesini oluşturan `createIterator()` adlı bir yöntemle sahiptir.
- ◇ Somut Koleksiyon (**ConcreteAggregate**), verileri asıl saklayan sınıftır. İstendiğinde, kendi iç yapısını en

verimli şekilde gezecek olan somut yineleyiciyi (`ConcreteIterator`) geri döndürür.

Şimdi desenimizi kodlayalım;

```
using System;
using System.Collections.Generic;
// 1. Soyut Iterator Arayüzü
public interface IIterator
{
    double First();
    double Next();
    bool HasNext();
    double CurrentItem();
}
// 2. Soyut Koleksiyon (Aggregate) Arayüzü
public interface IAggregate
{
    void AddItem(double item);
    int Count { get; }
    double GetItem(int index);
    IIterator CreateIterator();
}
// 3. Somut Iterator (Concrete Iterator)
public class ConcreteIterator : IIterator
{
    private readonly IAggregate _aggregate;
    private int _current = 0;
    public ConcreteIterator(IAggregate aggregate)
    {
        _aggregate = aggregate;
    }
    public double First()
    {
        _current = 0;
        return CurrentItem();
    }
    public double Next()
    {
        if (HasNext())
        {
            return _aggregate.GetItem(_current++);
        }
        return -1.0;
    }
    public bool HasNext() => _current < _aggregate.Count;
    public double CurrentItem() => _aggregate.GetItem(_current);
}
// 4. Somut Koleksiyon (Concrete Aggregate)
public class ConcreteAggregate : IAggregate
{
    private readonly List<double> _items = new List<double>();
    public void AddItem(double item) => _items.Add(item);
    public int Count => _items.Count;
    public double GetItem(int index) => _items[index];
    public IIterator CreateIterator()
    {
        return new ConcreteIterator(this);
    }
}
// 5. Client (İstemci)
class Program
{
```

```

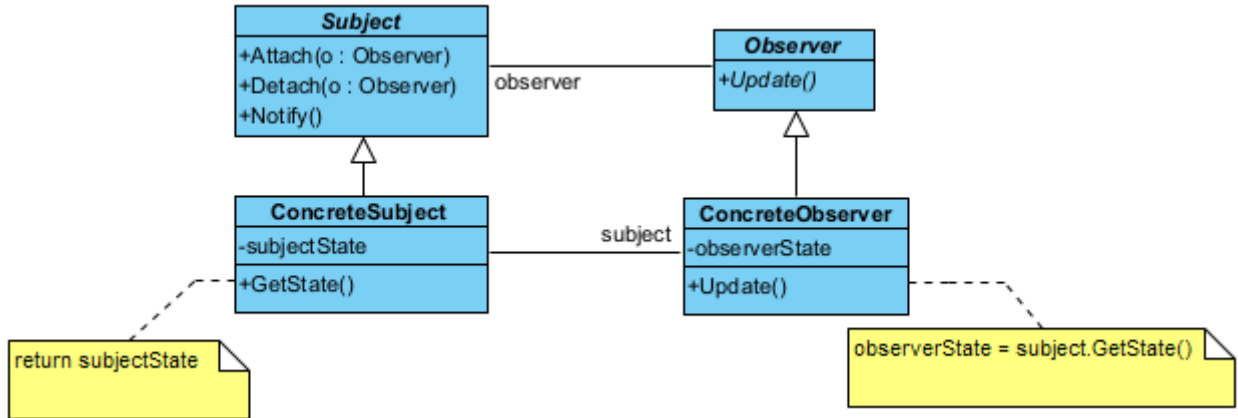
static void Main()
{
    var aggregate = new ConcreteAggregate();
    aggregate.AddItem(10.5);
    aggregate.AddItem(20.7);
    aggregate.AddItem(30.9);
    var iterator = aggregate.CreateIterator();
    Console.WriteLine("Koleksiyon elemanları geziliyor:");
    // Standart Iterator döngü yapısı
    for (iterator.First(); iterator.HasNext(); )
    {
        Console.WriteLine($"Değer: {iterator.Next()}");
    }
}
}
/* Program Çıktısı:
Koleksiyon elemanları geziliyor:
Değer: 10.5
Değer: 20.7
Değer: 30.9
*/

```

C# dünyasında yineleyici deseni o kadar temeldir ki, dilin içine `IEnumerable` ve `IEnumerator` arayüzlerine gömülmüştür.

§18.4.4 Gözlemci

Gözlemci deseni (**observer pattern**), sistemdeki diğer nesnelerin durum değişiklikleri hakkında bir veya daha fazla nesnenin bilgilendirilmesini sağlar.



Şekil 18.16: Gözlemci Deseni UML Diyagramı

Bu tasarım deseni, nesneler arasında bire-çok (**one-to-many**) bir bağımlılık kurmak için kullanılır. Bir nesnenin durumu değiştiğinde, ona bağlı olan tüm diğer nesnelerin (abonelerin) bu değişiklikten otomatik olarak haberdar edilmesini ve güncellenmesini sağlar. Bu desen, modern yazılımlardaki "olay güdümlü" (**event-driven**) sistemlerin ve MVC (**Model-View-Controller**) mimarisinin kalbidir.

- ◆ Bağlam (**Subject**), takip edilen asıl yayıncı nesnedir. Durumu değiştiğinde etrafa haber salar. İçinde bir "Gözlemci Listesi" tutar. Yeni gözlemci eklemek (**attach()**), çıkarmak (**detach()**) ve hepsine haber vermek (**notify()**) için yöntemler barındırır.
- ◆ Gözlemci (**Observer**), güncelleme mesajlarını alabilmek için gerekli olan standart abone arayüzünü tanımlar. Genellikle sadece tek bir **update()** metodu içerir.
- ◆ Somut Yayıncı (**ConcreteSubject**), gerçek veriyi saklayan sınıftır (Örnek olarak bir borsa takip sistemi veya haber sitesi). Kendi içindeki veri değiştiğinde, miras aldığı **notify()** metodunu çağırarak tüm aboneleri tetikler.
- ◆ Somut Gözlemci (**ConcreteObserver**), haberi alan ve buna göre tepki veren sınıftır. **update()** yöntemi tetiklendiğinde, yayıncıdan gelen yeni veriyi alır ve kendi içinde işler (Ekranı yazar, veritabanına kaydeder vb.).

Şimdi desenimizi kodlayalım;

```

using System;
using System.Collections.Generic;
using System.Linq;

// 1. Gözlemci: Observer Arayüzü
public interface IObserver
{
    void Update(int state);
}

// 2. Bağlam/Yayıncı: Taban Sınıfı
public abstract class Subject
{
    private readonly List<IObserver> _observers = new List<IObserver>();
    public void Attach(IObserver observer)
    {
        _observers.Add(observer);
    }
    public void Detach(IObserver observer)
    {
        _observers.Remove(observer);
    }
    protected void Notify(int state)
    {
        foreach (var observer in _observers.ToList())
        {
            observer.Update(state);
        }
    }
}

// 3. Somut Bağlam/Yayıncı (Concrete Subject)
public class NewsAgency : Subject
{
    private int _newsId = 0;
    public void SetNews(int id)
    {
        _newsId = id;
        Console.WriteLine($"Haber Ajansı: Yeni haber yayınlandı (ID: {id})");
        Notify(_newsId); // Otomatik bildirim
    }
}

// 4. Somut Gözlemci (Concrete Observer)
public class NewsChannel : IObserver
{
    private readonly string _name;

    public NewsChannel(string name)
    {
        _name = name;
    }
    public void Update(int state)
    {
        Console.WriteLine($" -> {_name} kanalı bilgiyi aldı. Yeni haber ID: {state}");
    }
}

// 5. Client (İstemci)
class Program
{
    static void Main()
    {
        var agency = new NewsAgency();
        var channel1 = new NewsChannel("TRT Haber");
        var channel2 = new NewsChannel("NTV");
    }
}

```

```

var channel3 = new NewsChannel("CNN Türk");
agency.Attach(channel1);
agency.Attach(channel2);
agency.Attach(channel3);
agency.SetNews(101);
Console.WriteLine("-----");
agency.Detach(channel2); // NTV ayrılıyor
agency.SetNews(102);
}
}
/* Program Çıktısı:
Haber Ajansı: Yeni haber yayınlandı (ID: 101)
-> TRT Haber kanalı bilgiyi aldı. Yeni haber ID: 101
-> NTV kanalı bilgiyi aldı. Yeni haber ID: 101
-> CNN Türk kanalı bilgiyi aldı. Yeni haber ID: 101
-----
Haber Ajansı: Yeni haber yayınlandı (ID: 102)
-> TRT Haber kanalı bilgiyi aldı. Yeni haber ID: 102
-> CNN Türk kanalı bilgiyi aldı. Yeni haber ID: 102
*/

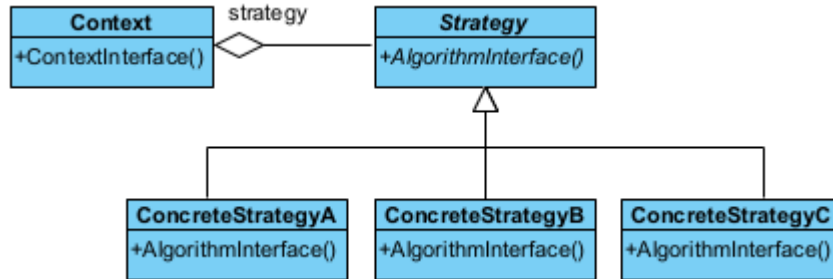
```

Bu desen aşağıdaki durumlarda kullanılır;

- ◊ Bir soyutlama (**abstraction**) sonucu ortaya çıkan bağımsız iki farklı yönün (**aspect**) ortaya çıktığı durumlarda, bu yönler birbirinden bağımsız olarak geliştirilebilir ve değiştirilebilir.
- ◊ Bir değişikliğin başka değişiklikleri gerektirdiği durumlar.
- ◊ Bir nesnenin, diğer nesnelerdeki değişiklikleri kendine vazife etmemesini sağlayan durumlar.

§18.4.5 Strateji

Strateji deseni (**strategy pattern**), belirli bir davranışı gerçekleştirmek için değiştirilebilen kapsüllenmiş (**encapsulated**) algoritmalar kümesini tanımlar.



Şekil 18.17: Strateji Deseni UML Diyagramı

Bu desenin görevi, bir işi yapmanın birden fazla yolu varsa, bu yolları (algoritmaları) sınıflara ayırır ve birbirinin yerine kullanılabilir hale getirir.

- ◊ Bağlam (**Context**), işi yapacak olan sınıf. Ancak işi "nasıl" yapacağını bir strateji nesnesine sorar.
- ◊ Strateji (**Strategy**): Tüm algoritmaların uyması gereken ara yüzdür.
- ◊ Somut Strateji (**ConcreteStrategy**), farklı algoritmalar ile tanımlı stratejilerdir.
(Örnek olarak "Hızlı Sıralama", "Kabarcık Sıralama" veya "Kredi Kartıyla Öde", "Nakit Öde")

Şimdi desenimizi kodlayalım;

```

using System;
// 1. Strateji Arayüzü
public interface IStrategy
{
    void Execute();
}
// 2. Somut Stratejiler
public class ConcreteStrategyA : IStrategy
{
    public void Execute()
    {
        Console.WriteLine("Algoritma A: Hızlı ama daha fazla bellek kullanıyor.");
    }
}

```

```

    }
}
public class ConcreteStrategyB : IStrategy
{
    public void Execute()
    {
        Console.WriteLine("Algoritma B: Yavaş ama bellek dostu.");
    }
}
public class ConcreteStrategyC : IStrategy
{
    public void Execute()
    {
        Console.WriteLine("Algoritma C: Dengeli bir yaklaşım.");
    }
}
// 3. Bağlam (Context)
public class Context
{
    private IStrategy _strategy;
    public Context(IStrategy strategy)
    {
        _strategy = strategy;
    }
    public void SetStrategy(IStrategy strategy)
    {
        _strategy = strategy;
    }
    public void Run()
    {
        if (_strategy != null)
        {
            _strategy.Execute();
        }
        else
        {
            Console.WriteLine("Hata: Henüz bir strateji belirlenmedi!");
        }
    }
}
// 4. Client (İstemci)
class Program
{
    static void Main()
    {
        // Strateji nesnelerini oluşturuyoruz
        IStrategy s1 = new ConcreteStrategyA();
        IStrategy s2 = new ConcreteStrategyB();
        IStrategy s3 = new ConcreteStrategyC();
        // Bağlam (Context) nesnesini oluştur ve başlangıç stratejisini ata
        Context context = new Context(s1);
        Console.WriteLine("--- Durum 1 ---");
        context.Run();
        // Çalışma zamanında stratejiyi değiştir
        context.SetStrategy(s2);
        Console.WriteLine("--- Durum 2 ---");
        context.Run();
        context.SetStrategy(s3);
        Console.WriteLine("--- Durum 3 ---");
        context.Run();
    }
}

```

```

/* Program Çıktısı:
--- Durum 1 ---
Algoritma A: Hızlı ama daha fazla bellek kullanıyor.
--- Durum 2 ---
Algoritma B: Yavaş ama bellek dostu.
--- Durum 3 ---
Algoritma C: Dengeli bir yaklaşım.
*/

```

Bu desenin **bağlam** nesnesinin, diğer tasarım desenlerinin biri ile iç içe kullanılması halinde, bu desen **sıfır desen** (**null pattern**) olarak adlandırılır. Sıfır desende, **Context** nesnesi akıllıdır ve her zaman ne yapacağını bilir. Bu desen aşağıdaki durumda kullanılır;

- ◊ Aralarında ilişki bulunan birçok sınıfın davranışlara bağlı olarak anlaşmazlığa düşmesini önlemek için,
- ◊ Birden çok algoritmanın olması durumunda,
- ◊ Algoritma, istemcinin bilmeyeceği bir veri yapısına sahip olduğu durumlarda,
- ◊ Bir sınıf kendi yöntemlerinde çok fazla sayıda duruma göre seçim (**conditional choice**) kullanıyorsa.

Aşağıda bu desene ilişkin sıralama algoritmalarının seçildiği gerçek bir örnek verilmiştir;

```

using System;
using System.Collections.Generic;
// 1. --- Abstract Strategy (Strateji Arayüzü) ---
public interface ISortStrategy
{
    void Sort(List<int> data);
}
// 2. --- Concrete Strategy A: Quick Sort ---
public class QuickSort : ISortStrategy
{
    public void Sort(List<int> data)
    {
        Console.WriteLine("QuickSort algoritması kullanılıyor...");
        // C# List.Sort() metodu genellikle QuickSort türevi (Introspective Sort) kullanır.
        data.Sort();
    }
}
// 3. --- Concrete Strategy B: Merge Sort ---
public class MergeSort : ISortStrategy
{
    public void Sort(List<int> data)
    {
        Console.WriteLine("MergeSort algoritması kullanılıyor...");
        // Örnek amaçlı: C# tarafında harici bir kütüphane veya manuel
        // implementasyon yerine LINQ OrderBy kullanımı stabil bir sıralama sağlar.
        var sorted = data.OrderBy(n => n).ToList();
        data.Clear();
        data.AddRange(sorted);
    }
}
// 4. --- Context: Sorter ---
public class Sorter
{
    private ISortStrategy? _strategy;
    // Stratejiyi çalışma zamanında değiştirmemizi sağlar (Property veya Method)
    public void SetStrategy(ISortStrategy strategy)
    {
        _strategy = strategy;
    }
    public void Sort(List<int> data)
    {
        if (_strategy == null)
        {
            Console.WriteLine("Hata: Önce bir strateji belirlenmelidir!");
        }
    }
}

```

```

        return;
    }
    _strategy.Sort(data);
}
}
// 5. --- Client (İstemci) ---
class Program
{
    static void Main()
    {
        List<int> sayilar = new List<int> { 34, 12, 5, 78, 1, 45 };
        Sorter siralayici = new Sorter();

        Console.Write("Orijinal liste: ");
        PrintList(sayilar);
        // 1. QuickSort Stratejisini Seçelim
        siralayici.SetStrategy(new QuickSort());
        siralayici.Sort(sayilar);
        PrintList(sayilar);
        // Listeyi tekrar karıştıralım
        sayilar = new List<int> { 99, 2, 56, 11, 0, 7 };
        Console.Write("\nYeni liste: ");
        PrintList(sayilar);
        // 2. MergeSort Stratejisine Geçelim
        siralayici.SetStrategy(new MergeSort());
        siralayici.Sort(sayilar);
        PrintList(sayilar);
    }
    static void PrintList(List<int> data)
    {
        Console.WriteLine(string.Join(" ", data));
    }
}

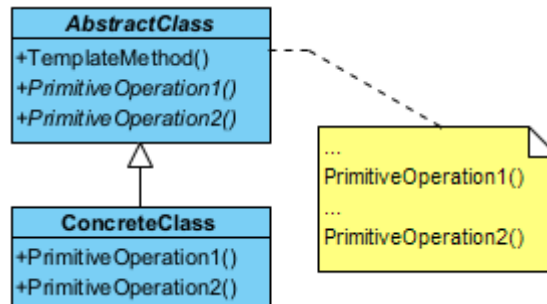
/* Programın Çıktısı:
Orijinal liste: 34 12 5 78 1 45
QuickSort algoritması kullanılıyor...
1 5 12 34 45 78

Yeni liste: 99 2 56 11 0 7
MergeSort algoritması kullanılıyor...
0 2 7 11 56 99
*/

```

§18.4.6 Şablon Yöntem

Şablon yöntem deseninde (**template method pattern**), bir algoritmanın çerçevesini belirler ve uygulayıcı sınıfların gerçek davranışı tanımlamasına olanak tanır.



Şekil 18.18: Şablon Yöntem Deseni UML Diyagramı

Bu desen, nesne yönelimli programlamada "algoritma iskeleti" oluşturmak için kullanılır. Bir işin adımlarını (sırasını) belirler ancak bu adımların bazılarının nasıl yapılacağını alt sınıflara bırakır. Kod tekrarını

önlemek ve bir işlemin temel yapısını korumak için mükemmeldir.

- ◊ Soyut Sınıf (**AbstractClass**), algoritmanın ana iskeletini ve şablon yöntemi (**TemplateMethod()**) barındırır. Değişmez adımları kendi içinde tanımlar, değişebilecek adımları ise soyut (**virtual**) olarak işaretler. Şablon yöntem genellikle **final** olarak tanımlanır ki algoritmanın sırası bozulmasın.
- ◊ Somut Sınıf (**ConcreteClass**); Soyut sınıftaki "boşlukları" doldurur. Algoritmanın değişken adımlarını kendi ihtiyacına göre özelleştirir. Algoritmanın genel akışına müdahale edemez, sadece kendisine izin verilen adımları yazar.

Şimdi desenimizi kodlayalım;

```
using System;
// 1. Soyut Sınıf (Abstract Class)
public abstract class AbstractClass
{
    // Şablon Yöntem: Algoritmanın iskeletini sabitler.
    // 'sealed' yapılamaz çünkü kalıtım gereklidir, ancak metodun kendisi
    // alt sınıflar tarafından değiştirilemesin diye sanal (virtual) yapılmaz.
    public void TemplateMethod()
    {
        BaseOperation();           // Ortak adım
        RequiredOperation1();       // Özelleştirilebilir adım
        Hook();                     // İsteğe bağlı adım (Hook)
        RequiredOperation2();       // Özelleştirilebilir adım
    }
    // Temel operasyon: Tüm alt sınıflar için aynıdır.
    private void BaseOperation()
    {
        Console.WriteLine("Sistem: Algoritmanın ortak temel adımı çalıştırılıyor.");
    }
    // Alt sınıfların mutlaka uygulaması gereken adımlar
    protected abstract void RequiredOperation1();
    protected abstract void RequiredOperation2();
    // Hook (Kanca): Alt sınıflar isterse geçersiz kılar, istemezse boş kalır.
    // C#'ta boş bir gövde ile 'virtual' tanımlanır.
    protected virtual void Hook() { }
}
// 2. Somut Sınıf (Concrete Class)
public class ConcreteClass : AbstractClass
{
    protected override void RequiredOperation1()
    {
        Console.WriteLine("ConcreteClass: Adım 1 özel olarak uygulandı.");
    }
    protected override void RequiredOperation2()
    {
        Console.WriteLine("ConcreteClass: Adım 2 özel olarak uygulandı.");
    }
    // İsteğe bağlı: Hook'u geçersiz kılarız.
    protected override void Hook()
    {
        Console.WriteLine("ConcreteClass: Hook (Kanca) devreye girdi.");
    }
}
// 3. Client (İstemci)
class Program
{
    static void Main()
    {
        AbstractClass algoritma = new ConcreteClass();
        Console.WriteLine("--- Algoritma Başlatılıyor ---");
        algoritma.TemplateMethod();
    }
}
```



```

/* Program Çıktısı:
--- Algoritma Başlatılıyor ---
Sistem: Algoritmanın ortak temel adımı çalıştırılıyor.
ConcreteClass: Adım 1 özel olarak uygulandı.
ConcreteClass: Hook (Kanca) devreye girdi.
ConcreteClass: Adım 2 özel olarak uygulandı.
*/

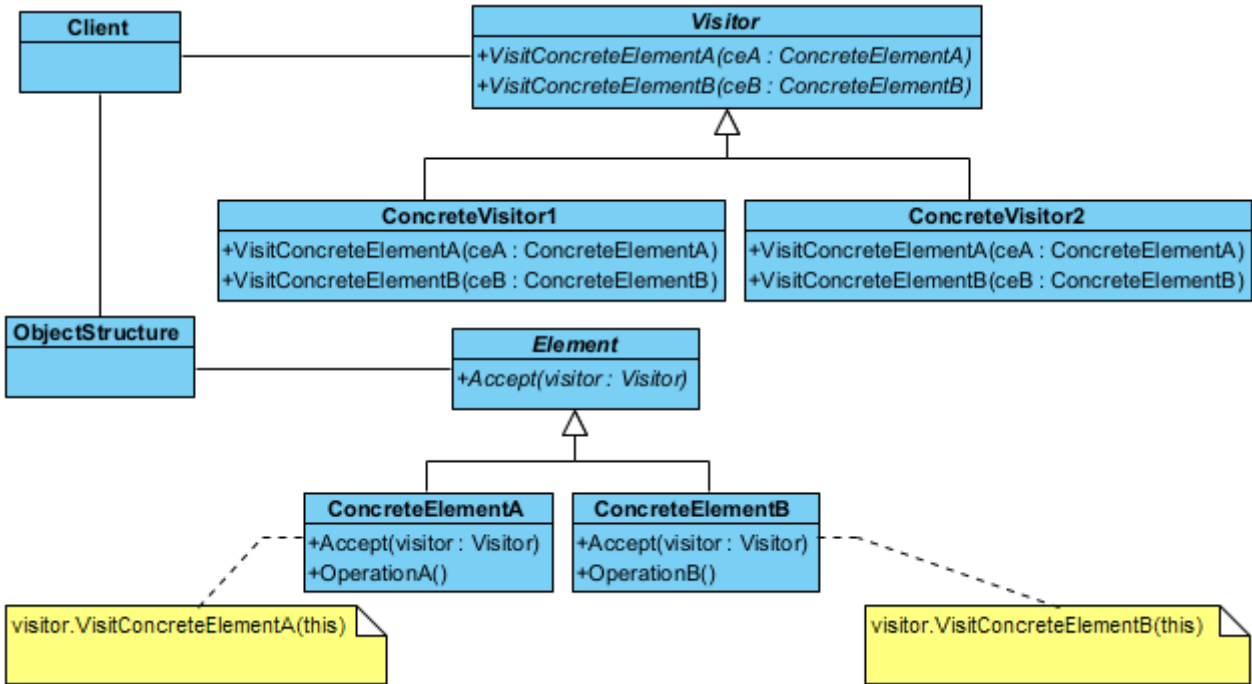
```

Bu desende, kullanılacak algoritmalarından hangilerinin standart, hangilerinin somut sınıflara özgü olacağı belirlenmelidir;

- ◊ İçinde "Bizi çağırmayın! Biz sizi çağıracağız." sloganını taşıyan yöntemler olan bir soyut bir taban sınıf tasarlanmalıdır.
- ◊ Taban sınıf içinde algoritmanın kabuğunu oluşturacak standart adımları içeren şablon yöntem tanımlanır.
- ◊ Somut sınıflarda detaylandırılacak yöntemlere ilişkin sanal yöntemler taban sınıfta tanımlanır.
- ◊ Şablon yöntem içinde algoritmayı oluşturan yöntemler çağrılır.
- ◊ Şablon yöntem içinde yer almayan tüm detay kodlamalar somut sınıflar için de yapılır.

§18.4.7 Ziyaretçi

Ziyaretçi deseni (visitor pattern), bir nesne grubunun (koleksiyonun) yapısını değiştirmeden, bu nesneler üzerinde uygulanacak yeni işlemleri dışarıdan eklememizi sağlayan davranışsal bir desendir. Bu desen, "Veri Yapısı" ile "Algoritma"yı birbirinden ayırır. Eğer elinizde sabit bir sınıf hiyerarşisi varsa (örneğin bir dokümandaki paragraflar, resimler, tablolar) ve sürekli bu sınıflar üzerinde yeni işlemler (PDF'e dönüştür, HTML'e dönüştür, kelime say) tanımlamanız gerekiyorsa, her seferinde o sınıfların içine girip kod yazmak yerine ziyaretçi desenini kullanırsınız.



Şekil 18.19: Ziyaretçi Deseni UML Diyagramı

- ◊ Ziyaretçi (**Visitor**), koleksiyondaki her bir somut nesne türü için bir ziyaret yöntemi (**visit()**) tanımlayan soyut bir sınıftır. "Eğer bir Paragraf nesnesine gidersem ne yapacağım?", "Eğer bir Resim nesnesine gidersem ne yapacağım?" sorularının imzasını belirler.
- ◊ Somut Ziyaretçi (**ConcreteVisitor**), gerçek operasyonu gerçekleştirir. Algoritmanın kendisidir. Örneğin **ExportToPDFVisitor** sınıfı, her nesne türünün PDF'e nasıl dönüştürüleceğini bilir.
- ◊ Eleman (**Element**), bir ziyaretçiyi kabul edebilmek için standart bir "kapı" açan soyut bir sınıftır. Ziyaretçi kabul etmek için **accept(Visitor* v)** yöntemini barındırır.
- ◊ Somut Elemanlar (**ConcreteElement**), veri yapısının asıl parçalarıdır. **accept()** yöntemi çağrıldığında, ziyaretçiye "Ben bir [NesneTürü]'yüm, beni ziyaret et" der (**v->visit(this)**). Buna yazılım dünyasında **çifte sevkیات (double dispatch)** denir.

Şimdi desenimizi kodlayalım;

```
using System;
using System.Collections.Generic;
// 1. Visitor (Ziyaretçi) Arayüzü
public interface IVisitor
{
    // C#'ta aşırı yükleme (overloading) sayesinde metod isimlerini aynı tutabiliriz
    void Visit(ConcreteElementA element);
    void Visit(ConcreteElementB element);
}
// 2. Element (Eleman) Arayüzü
public interface IElement
{
    // Double Dispatch mekanizmasının kalbi: Accept metodu
    void Accept(IVisitor visitor);
}
// 3. Somut Elemanlar
public class ConcreteElementA : IElement
{
    public void Accept(IVisitor visitor)
    {
        // Çalışma zamanında visitor'ın hangi metodunun çağrılacağı burada netleşir
        visitor.Visit(this);
    }
    public void OperationA()
    {
        Console.WriteLine("ConcreteElementA: Özel işlem çalıştırılıyor.");
    }
}

public class ConcreteElementB : IElement
{
    public void Accept(IVisitor visitor)
    {
        visitor.Visit(this);
    }
    public void OperationB()
    {
        Console.WriteLine("ConcreteElementB: Özel işlem çalıştırılıyor.");
    }
}
// 4. Somut Ziyaretçiler
public class ConcreteVisitor1 : IVisitor
{
    public void Visit(ConcreteElementA element)
    {
        Console.WriteLine("Visitor 1: ElementA ziyaret edildi.");
        element.OperationA();
    }
    public void Visit(ConcreteElementB element)
    {
        Console.WriteLine("Visitor 1: ElementB ziyaret edildi.");
        element.OperationB();
    }
}
public class ConcreteVisitor2 : IVisitor
{
    public void Visit(ConcreteElementA element)
    {
        Console.WriteLine("Visitor 2: ElementA üzerinde farklı bir işlem yapılıyor.");
    }
    public void Visit(ConcreteElementB element)
    {

```

```

        Console.WriteLine("Visitor 2: ElementB üzerinde farklı bir işlem yapılıyor.");
    }
}
// 5. Nesne Yapısı (Object Structure)
public class ObjectStructure
{
    private readonly List<IElement> _elements = new List<IElement>();
    public void Add(IElement e) => _elements.Add(e);
    public void Accept(IVisitor v)
    {
        foreach (var element in _elements)
        {
            element.Accept(v);
        }
    }
}
// 6. Client (İstemci)
class Program
{
    static void Main()
    {
        ObjectStructure structure = new ObjectStructure();
        structure.Add(new ConcreteElementA());
        structure.Add(new ConcreteElementB());
        IVisitor v1 = new ConcreteVisitor1();
        IVisitor v2 = new ConcreteVisitor2();
        Console.WriteLine("--- Ziyaretçi 1 Başlıyor ---");
        structure.Accept(v1);
        Console.WriteLine("\n--- Ziyaretçi 2 Başlıyor ---");
        structure.Accept(v2);
    }
}
/* Program Çıktısı:
--- Ziyaretçi 1 Başlıyor ---
Visitor 1: ElementA ziyaret edildi.
ConcreteElementA: Özel işlem çalıştırılıyor.
Visitor 1: ElementB ziyaret edildi.
ConcreteElementB: Özel işlem çalıştırılıyor.

--- Ziyaretçi 2 Başlıyor ---
Visitor 2: ElementA üzerinde farklı bir işlem yapılıyor.
Visitor 2: ElementB üzerinde farklı bir işlem yapılıyor.
*/

```

Bu desen aşağıdaki durumlarda kullanılır;

- ◊ Nesneler birden fazla olup birçok ara yüze sahip olduğunda, bu nesnelerin somut sınıfları ile bir işlem yapmak gerektiğinde,
- ◊ Sınıflar üzerinde yapılacak işlemlerin, sınıflar üzerinde bir iz bırakmamasını sağlamak amacıyla,
- ◊ Nadir değişen sınıf yapılarına, sınıf yapısını değiştirmeden, yeni işlemler eklemek gerektiğinde.

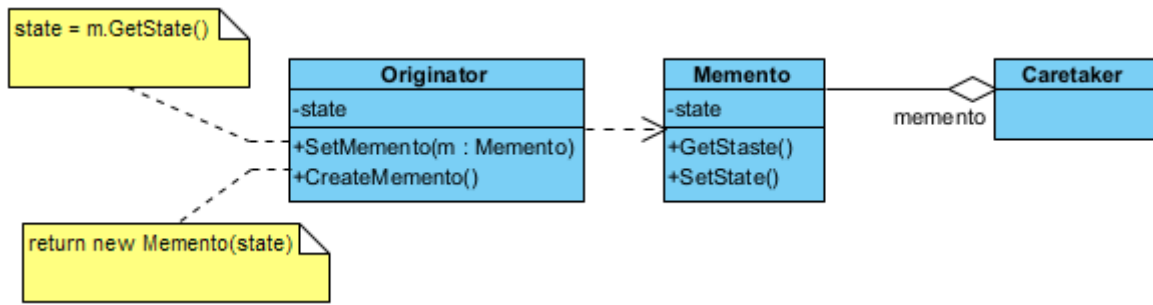
Bu deseni kullanmanın aşağıda sıralanan üstünlükleri sağlar;

- ◊ Yeni işlemleri (**operation**) sisteme eklemeyi kolaylaştırır.
- ◊ Birbiriyle ilişkili olan işlemleri ilişkili olmayandan ayırmaya yardımcı olur.
- ◊ Yeni **ConcreteElement** nesnelerinin eklenmesini zorlaştırır.
- ◊ Bütün nesne hiyerarşisi baştanbaşa ziyaret edilebilir. Sınıflardaki tüm durumlar (**state**) biriktirilebilir.
- ◊ Bazı durumlarda kapsülleme (**encapsulation**) işlemini engeller. Yani nesnenin iç durumlarından hareketle herkese açık işlemler tanımlanabilir.

§18.4.8 Hatıra

Hatıra deseni (**memento pattern**), nesnelerin bir andaki durumlarını (**state**) saklayıp bir başka zaman da bu duruma geri dönmek gerektiğinde kullanılır. Bir nesnenin iç durumunun yakalanmasına ve dışarıda saklanmasına olanak tanır, böylece daha sonra geri yüklenebilir fakat tüm bunlar **kılıflamayı** (**encapsulation**)

ihlal etmeden yapılır.



Şekil 18.20: Hatıra Deseni UML Diyagramı

Hatıra deseninin amacı, nesnenin iç durumunu (**state**) bozmadan saklamak ve daha sonra geri yüklemektir.

- Yaratıcı (**Originator**), durumu değişen asıl nesnedir. Kendi "anlık görüntüsünü" (**snapshot**) oluşturur ve geri yükler.
- Hatıra (**Memento**), sadece veriyi tutan pasif nesnedir. Dışarıdan değiştirilemez; sadece **Originator** içeriğini okuyabilir.
- Bakıcı (**Caretaker**), hatıraları güvenli bir yerde (liste veya yığın) saklar. İçeriği asla değiştirmez, sadece saklar ve geri verir.

Şimdi desenimizi kodlayalım;

```

using System;
using System.Collections.Generic;
// 1. Originator: Durumu değişen ve "hatıra" üreten asıl nesne
public class Originator
{
    private int _state;
    public void SetState(int state)
    {
        _state = state;
        Console.WriteLine($"Originator: Durum değiştirildi -> {_state}");
    }
    // Mevcut durumu bir Memento nesnesine paketler
    public Memento Save()
    {
        Console.WriteLine($"Originator: Durum yedekleniyor... ({_state})");
        return new Memento(_state);
    }
    // Bir Memento nesnesinden durumu geri yükler
    public void Restore(Memento memento)
    {
        if (memento != null)
        {
            _state = memento.GetState();
            Console.WriteLine($"Originator: Durum geri yüklendi -> {_state}");
        }
    }
}
// --- Memento: Durumu saklayan nesne (İç Sınıf Olarak) ---
// C#'ta memento'yu iç sınıf yaparak Originator dışındakilerin verilere erişmesini
// → kısıtlarız.
public class Memento
{
    private readonly int _state;
    // Constructor internal veya private yapılarak sadece Originator erişimi sağlanır
    internal Memento(int state)
    {
        _state = state;
    }
    internal int GetState() => _state;
}
  
```

```

    }
}
// 2. Caretaker: Hatıraları saklayan bekçi
public class Caretaker
{
    private readonly Stack<Originator.Memento> _history = new Stack<Originator.Memento>();
    private readonly Originator _originator;
    public Caretaker(Originator originator)
    {
        _originator = originator;
    }
    public void Backup()
    {
        _history.Push(_originator.Save());
    }
    public void Undo()
    {
        if (_history.Count == 0) return;

        Originator.Memento memento = _history.Pop();
        _originator.Restore(memento);
    }
}
// 3. İstemci (Client)
class Program
{
    static void Main()
    {
        Originator player = new Originator();
        Caretaker saveManager = new Caretaker(player);
        player.SetState(100); // Bölüm 1
        saveManager.Backup(); // Kayıt noktası
        player.SetState(50); // Can azaldı
        player.SetState(0); // Öldü!
        Console.WriteLine("--- Geri Yükleniyor ---");
        saveManager.Undo(); // Bölüm 1'e geri dön
    }
}
/* Program Çıktısı:
Originator: Durum değiştirildi -> 100
Originator: Durum yedekleniyor... (100)
Originator: Durum değiştirildi -> 50
Originator: Durum değiştirildi -> 0
--- Geri Yükleniyor ---
Originator: Durum geri yüklendi -> 100
*/

```

Aşağıdaki örnekte bir metin editörünün geçmiş özelliği modellenmiştir;

```

using System;
using System.Collections.Generic;
// --- Memento ---
// Sadece saklanacak veriyi tutan pasif nesne.
public class EditorMemento
{
    // C#'ta 'readonly' kullanarak verinin sonradan değiştirilmesini engelleriz.
    private readonly string _content;
    public EditorMemento(string text)
    {
        _content = text;
    }
    public string GetSavedContent() => _content;
}

```

```
// --- Originator ---
// Durumu kaydedilecek ve geri yüklenecek olan asıl nesne.
public class TextEditor
{
    private string _text = "";
    public void Type(string newText) => _text += newText;
    public string GetText() => _text;
    // Mevcut durumu bir Memento içine paketler.
    public EditorMemento Save()
    {
        return new EditorMemento(_text);
    }
    // Bir Memento'dan durumu geri yükler.
    public void Restore(EditorMemento memento)
    {
        if (memento != null)
        {
            _text = memento.GetSavedContent();
        }
    }
}

// --- Caretaker ---
// Geçmiş (Memento listesini) yöneten sınıf.
public class History
{
    // Geri alma işlemleri için LIFO (Stack) yapısı en doğrusudur.
    private readonly Stack<EditorMemento> _mementos = new Stack<EditorMemento>();
    public void Push(EditorMemento m)
    {
        _mementos.Push(m);
    }
    public EditorMemento Pop()
    {
        if (_mementos.Count == 0) return null;
        return _mementos.Pop();
    }
}

// --- Client (İstemci) ---
class Program
{
    static void Main()
    {
        TextEditor editor = new TextEditor();
        History history = new History();
        // 1. Yazmaya başla ve ilk durumu kaydet
        editor.Type("Merhaba ");
        history.Push(editor.Save());
        Console.WriteLine($"Güncel Metin: {editor.GetText()}");
        // 2. Yazmaya devam et ve ikinci durumu kaydet
        editor.Type("Dünya!");
        history.Push(editor.Save());
        Console.WriteLine($"Güncel Metin: {editor.GetText()}");
        // 3. Bir hata yapalım
        editor.Type(" Yanlış bir ekleme.");
        Console.WriteLine($"Hata Sonrası: {editor.GetText()}");
        // 4. Geri al (Undo)
        // Son eklenen hatıra "Merhaba Dünya!" idi (history.Pop()).
        history.Pop(); // "Merhaba Dünya!" çıktı
        var previousState = history.Pop(); // "Merhaba " çıktı
        if (previousState != null)
        {
            editor.Restore(previousState);
        }
    }
}
```

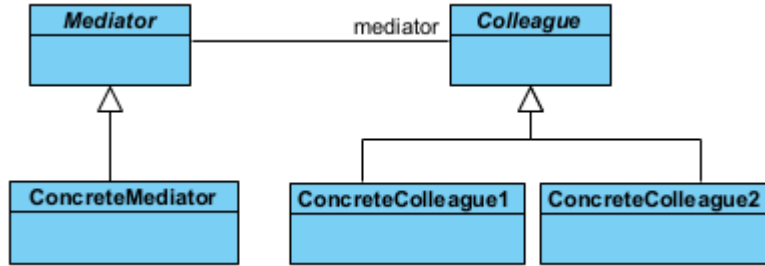
```

        Console.WriteLine($"Geri Alındı: {editor.GetText()}");
    }
}
/* Program Çıktısı:
Güncel Metin: Merhaba
Güncel Metin: Merhaba Dünya!
Hata Sonrası: Merhaba Dünya! Yanlış bir ekleme.
Geri Alındı: Merhaba
*/

```

§18.4.9 Ara Bulucu

Ara Bulucu deseni (**mediator pattern**), sınıfların arasındaki iletişimi basitleştirir. Bir iletişim nesnesi tanımlanır, diğer nesneler bu nesne üzerinden birbirleriyle haberleşir. Diğer nesnelerin birbiriyle doğrudan iletişim kurmasına izin verilmez. Amaç, iletişimin kontrol altına alınmasıdır. Bu tasarım deseni, bir dizi nesne



Şekil 18.21: Ara Bulucu Deseni UML Diyagramı

arasındaki karmaşık iletişim ağını basitleştirmek için kullanılır. Nesnelerin birbirleriyle doğrudan konuşmasını engeller ve tüm iletişimi tek bir "merkezi aracı" üzerinden yürütmelerini sağlar. Bu desen, özellikle nesne sayısı arttığında ortaya çıkan "örümcek ağı" gibi karmaşık bağlantıları (**tight coupling**) çözmek için birebirdir.

- ◇ Soyut Aracı (**Mediator**), nesneler arasındaki iletişim protokolünü (arayüzünü) tanımlar. "Hangi olay gerçekleştiğinde kimlere haber verilecek?" sorusunun genel çerçevesini çizer.
- ◇ Somut Aracı (**ConcreteMediator**), iletişim trafiğini yöneten asıl "santral" binasıdır. tüm meslektaşları (**Colleague**) tanır ve aralarındaki koordinasyonu sağlar. Bir bileşenden mesaj geldiğinde, mantık çerçevesinde diğerlerine iletir.
- ◇ Meslektaş (**Colleague**) Sınıflar, kendi işlevini yerine getiren bağımsız sınıflardır. Diğer sınıfların varlığından haberdar değildirler. Sadece "Aracı"ya (**Mediator**) haber verirler. "Ben düğmeye bastım, geri-sini aracı halletsin" mantığıyla çalışırlar.

Şimdi desenimizi kodlayalım;

```

using System;
using System.Collections.Generic;
// 1. Mediator (Ara bulucu) Arayüzü
public interface IMediator
{
    void RegisterColleague(Colleague colleague);
    void RelayMessage(string message, Colleague sender);
}
// 2. Colleague (Meslektaş) Taban Sınıfı
public abstract class Colleague
{
    protected string Name;
    protected IMediator Mediator;
    protected Colleague(string name, IMediator mediator)
    {
        Name = name;
        Mediator = mediator;
    }
    public string GetName() => Name;
    public void Send(string message)
    {

```

```

        Mediator.RelayMessage(message, this);
    }
    public virtual void Receive(string message, string senderName)
    {
        Console.WriteLine($"[{Name}] {senderName} kişisinden mesaj aldı: {message}");
    }
}
// 3. Somut Ara bulucu (Concrete Mediator)
public class ChatRoom : IMediator
{
    private readonly List<Colleague> _colleagues = new List<Colleague>();
    public void RegisterColleague(Colleague colleague)
    {
        _colleagues.Add(colleague);
    }
    public void RelayMessage(string message, Colleague sender)
    {
        Console.WriteLine($"--- LOG: {sender.GetName()} bir mesaj yayınladı ---");
        foreach (var colleague in _colleagues)
        {
            // Mesajı gönderen kişiye geri göndermiyoruz
            if (colleague != sender)
            {
                colleague.Receive(message, sender.GetName());
            }
        }
    }
}
// 4. Somut Meslektaş (Concrete Colleague)
public class User : Colleague
{
    public User(string name, IMediator mediator) : base(name, mediator) { }
}
// 5. İstemci (Client)
class Program
{
    static void Main()
    {
        // Arabulucu nesnesi
        IMediator oda = new ChatRoom();
        // Meslektaşlar (Kullanıcılar)
        User ilhan = new User("Ilhan", oda);
        User mehmet = new User("Mehmet", oda);
        User ayse = new User("Ayse", oda);
        // Kayıt işlemleri
        oda.RegisterColleague(ilhan);
        oda.RegisterColleague(mehmet);
        oda.RegisterColleague(ayse);
        // İletişim başlıyor
        ilhan.Send("Selam millet!");
        mehmet.Send("Merhabalar Ilhan.");
        ayse.Send("Nasılsınız?");
    }
}
/* Program Çıktısı:
--- LOG: Ilhan bir mesaj yayınladı ---
[Mehmet] Ilhan kişisinden mesaj aldı: Selam millet!
[Ayse] Ilhan kişisinden mesaj aldı: Selam millet!
--- LOG: Mehmet bir mesaj yayınladı ---
[Ilhan] Mehmet kişisinden mesaj aldı: Merhabalar Ilhan.
[Ayse] Mehmet kişisinden mesaj aldı: Merhabalar Ilhan.
--- LOG: Ayse bir mesaj yayınladı ---

```



```
[Ilhan] Ayse kişisinden mesaj aldı: Nasılsınız?
[Mehmet] Ayse kişisinden mesaj aldı: Nasılsınız?
*/
```

Bu desende, sistemde yalnızca bir adet ara bulucu olacaksa soyut **Mediator** nesnesi tanımlanmayabilir. Ayrıca ara bulucu ile meslektaşlar arasındaki iletişim iki türlü yapılır;

- ◊ Birisinde bu iletişim gözlemci deseni (**observer pattern**) ile sağlanır. **Colleague** sınıfı gözlemci desenindeki bağlam **Subject** sınıfı gibi davranır.
- ◊ Diğerinde ise **Mediator** nesnesine, görüşecekleri meslektaşları belirterek doğrudan ileti gönderirler.

Aşağıda havaalanındaki kule ile uçaklar arasındaki iletişimi modelleyen bir örnek verilmiştir;

```
using System;
using System.Collections.Generic;
// --- Mediator Arayüzü ---
public interface IAirTrafficControl
{
    void SendMessage(string message, Airplane originator);
    void RegisterAirplane(Airplane airplane);
}
// --- Colleague (Meslektaş/Bileşen) ---
public class Airplane
{
    protected IAirTrafficControl Mediator;
    protected string CallSign;
    public Airplane(IAirTrafficControl mediator, string callSign)
    {
        Mediator = mediator;
        CallSign = callSign;
    }
    public virtual void Send(string message)
    {
        Console.WriteLine($"[{CallSign}] Kuleye mesaj gönderiyor: {message}");
        Mediator.SendMessage(message, this);
    }
    public virtual void Receive(string message)
    {
        Console.WriteLine($"[{CallSign}] Mesaj aldı: {message}");
    }
    public string GetCallSign() => CallSign;
}

// --- Concrete Mediator (Somut Aracı) ---
public class AtcTower : IAirTrafficControl
{
    private readonly List<Airplane> _airplanes = new List<Airplane>();
    public void RegisterAirplane(Airplane airplane)
    {
        if (!_airplanes.Contains(airplane))
        {
            _airplanes.Add(airplane);
        }
    }
    public void SendMessage(string message, Airplane originator)
    {
        foreach (var airplane in _airplanes)
        {
            // Mesajı gönderen uçak hariç herkese ilet
            if (airplane != originator)
            {
                airplane.Receive($"Kule'den {originator.GetCallSign()} bilgisi: {message}");
            }
        }
    }
}
```

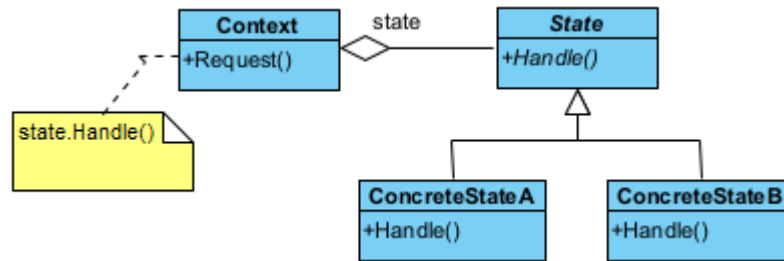
```

    }
}
// --- Client (İstemci) ---
class Program
{
    static void Main()
    {
        // 1. Aracı (Kule) oluşturulur
        AtcTower tower = new AtcTower();
        // 2. Meslektaşlar (Uçaklar) oluşturulur ve kuleye bildirilir
        Airplane boeing = new Airplane(tower, "THY-123");
        Airplane airbus = new Airplane(tower, "PGS-456");
        Airplane cessna = new Airplane(tower, "OZEL-789");
        tower.RegisterAirplane(boeing);
        tower.RegisterAirplane(airbus);
        tower.RegisterAirplane(cessna);
        // 3. İletişim merkezi üzerinden yürütülür
        boeing.Send("Pist 1'e iniş izni istiyorum.");
        Console.WriteLine(new string('-', 42));
        cessna.Send("Pistten çıkış yapıldı.");
    }
}
/* Program Çıktısı:
[THY-123] Kuleye mesaj gönderiyor: Pist 1'e iniş izni istiyorum.
[PGS-456] Mesaj aldı: Kule'den THY-123 bilgisi: Pist 1'e iniş izni istiyorum.
[OZEL-789] Mesaj aldı: Kule'den THY-123 bilgisi: Pist 1'e iniş izni istiyorum.
-----
[OZEL-789] Kuleye mesaj gönderiyor: Pistten çıkış yapıldı.
[THY-123] Mesaj aldı: Kule'den OZEL-789 bilgisi: Pistten çıkış yapıldı.
[PGS-456] Mesaj aldı: Kule'den OZEL-789 bilgisi: Pistten çıkış yapıldı.
*/

```

§18.4.10 Durum

Durum deseni (state pattern), nesnenin durumunu davranışına bağlar, nesnenin içsel durumuna göre farklı şekillerde davranmasına olanak tanır.



Şekil 18.22: Durum Deseni UML Diyagramı

Bu desenin amacı, nesnenin iç durumu değiştiğinde davranışını da değiştirmek. Nesne sanki sınıfını değiştiriyormuş gibi görünür.

- ◊ Bağlam (Context), istemcinin (Client) etkileşime girdiği ana sınıftır. Mevcut durum nesnesini bir referans olarak tutar.
- ◊ Soyut Durum (State), tüm somut durumların uyması gereken arayüzü (Interface) tanımlar.
- ◊ Somut Durumlar (ConcreteState), Her sınıf, belirli bir durumdaki davranışını temsil eder (Örn: Oynatılıyor, Duraklatıldı). Bir durumdan diğerine geçiş mantığını genellikle bu sınıflar yönetir.

Şimdi desenimizi kodlayalım;

```

using System;
// 1. State Arayüzü (Soyut Durum)
public abstract class State
{
    public abstract void Handle(Context context);
}

```

```
// 2. Context (Bağlam)
public class Context
{
    private State _currentState;
    public Context(State state)
    {
        TransitionTo(state);
    }
    public void TransitionTo(State state)
    {
        Console.WriteLine($"Context: Durum değiştiriliyor ({state.GetType().Name})...");
        _currentState = state;
    }
    public void Request()
    {
        _currentState.Handle(this);
    }
}

// 3. Somut Durumlar
public class ConcreteStateA : State
{
    public override void Handle(Context context)
    {
        Console.WriteLine("ConcreteStateA: Talep işleniyor. B durumuna geçiş tetikleniyor.");
        context.TransitionTo(new ConcreteStateB());
    }
}

public class ConcreteStateB : State
{
    public override void Handle(Context context)
    {
        Console.WriteLine("ConcreteStateB: Talep işleniyor. A durumuna geri dönülüyor.");
        context.TransitionTo(new ConcreteStateA());
    }
}

// 4. İstemci (Client)
class Program
{
    static void Main()
    {
        // Başlangıç durumu A ile Context oluşturuluyor
        Context context = new Context(new ConcreteStateA());
        // İstekler gönderildikçe durumlar kendi içlerinde birbirini tetikler
        context.Request(); // A -> B geçişi
        context.Request(); // B -> A geçişi
        context.Request(); // A -> B geçişi
    }
}

/* Program Çıktısı:
Context: Durum değiştiriliyor (ConcreteStateA)...
ConcreteStateA: Talep işleniyor. B durumuna geçiş tetikleniyor.
Context: Durum değiştiriliyor (ConcreteStateB)...
ConcreteStateB: Talep işleniyor. A durumuna geri dönülüyor.
Context: Durum değiştiriliyor (ConcreteStateA)...
ConcreteStateA: Talep işleniyor. B durumuna geçiş tetikleniyor.
Context: Durum değiştiriliyor (ConcreteStateB)...
*/
```

Bu desenin bağlam (**Context**) nesnesinin, diğer tasarım desenleri ile iç içe kullanılması halinde, bu desen, **sıfır desen** (**null pattern**) olarak adlandırılır. Ayrıca bu desenle ilgili olarak iki önemli şeyden bahsetmek gerekir:

- ◊ Sahip olunan duruma göre icra edilecek davranış, tanımlanan **State** sınıfına bağlıdır. Yani ilgili davra-

nış somut sınıflardan biri tarafından icra edilir. Bu nedenle somut state (**ConcreteState**) sınıfları içinde tanımlanan yöntemler, durumu kullanan bağlam (**Context**) sınıf içinde tanımlanmak zorundadır.

- ◊ Durumu kullanan bağlam (**Context**) sınıf içinde hangi duruma geçildiğinin anlaşılması için ilgili duruma ilişkin tanımlanan özellik (**property**) içinde **set** yöntemi kullanılmak zorundadır.

Bu desen netice olarak, durum geçişlerini (**state transition**) açığa çıkarır. Bu deseni kullanırken aşağıda verilen durumlar özellikle irdelenmelidir;

- ◊ **Durum geçişleri kim tarafından tanımlanmalıdır?**: Bu desen, hangi şartlarda durumların ayrıştırılacağını belirtmez. Eğer şartlar belirgin ise bu işlem bağlam (**Context**) nesnesi tarafından yapılır. Bu şartlar belirgin değil ve karmaşık ise bu işlem **State** nesnesi ve halefi (**successor**) tarafından yapılır.
- ◊ **Tablo tabanlı alternatif**: Bu durumda durum geçişleri bir tablo aracılığı ile yapılır. Tablo üzerine sağlanacak her bir durum yazılır. Bu durumda, durum geçişlerini sağlayacak değişiklikler yeniden kodlama gerektirmez. Ancak bu yöntemde hız düşer. Durum geçişleri tekdüze (**uniform**) tanımlanmak zorundadır.
- ◊ **State nesnelerini yaratma ve yok etme**: Bu nesneler çalıştırma anında yaratılıp yok edildiklerinden, bu süre zarfında yapılacak işlemler için bir çözüm bulunmalıdır. İki yaklaşım vardır. Birincisi, State nesnesini ihtiyaç olduğunda yaratmak kullanmak ve öldürmektir. İkincisi ise bu nesneleri baştan yaratıp hiç öldürmemektir. Genellikle birinci yöntem tercih edilir.
- ◊ **Dinamik kalıtım (**dynamic inheritance**) kullanma**: Bazı durumlarda nesne, ait olduğu sınıfı çalıştırma anında değiştirir. Bu durumda talep edilen isteğin yerine getirilmesi tam olarak başarılabilir. Ancak birçok nesne yönelimli programlama dili bunu desteklemez.

Bu desen için en gerçekçi örneklerden biri **otomat makineleridir** (**vending machine**). Makinenin davranışı; içinde para olup olmamasına, ürünün kalıp kalmamasına veya ürün seçilip seçilmediğine göre tamamen değişir. Eğer para yoksa "Ürün Seç" düğmesi çalışmaz; para varsa çalışır;

```
using System;
// 1. State Arayüzü (Soyut Durum)
public abstract class State
{
    public abstract void InsertMoney();
    public abstract void EjectMoney();
    public abstract void SelectProduct();
    public abstract void Dispense();
}
// 2. Context: VendingMachine
public class VendingMachine
{
    private State _currentState;
    private int _count = 0;
    public VendingMachine(int productCount)
    {
        _count = productCount;
        if (_count > 0)
            _currentState = new NoMoneyState(this);
        else
            _currentState = new SoldOutState();
    }
    public void SetState(State state)
    {
        _currentState = state;
    }
    // İstemci tarafındaki komutlar
    public void InsertMoney() => _currentState.InsertMoney();
    public void EjectMoney() => _currentState.EjectMoney();
    public void SelectProduct() => _currentState.SelectProduct();
    public void Dispense() => _currentState.Dispense();
    public void ReleaseProduct()
    {
        if (_count > 0)
        {
            Console.WriteLine(">>> ÜRÜN VERİLDİ. Afiyet olsun!");
            _count--;
        }
    }
}
```

```

    }
}
public int GetCount() => _count;
}

// 3. Somut Durumlar (Concrete States)
// Para Yok Durumu
public class NoMoneyState : State
{
    private readonly VendingMachine _machine;
    public NoMoneyState(VendingMachine machine) => _machine = machine;
    public override void InsertMoney()
    {
        Console.WriteLine("Para kabul edildi.");
        _machine.SetState(new HasMoneyState(_machine));
    }
    public override void EjectMoney() => Console.WriteLine("Hata: Zaten para yok.");
    public override void SelectProduct() => Console.WriteLine("Hata: Önce para yüklemelisiniz.");
    public override void Dispense() => Console.WriteLine("Hata: Ödeme yapılmadan ürün
        ↪ verilmez.");
}

// Para Var Durumu
public class HasMoneyState : State
{
    private readonly VendingMachine _machine;
    public HasMoneyState(VendingMachine machine) => _machine = machine;
    public override void InsertMoney() => Console.WriteLine("Bilgi: Zaten para yüklü.");
    public override void EjectMoney()
    {
        Console.WriteLine("Para iade ediliyor...");
        _machine.SetState(new NoMoneyState(_machine));
    }
    public override void SelectProduct()
    {
        Console.WriteLine("Ürün seçildi, hazırlanıyor...");
        _machine.SetState(new SoldState(_machine));
        _machine.Dispense(); // Otomatik teslimat tetikleniyor
    }
    public override void Dispense() => Console.WriteLine("Bilgi: Önce ürün seçimi
        ↪ yapmalısınız.");
}

// Ürün Satıldı/Teslimat Durumu
public class SoldState : State
{
    private readonly VendingMachine _machine;
    public SoldState(VendingMachine machine) => _machine = machine;
    public override void InsertMoney() => Console.WriteLine("Lütfen bekleyin: Ürününüz
        ↪ hazırlanıyor.");
    public override void EjectMoney() => Console.WriteLine("Hata: İşlem geri alınamaz, ürün
        ↪ veriliyor.");
    public override void SelectProduct() => Console.WriteLine("Hata: İşlem zaten devam ediyor.");
    public override void Dispense()
    {
        _machine.ReleaseProduct();
        if (_machine.GetCount() > 0)
            _machine.SetState(new NoMoneyState(_machine));
        else
        {
            Console.WriteLine("Dikkat: Son ürün satıldı!");
            _machine.SetState(new SoldOutState());
        }
    }
}

```

```

}
// Stok Yok Durumu
public class SoldOutState : State
{
    public override void InsertMoney() => Console.WriteLine("Hata: Stok bitti, para
    → yükleyemezsiniz.");
    public override void EjectMoney() => Console.WriteLine("Hata: Para yok.");
    public override void SelectProduct() => Console.WriteLine("Hata: Stok bitti.");
    public override void Dispense() => Console.WriteLine("Hata: Stok yok.");
}
// 4. İstemci (Client)
class Program
{
    static void Main()
    {
        VendingMachine machine = new VendingMachine(2);
        Console.WriteLine("--- Senaryo 1: Başarılı Alım ---");
        machine.InsertMoney();
        machine.SelectProduct();
        Console.WriteLine("--- Senaryo 2: Para İadesi ---");
        machine.InsertMoney();
        machine.EjectMoney();
        Console.WriteLine("--- Senaryo 3: Son Ürünü Alma ---");
        machine.InsertMoney();
        machine.SelectProduct();
        Console.WriteLine("--- Senaryo 4: Stok Bittiğinde Deneme ---");
        machine.InsertMoney();
    }
}
/*Program Çıktısı:
--- Senaryo 1: Başarılı Alım ---
Para kabul edildi.
Ürün seçildi, hazırlanıyor...
>>> ÜRÜN VERİLDİ. Afiyet olsun!
--- Senaryo 2: Para İadesi ---
Para kabul edildi.
Para iade ediliyor...
--- Senaryo 3: Son Ürünü Alma ---
Para kabul edildi.
Ürün seçildi, hazırlanıyor...
>>> ÜRÜN VERİLDİ. Afiyet olsun!
Dikkat: Son ürün satıldı!
--- Senaryo 4: Stok Bittiğinde Deneme ---
Hata: Stok bitti, para yükleyemezsiniz.
*/

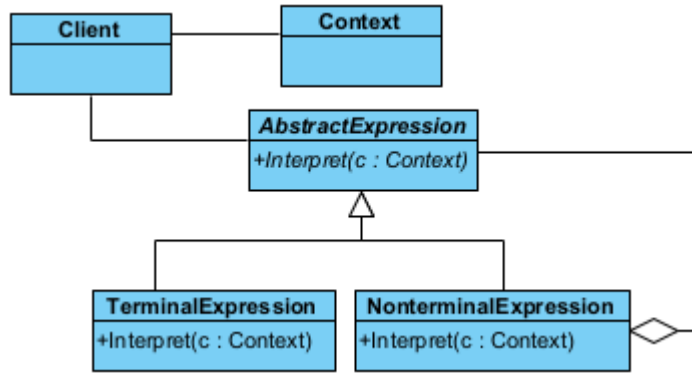
```

§18.4.11 Yorumlayıcı

Yorumlayıcı deseni (**interpreter pattern**), programlama dili yorumlama özelliğini uygulamalarımıza katmanın yolunu gösterir. Yani bir dil bilgisi için bir temsili ve aynı zamanda dil bilgisini anlayıp ona göre hareket etmeyi sağlayacak bir mekanizmayı tanımlar.

Bu desenin amacı, belirli bir gramer yapısına sahip ifadeleri (matematiksel işlemler, SQL komutları vb.) çözümlemektir.

- ◊ Soyut İfade (**AbstractExpression**), tüm düğümlerin sahip olduğu **interpret()** yöntemini tanımlayan ara yüzüdür.
- ◊ Uç İfade (**TerminalExpression**), gramerin en küçük birimleridir (Sayılar veya değişkenler). Daha fazla parçalanamazlar.
- ◊ Uç Olmayan İfade (**NonTerminalExpression**), gramerin kurallarını (Toplama, Çarpma vb.) temsil eder. Genellikle içinde diğer ifadeleri (uç yada veya uç olmayan) barındırır.
- ◊ Bağlam (**Context**), yorumlama sırasında ihtiyaç duyulan küresel bilgileri (Değişken değerleri, sembol tabloları) taşır.



Şekil 18.23: Yorumlayıcı Deseni UML Diyagramı

Şimdi desenimizi kodlayalım;

```

using System;
using System.Collections.Generic;
// 1. Context (Bağlam): Değişkenlerin değerlerini saklar.
public class Context
{
    private readonly Dictionary<string, int> _variables = new Dictionary<string, int>();
    public void SetVariable(string name, int value) => _variables[name] = value;
    public int GetVariable(string name)
    {
        if (_variables.TryGetValue(name, out int value)) return value;
        throw new KeyNotFoundException($"Değişken bulunamadı: {name}");
    }
}
// 2. AbstractExpression (Soyut İfade)
public interface IExpression
{
    int Interpret(Context context);
}
// 3. TerminalExpression (Uç İfadeler)
public class NumberExpression : IExpression
{
    private readonly int _number;
    public NumberExpression(int number) => _number = number;
    public int Interpret(Context context) => _number;
}
public class VariableExpression : IExpression
{
    private readonly string _name;
    public VariableExpression(string name) => _name = name;
    public int Interpret(Context context) => context.GetVariable(_name);
}
// 4. NonterminalExpression (Uç Olmayan İfadeler / Operatörler)
public class AddExpression : IExpression
{
    private readonly IExpression _left;
    private readonly IExpression _right;
    public AddExpression(IExpression left, IExpression right)
    {
        _left = left;
        _right = right;
    }
    public int Interpret(Context context)
    {
        return _left.Interpret(context) + _right.Interpret(context);
    }
}

```

```
// 5. İstemci (Client)
class Program
{
    static void Main()
    {
        Context context = new Context();
        context.SetVariable("x", 10);
        context.SetVariable("y", 5);
        // İfade: (x + y) + 20
        // C#'ta nesne ağacı şu şekilde kurulur:
        IExpression expression = new AddExpression(
            new AddExpression(new VariableExpression("x"), new VariableExpression("y")),
            new NumberExpression(20)
        );

        Console.WriteLine($"Sonuç: {expression.Interpret(context)}");
    }
}
/* Program Çıktısı:
Sonuç: 35
*/
```

Bu deseni uygularken aşağıda verilen durumlar irdelenmelidir;

- ♦ **Soyut söz dizimin oluşturulması:** Bu desen yorumlanacak dile ilişkin soyut bir söz dizimin nasıl olması gerektiğini açıklamaz. Tabloya dayalı, ağaç yapısı şeklinde el becerisi ile veya doğrudan istemci tarafından yapılabilir.
- ♦ **Yorumlayacak işlemin tanımlanması:** Eğer işlemler ortak bir yapıdaysa yeni bir *Expression* nesnesi tanımlanarak yapılabilir. Daha karmaşık durumlarda ziyaretçi deseni kullanılır. Programlama dillerinin çoğu soyut bir söz dizimine sahiptir ve yorumlanacak öğeler ayrı bir Visitor nesnesi tarafından yorumlanır.
- ♦ **Cümle sonlarını bitiren ortak semboller için sineksiklet deseni kullanılabilir:** Cümle sonlarındaki semboller genellikle bilgi saklamazlar. Yorumlamaya gerek duyulmazlar. Bu nedenle sineksiklet deseni (*flyweight pattern*) yorumlanacak yerler için bir eleme yapar.

Aşağıda bir metindeki ifadeyi hesaplayan bir uygulama örneği verilmiştir;

```
using System;
using System.Collections.Generic;
using System.Linq;
// --- Temel İfade Yapıları ---
public interface IExpression
{
    int Interpret();
}
public class NumberExpression : IExpression
{
    private readonly int _number;
    public NumberExpression(int n) => _number = n;
    public int Interpret() => _number;
}
public class AdditionExpression : IExpression
{
    private readonly IExpression _left, _right;
    public AdditionExpression(IExpression l, IExpression r) { _left = l; _right = r; }
    public int Interpret() => _left.Interpret() + _right.Interpret();
}
public class MultiplicationExpression : IExpression
{
    private readonly IExpression _left, _right;
    public MultiplicationExpression(IExpression l, IExpression r) { _left = l; _right = r; }
    public int Interpret() => _left.Interpret() * _right.Interpret();
}
// --- Dinamik Interpreter Sınıfı ---
```



```

public class Interpreter
{
    public int Evaluate(string expression)
    {
        var expressionTree = BuildExpressionTree(expression);
        return expressionTree.Interpret();
    }
    private IExpression BuildExpressionTree(string rawExpression)
    {
        // Boşluklara göre ayırarak tokenları elde edelim
        string[] tokens = rawExpression.Split(new[] { ' ' },
            ↪ StringSplitOptions.RemoveEmptyEntries);
        Stack<IExpression> nodes = new Stack<IExpression>();
        Stack<char> operators = new Stack<char>();
        // İşlem önceliği sözlüğü: Çarpma (+) dan üstündür
        var precedence = new Dictionary<char, int> { { '+', 1 }, { '*', 2 } };
        void ApplyOp()
        {
            var right = nodes.Pop();
            var left = nodes.Pop();
            char op = operators.Pop();
            if (op == '+')
                nodes.Push(new AdditionExpression(left, right));
            else if (op == '*')
                nodes.Push(new MultiplicationExpression(left, right));
        }
        foreach (string token in tokens)
        {
            if (int.TryParse(token, out int number))
            {
                nodes.Push(new NumberExpression(number));
            }
            else
            {
                char currentOp = token[0];
                while (operators.Count > 0 && precedence[operators.Peek()] >=
                    ↪ precedence[currentOp])
                {
                    ApplyOp();
                }
                operators.Push(currentOp);
            }
        }
        while (operators.Count > 0) ApplyOp();
        return nodes.Pop();
    }
}
// --- Client (İstemci) ---
class Program
{
    static void Main()
    {
        Interpreter parser = new Interpreter();

        string expr1 = "2 + 3 * 4"; // 14
        string expr2 = "10 * 2 + 5 * 3"; // 35
        string expr3 = "1 + 2 + 3 + 4"; // 10

        Console.WriteLine($"{expr1} = {parser.Evaluate(expr1)}");
        Console.WriteLine($"{expr2} = {parser.Evaluate(expr2)}");
        Console.WriteLine($"{expr3} = {parser.Evaluate(expr3)}");
    }
}

```

}

18.5 MVC Mimari Deseni

Model-View-Controller (MVC) mimari deseni (architectural pattern) ilk kez 1979'da *Trygve Reenskaug* tarafından "Applications Programming in Smalltalk-80: How to use Model-View-Controller" makalesiyle tanımlanmıştır. Günümüze kadar bu desenin "Model View Presenter-MVP" gibi birçok türevi geliştirilmiştir.

Bu mimari deseni kullanmak, iş süreci ile ilgili olan katmanın gösterim katmanından bağımsız tasarlanmasını sağlar. Bunun yanında sunum katmanını bağımsız olarak kontrol eden bir nesneye sahip oluruz. Bu katmanlar aslında fiziksel katman değildir. Bu nedenle bu desene uygun bağımsız olan katmanların bakımı kolaylaşır, hem de yeniden kullanımını (**reusing**) artırılmış olunur. Teknik olarak MVC bir sunum katmanı desendir (**UI Pattern**).

MVC desenini Borland firması, Visual Component Library (VCL) ve Component Library for Cross Platform (CLX) bileşenleri altyapısında yıllarca kullanmıştır. Benzer şekilde Apple firması, Cocoa için bu deseni kullanmıştır. Microsoft firması da Document/View mimarisi olarak yıllarca kullanmıştır. Günümüzde .NET Framework Web Uygulamaları bu kapsamda daha kolay ve yönetilebilir olarak geliştirilmektedir.

§18.5.1 MVC Deseni Temel Yapısı

MVC (**Model-View-Controller**), bir yazılım projesinde kodları görevlerine göre üç ana parçaya ayıran bir mimari desendir. Temel amacı, veriyi (mantık), kullanıcı arayüzünü ve bu ikisi arasındaki iletişimi birbirinden bağımsız hale getirerek kodun yönetimini kolaylaştırmaktır. Bunlar aşağıda detaylı olarak açıklanmıştır;

- ♦ **Model** : Sadece işlevsel davranışları yerine getiren kodlar burada toplanır. Kullanıcıya gösterilen ara yüzde kullanıcının gördüğü ve isteklerini ilettiği kısım dışında kalan konular bu katmanda gerçekleştirilir. Bazen kullanıcı tarafından modelin o andaki durumu (**state**) hakkında bilgi almasına izin verilir. Gösterimler (**view**) bilgiyi modelden aldığından, önceden modele kayıt (**subscribe**) olmak zorundadır. Bunun yanında model üzerinde bilgi değişikliği olduğunda bütün gösterimler bundan haberdar edilir (**notify**). Uygulamanın beynidir. Veritabanı işlemleri, hesaplamalar ve kurallar burada döner. "Veri nedir?" sorusuna cevap verir.
- ♦ **Gösterim (View)**: MVC deseninde birden fazla gösterim olmasına izin verilir. Gösterim, modele ilişkin bir kullanıcı ara yüzüdür. Yani modelden kullanıcıya gösterilecek değerleri alır ve kullanıcıya gösterir. Kullanıcının gördüğü ekrandır (HTML, CSS, butonlar). Sadece veriyi gösterir, arka planda ne olduğuyla ilgilenmez. "Veri nasıl görünmeli?" sorusuna cevap verir.
- ♦ **Denetçi (Controller)**: Denetçi, kullanıcı tarafından etkileşilen gösterimden gelen isteklere göre modeli günceller. Denetçi modeli güncellemek için, modelin değişik yöntemlerini çağırır ve modelin kendi durumunu (**state**) değiştirmesini sağlar. Buna bağlı olarak da model, tüm gösterimleri bundan haberdar eder. Kullanıcıdan gelen istekleri karşılar. Modelden veriyi alır ve hangi gösterimde gösterileceğine karar verir. "Hangi veri nereye gitmeli?" sorusuna cevap verir.

Basit bir restoran senaryosunda MVC'yi bir restorana benzetebiliriz;

- ♦ **Aşçı (Model)**: Mutfağın içindedir. Yemeği hazırlar, malzemeleri kontrol eder. Müşteriyi görmez, sadece siparişe göre yemeği (veriyi) üretir.
- ♦ **Yemek Masası/Sunum (View)**: Müşterinin önüne gelen tabak ve sunumdur. Müşteri yemeğin nasıl piştiğini görmez, sadece sunulan tabağı görür.
- ♦ **Garson (Controller)**: Müşteriden siparişi alır, aşçıya iletir. Yemek piştiğinde ise tabağı alır ve müşteriye servis eder.

MVC kullanmalıyız çünkü;

- ♦ **Kod Karmaşasını Önler**: Tasarımcı sadece **View** ile, yazılımcı sadece **Model/Controller** ile ilgilenir.
- ♦ **Bakım Kolaylığı**: Görünümü değiştirmek istediğinizde veritabanı koduna dokunmanız gerekmez.
- ♦ **Yeniden Kullanılabilirlik**: Aynı **Model** verisini (örneğin kullanıcı listesi), hem bir web sayfasında hem de bir mobil uygulamada farklı **View**'lar ile gösterebilirsiniz.

C# nesne yönelimli bir dil olduğu için MVC yapısını sınıflar arasındaki ilişkiyle kurgulamak, mantığı anlamak açısından çok faydalıdır. Bir "Öğrenci Bilgi Sistemi" üzerinden, bileşenlerin birbirine bağımlılığını (**coupling**) minimize edecek şekilde bir örnek verilebilir. Öğrenci Bilgi Sisteminde her bileşen kendi sorumluluğunu üstlenir: **Model** veriyi tutar, **View** ekrana basar, **Controller** ise akışı yönetir;

1. Veri ve İş Mantığı (**Model**): Sadece veriyi ve bu veriye erişim yöntemlerini içerir. Görselleştirme ile hiç ilgilenmez.

```
// 1. Model: Veriyi ve iş mantığını temsil eder.
public class OgrenciModel
{
```

```

public string Isim { get; set; }
public string Numara { get; set; }
public OgrenciModel(string isim, string numara)
{
    Isim = isim;
    Numara = numara;
}

```

2. Görünüm (View): Verinin kullanıcıya nasıl sunulacağını belirler. Genellikle sadece "Yazdır" veya "Göster" gibi yöntemleri vardır.

// 2. View: Verinin kullanıcıya nasıl sunulacağından sorumludur.

```

public class OgrenciView
{
    public void OgrenciDetaylariniBas(string isim, string numara)
    {
        Console.WriteLine("--- Öğrenci Bilgileri ---");
        Console.WriteLine($"Ad: {isim}");
        Console.WriteLine($"No: {numara}");
        Console.WriteLine("-----");
    }
}

```

3. Denetleyici (Controller): Veri ve Gösterim arasındaki köprüdür. Kullanıcıdan gelen güncellemeleri Veriye yansıtır ve Gösterimi güncellenmeye zorlar.

// 3. Controller: Model ve View arasındaki köprüdür.

```

public class OgrenciController
{
    private readonly OgrenciModel _model;
    private readonly OgrenciView _view;
    public OgrenciController(OgrenciModel model, OgrenciView view)
    {
        _model = model;
        _view = view;
    }
    // Model üzerindeki veriyi güncelleme/okuma proxy metodları
    public void SetOgrenciIsim(string isim) => _model.Isim = isim;
    public string GetOgrenciIsim() => _model.Isim;

    public void SetOgrenciNumara(string numara) => _model.Numara = numara;
    public string GetOgrenciNumara() => _model.Numara;
    // View'u güncelleme komutu
    public void GorunumuGuncelle()
    {
        _view.OgrenciDetaylariniBas(_model.Isim, _model.Numara);
    }
}

```

4. Sistemin Çalıştırılması:

// 4. İstemci (Client)

```

class Program
{
    static void Main()
    {
        // 1. Model'i (Veriyi) oluştur
        OgrenciModel model = new OgrenciModel("Ahmet Öztürk", "2026001");
        // 2. View'u (Arayüzü) oluştur
        OgrenciView view = new OgrenciView();
        // 3. Controller'ı bağla
        OgrenciController controller = new OgrenciController(model, view);
        // İlk hali ekrana bas
        controller.GorunumuGuncelle();
        // Veriyi Controller üzerinden güncelle
        controller.SetOgrenciIsim("Mehmet Demir");
        // Güncel hali tekrar bas
    }
}

```

```

        Console.WriteLine("Veri güncellendikten sonra:");
        controller.GorunumuGuncelle();
    }
}
/* Program Çıktısı:
--- Öğrenci Bilgileri ---
Ad: Ahmet Öztürk
No: 2026001
-----
Veri güncellendikten sonra:
--- Öğrenci Bilgileri ---
Ad: Mehmet Demir
No: 2026001
-----
*/

```

Buradaki mimari ile ilgili şunlar söylenebilir;

- ◊ **Bağımlılık Yönetimi (dependency management)** : MVC'nin ana varlık sebebi **gevşek bağıllık (loose coupling)** oluşturmaktır.
 - **Tek Yönlü Bağımlılık**: İdeal bir MVC yapısında bağımlılık hiyerarşisi şöyledir:
`View → Controller → Model`
`Model`, kendisini kimin görüntülediğini veya kimin yönettiğini bilmez. Bu, `Model`'in farklı platformlarda (Web, Mobil, Konsol) tekrar kullanılabilmesini sağlar.
 - **Arayüzler (interfaces)**: `Controller`, `View`'a doğrudan bağımlı olmak yerine bir `IView` arayüzüne bağımlı hale getirilir. Bu sayede birim testleri (**unit test**), gerçek bir ekran olmadan sahte (**mock**) nesnelerle `Controller`'ı test edebilir.
- ◊ **Bellek Yönetimi (memory management)**: MVC yapılarında nesnelerin yaşam döngüsü (**lifetime**), bellek sızıntılarını önlemek için dikkatle yönetilmelidir.
 - **Yaşam Süresi Sorumluluğu**: Genellikle `Controller`, `View` ve `Model`'den daha kısa ömürlüdür. Bir işlem bittiğinde `Controller` bellekten düşebilir ancak `Model` (veri) kalmaya devam edebilir.
 - **Çöp Toplayıcı GC**: `Controller` içinde `Model`'e ait olaylara (**events**) abone olursanız ve aboneliği iptal etmezseniz, dinlemeyi bırakanlar (**lapsed listener**) denilen bellek sızıntısı oluşur. `Controller` yok olsa bile, `Model` içindeki olay referansı `Controller`'ı bellekte tutmaya devam eder.
- ◊ **Gözlemci Deseni (observer pattern) ile İlişkisi**: MVC'nin "gizli sosu" genellikle gözlemci (**observer**) desenidir.
 - **Model-View Senkronizasyonu**: Model'deki veri değiştiğinde View'un otomatik güncellenmesi isteniyorsa, Model bir Subject (Konu), View ise bir Observer (Gözlemci) rolü üstlenir.
 - **Olaylar ve Temsilciler**: İletişim **event** anahtar kelimesiyle çok doğal bir şekilde yapılır. `Model` bir veri değiştiğinde `OnDataChanged` olayını ateşler, `View` bu olayı dinler ve kendini yeniler.
 - **Denetçinin Rolü**: Bazı katı MVC yorumlarında `View` asla `Model`'i dinlemez; tüm değişim bildirimleri `Controller` üzerinden geçer. Ancak modern yaklaşımlarda, `View` doğrudan `Model`'deki (veya `ViewModel`'deki) değişimleri gözlemler.

§18.5.2 Gözlemci ile MVC Deseni İçin Örnek Proje

Bu deseni bir bardak ve bu bardağı dolduran garson örneğini ele alarak inceleyelim. MVC deseni için bardağı dolduracak ya da boşaltacak olan garson, denetçi; Doldurulacak ve boşaltılacak olan bardak ise modeldir. Bu örnek projede MVC mimarisinin bir yardımcı bileşen kullanmadan bir projede nasıl uygulandığı detaylı olarak anlatılacaktır.

Projenin Oluşturulması

Bunun için Visual Studio üzerinde ile bir Windows uygulaması örnek olarak oluşturulacaktır. Oluşacak uygulamanın yapısı aşağıda verilmiştir;

```

MVCWindowsUygulaması/ → Çözüm (Solution)
├── WinFormsApp1/ → Proje (Windows Forms Application Project)
│   ├── IController.cs → Interface (IController)
│   ├── IModel.cs → Interface (IModel)
│   ├── IView.cs → Interface (IView)
│   ├── Controllers.cs → Class (Controller)
│   ├── Models.cs → Class (Model)
│   ├── ProgressBarView.cs → UserControl (View)
│   └── PieView.cs → UserControl (View)

```

```

├── Form1.cs → Form (Windows Form)
└── Program.cs → Ana (Main) Program

```

Projeyi oluşturmak için;

1. **Visual Studio** üzerinde **File→New→Project/Solution...** seçilerek **MVCWindowsUygulaması** adlı çözüm içinde **WinFormsApp1** adlı projemiz **Windows Forms App** seçilerek oluşturulur.
2. Oluşan projede **Form1** adlı bir form ve **Program.cs** dosyasını otomatik olarak oluştur.

Projede MVC Ara Yüzlerinin Oluşturulması

Proje açık iken **Solution Explorer** üzerinde projeye sağ tıklanarak **Add→New Item...** seçilerek dosya adı **IController.cs** belirlenerek arayüzümüz için dosya oluşturulur ve aşağıdaki şekilde değiştirilir;

```

public interface IController
{
    //Denetçinin (garsonun) Davranışları
    void AlınanDoldurmaİsteği(int pArtırımOranı);
    void AlınanBoşaltmaİsteği(int pEksilmeOranı);
}

```

Daha sonra **Solution Explorer** üzerinde projeye sağ tıklanarak **Add→New Item...** seçilerek dosya adı **IModel.cs** belirlenerek arayüzümüz için dosya oluşturulur ve aşağıdaki şekilde değiştirilir;

```

/* Doluluk oranı, en az, en çok ne kadar doldurulacağı ve adı belirlenecek bir bardağın yani
→ modelin, denetçiden (garson) gelen istekleri yerine getirir.*/
public interface IModel
{
    //Özellikler
    string Adı { get; set; }
    int DolulukOranı { get; }
    int EnAz { get; }
    int EnÇok { get; }
    //Davranışlar
    void Doldur(int pArtırımOranı);
    void Boşalt(int pEksilmeOranı);
}

```

Sonra **Solution Explorer** üzerinde projeye sağ tıklanarak **Add→New Item...** seçilerek dosya adı **IView.cs** belirlenerek arayüzümüz için dosya oluşturulur ve aşağıdaki şekilde değiştirilir;

```

/* Bardağın doluluk oranını gösteren bir gösterime ilişkin ara yüzü aşağıdaki gibi
→ tanımlayabiliriz. Burada tanımlanan ara yüz, denetçinin görevlerinin bir kısmının ortaya
→ çıkarır. Yani dolu olan bir bardak için doldurma isteğine ve boş olan bir bardak için
→ boşaltma isteğine gösterim izin vermemelidir. */
public interface IView
{
    //View Davranışları
    void DoldurmaAktif();
    void DoldurmaPasif();
    void BoşaltmaAktif();
    void BoşaltmaPasif();
}

```

Son olarak **IController** ara yüzümüze hangi model ile çalışacağını söyleyen değişiklikler de yapılır;

```

public interface IController
{
    //Denetçinin (garsonun) Davranışları
    void AlınanDoldurmaİsteği(int pArtırımOranı);
    void AlınanBoşaltmaİsteği(int pEksilmeOranı);
    //MODEL Özelliği
    IModel Model { set; get; }
}

```

Projeye Gözlemci Deseninin Uygulanması

Bundan sonraki aşama biraz daha ustalık istemektedir. Çünkü modeldeki değişikliklerden gösterimin haberi olmasını istiyoruz. İşte bunun için Gözlemci desenini bu ara yüze uyguluyoruz. Bunun için ilk önce

IModel ara yüzü aşağıdaki şekilde değiştirilir;

/* Doluluk oranı, en az, en çok ne kadar doldurulacağı ve adı belirlenecek bir bardağın yani
→ modelin, denetçiden (garson) gelen istekleri yerine getirir.*/

```
using System.Collections; //ArrayList için
public interface IModel
{
    //Özellikler
    string Adı { get; set; }
    int DolulukOranı { get; }
    int EnAz { get; }
    int EnÇok { get; }
    //Davranışlar
    void Doldur(int pArtırımOranı);
    void Boşalt(int pEksilmeOranı);
    //Gözlemci Desenine İlişkin Davranışlar
    ArrayList Observers { get; }
    void AddObserver(IView pView);
    void RemoveObserver(IView pView);
    void NotifyObservers();
}
```

Devamında **IView** ara yüzü de aşağıdaki şekilde değiştirilir;

/* Bardağın doluluk oranını gösteren bir gösterime ilişkin ara yüzü aşağıdaki gibi
→ tanımlayabiliriz. Burada tanımlanan ara yüz, denetçinin görevlerinin bir kısmının ortaya
→ çıkarır. Yani dolu olan bir bardak için doldurma isteğine ve boş olan bir bardak için
→ boşaltma isteğine gösterim izin vermemelidir. */

```
public interface IView
{
    //View Davranışları
    void DoldurmaAktif();
    void DoldurmaPasif();
    void BoşaltmaAktif();
    void BoşaltmaPasif();

    //Gözlemci Deseni Davranışları: Modelden gelen bilgilere göre View güncellenir
    void Update(IModel pModel);
}
```

Son olarak gösterim ara yüzünden alınacak kullanıcı isteklerini denetçiye iletmek için **IView** ara yüzü aşağıdaki gibi değiştirilir;

/* Bardağın doluluk oranını gösteren bir gösterime ilişkin ara yüzü aşağıdaki gibi
→ tanımlayabiliriz. Burada tanımlanan ara yüz, denetçinin görevlerinin bir kısmının ortaya
→ çıkarır. Yani dolu olan bir bardak için doldurma isteğine ve boş olan bir bardak için
→ boşaltma isteğine gösterim izin vermemelidir. */

```
public interface IView
{
    //View Davranışları
    void DoldurmaAktif();
    void DoldurmaPasif();
    void BoşaltmaAktif();
    void BoşaltmaPasif();

    //View üzerinden kullanıcı isteklerini Controller'e iletmek için özellik tanımlama
    IController Control { set; }

    //Gözlemci Deseni Davranışları:
    //Modelden gelen bilgilere göre View güncellenir
    void Update(IModel pModel);
}
```

Model ve Denetçi Arayüzleri Gerçekleştirecek Sınıfların Tanımlanması

Artık verilen örneği gerçekleştirmek için yüzlerimiz hazır. Bu ara yüzlere bağlı kalarak sırasıyla soyut ve somut bir model (bardak), denetçiyi (hızlı veya çalışan garson) oluşturabiliriz.

Proje açık iken **Solution Explorer** üzerinde projeye sağ tıklanarak **Add→New Item...** seçilerek dosya adı **Models.cs** belirlenerek arayüzümüz için dosya oluşturulur ve aşağıdaki şekilde değiştirilir;

```
namespace WinFormsApp1
{
    using System.Collections; //ArrayList için
    abstract class SoyutKap : IModel
    {
        private ArrayList views = new ArrayList();
        private string adi;
        private int dolulukOranı;
        private int enAz;
        private int enÇok;
        public SoyutKap(string pAdi, int pEnAz, int pEnÇok,
            int pDolulukOranı)
        {
            this.adi = pAdi;
            this.enAz = pEnAz;
            this.enÇok = pEnÇok;
            this.dolulukOranı = pDolulukOranı;
        }
        public string Adı
        {
            get { return adi; }
            set { adi = value; }
        }
        public int DolulukOranı
        {
            get { return dolulukOranı; }
        }
        public int EnAz
        {
            get { return enAz; }
        }
        public int EnÇok
        {
            get { return enÇok; }
        }
        public void Doldur(int pArtırımOranı)
        {
            this.dolulukOranı += pArtırımOranı;
            if (dolulukOranı >= this.enÇok) dolulukOranı = enÇok;

            this.NotifyObservers();
        }
        public void Boşalt(int pEksilmeOranı)
        {
            this.dolulukOranı -= pEksilmeOranı;
            if (dolulukOranı <= this.enAz) dolulukOranı = enAz;

            this.NotifyObservers();
        }
        public ArrayList Observers
        {
            get { return views; }
        }
        public void AddObserver(IView pView)
        {
            views.Add(pView);
        }
    }
}
```



```

    }
    public void RemoveObserver(IView pView)
    {
        views.Remove(pView);
    }
    public void NotifyObservers()
    {
        foreach (IView view in views)
        {
            view.Update(this);
        }
    }
}
class somutBardak : SoyutKap
{
    public somutBardak(string pAdi) : base(pAdi, 0, 100, 0) { }
}
}

```

Proje açık iken **Solution Explorer** üzerinde projeye sağ tıklanarak **Add→New Item...** seçilerek dosya adı **Controllers.cs** belirlenerek arayüzümüz için dosya oluşturulur ve aşağıdaki şekilde değiştirilir;

```

namespace WinFormsApp1
{
    class HızlıGarson : IController
    {
        private IModel model;
        public IModel Model
        {
            set { model = value; }
            get { return model; }
        }
        public void AlınanDoldurmaİsteği(int pArtırımOranı)
        {
            if (model != null)
            {
                model.Doldur(pArtırımOranı * 10);
                DurumKontrol();
            }
        }
        public void AlınanBoşaltmaİsteği(int pEksilmeOranı)
        {
            if (model != null)
            {
                model.Boşalt(pEksilmeOranı * 10);
                DurumKontrol();
            }
        }
        public void DurumKontrol()
        {
            foreach (IView view in model.Observers)
            {
                if (model.DolulukOranı >= model.EnÇok)
                    view.DoldurmaPasif();
                else
                    view.DoldurmaAktif();

                if (model.DolulukOranı <= model.EnAz)
                    view.BoşaltmaPasif();
                else
                    view.BoşaltmaAktif();
            }
        }
    }
}

```



```

    }
}
class YavaşGarson : IController
{
    private IModel model;
    public IModel Model
    {
        set { model = value; }
        get { return model; }
    }
    public void AlınanDoldurmaİsteği(int pArtırımOranı)
    {
        if (model != null)
        {
            model.Doldur(pArtırımOranı);
            DurumKontrol();
        }
    }
    public void AlınanBoşaltmaİsteği(int pEksilmeOranı)
    {
        if (model != null)
        {
            model.Boşalt(pEksilmeOranı);
            DurumKontrol();
        }
    }
    public void DurumKontrol()
    {
        foreach (IView view in model.Observers)
        {
            if (model.DolulukOranı >= model.EnÇok)
                view.DoldurmaPasif();
            else
                view.DoldurmaAktif();

            if (model.DolulukOranı <= model.EnAz)
                view.BoşaltmaPasif();
            else
                view.BoşaltmaAktif();
        }
    }
}
}

```

Gösterim Arayüzleri için Kullanıcı Kontrollerinin Tanımlanması

Artık gösterimler için örnek olarak iki farklı **kullanıcı kontrolü** (**user control**) tasarlanacaktır. Bunlardan biri **ProgressBarView** kontrolüdür.

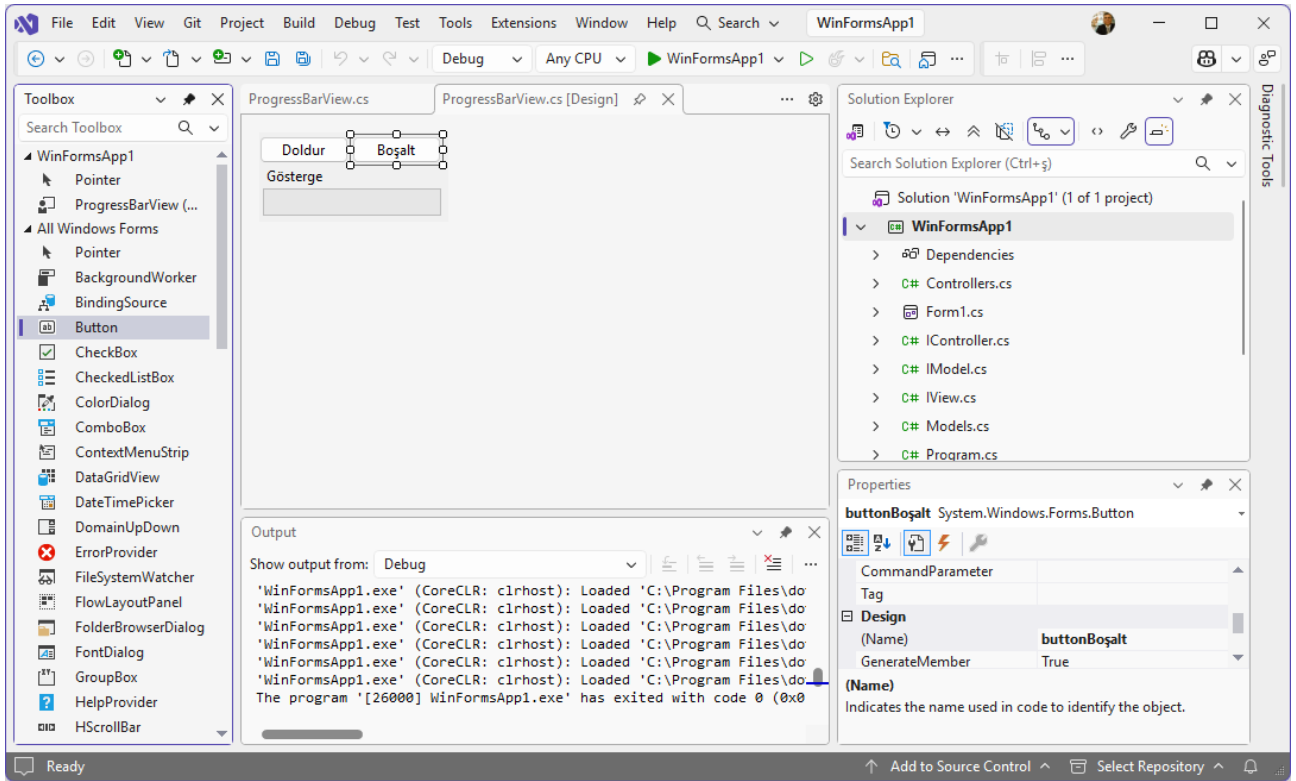
Açık olan projemize kullanıcı kontrolü eklemek için **Solution Explorer** üzerinde projemize sağ tıklanarak **Add → User Control (Windows Forms)...** seçilerek seçilir ve dosya adı **ProgressBarView.cs** belirlenerek kontrolümüz için dosya oluşturulur. Daha sonra **View → Toolbox** tıklanır ve açılan penceren kullanıcı kontrolüne sürükleyip bırak ile iki adet **Button** ve bir adet **ProgressBar** ve bir adet **Label** eklenir (Şekil 18.24);

Eklenen kontrollerin **Name** özellikleri **buttonDoldur**, **buttonBoşalt**, **labelGösterge** ve **progressBarDoluluk** olarak değiştirilir. Sonrasında butonların **onClick()** olayına çift tıklanır. Açılan **ProgressBarView.cs** dosyasında aşağıdaki değişiklikler yapılır;

```

namespace WinFormsApp1
{
    public partial class ProgressBarView : UserControl, IView //IView elle eklendi ve
    ↪ gerçekleştirildi.
    {
        private IController _control; //Elle Eklendi
        public IController Control

```



Şekil 18.24: ProgressBarView Kullanıcı Kontrolü

```

{
    set { //Elle Eklendi
        _control = value;
    }
}

public ProgressBarView()
{
    InitializeComponent();
}

private void buttonDoldur_Click(object sender, EventArgs e)
{
    _control.AlınanDoldurmaİstegi(1); //elle eklendi
}

private void buttonBoşalt_Click(object sender, EventArgs e)
{
    _control.AlınanBoşaltmaİstegi(1); //elle eklendi
}

public void DoldurmaAktif()
{
    this.buttonDoldur.Enabled = true; // elle eklendi
}

public void DoldurmaPasif()
{
    this.buttonDoldur.Enabled = false; //elle eklendi
}

public void BoşaltmaAktif()
{
    this.buttonBoşalt.Enabled = true; //elle eklendi
}

```

```

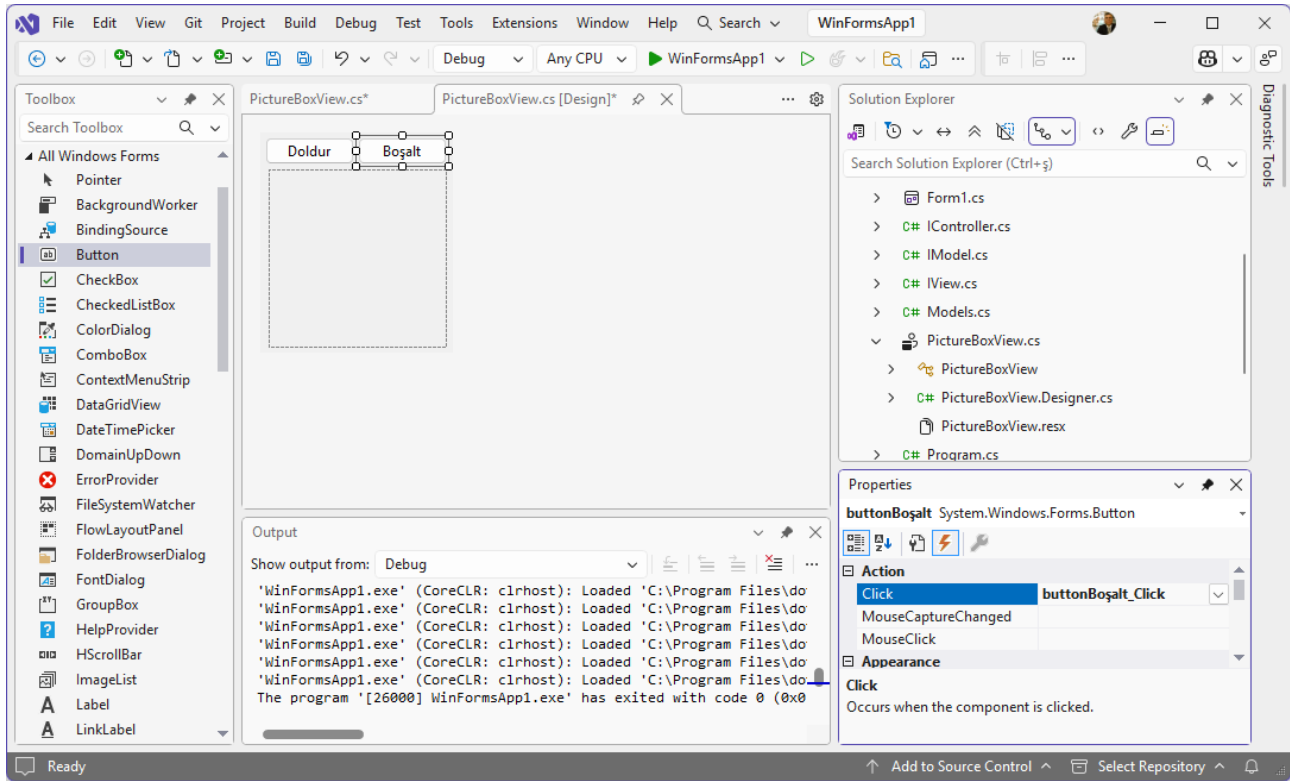
    }

    public void BoşaltmaPasif()
    {
        this.buttonBoşalt.Enabled = false; //elle eklendi
    }

    public void Update(IModel pModel)
    {
        //elle eklendi:
        this.labelGösterge.Text = string.Format("{0} Doluluğu (%):{1}",
            pModel.Adı, pModel.DolulukOranı);
        this.progressBarDoluluk.Value = pModel.DolulukOranı * 100 / pModel.EnÇok;
    }
}
}

```

İkinci kullanıcı kontrolü ise **PieView** kontrolüdür. Bunun için mevcut projemize **PictureBoxView** adında yeni bir kullanıcı kontrolü eklenir. Kullanıcı kontrolüne **Toolbox** üzerinden sürükleyip bırak ile iki adet **Button**, ve bir adet **PictureBox** eklenir (Şekil 18.25).



Şekil 18.25: PictureBox Kullanıcı Kontrolü

Eklenen kontrollerin **Name** özellikleri **buttonDoldur**, **buttonBoşalt** ve **pictureBoxPie** olarak değiştirilir ve butonların **onClick()** olayına çift tıklanır. Açılan **PieView.cs** dosyasında aşağıdaki değişiklikler yapılır;

```

namespace WinFormsApp1
{
    public partial class PictureBoxView : UserControl, IView //Iview eklendi ve gerçekleştirildi
    {
        private IController _control;
        public IController Control
        {
            set
            {
                _control = value;
            }
        }
    }
}

```

```

public PictureBoxView()
{
    InitializeComponent();
}

private void buttonDoldur_Click(object sender, EventArgs e)
{
    _control.AlınanDoldurmaİsteği(1); //Elle Eklendi
}

private void buttonBoşalt_Click(object sender, EventArgs e)
{
    _control.AlınanBoşaltmaİsteği(1); //Elle Eklendi
}

public void DoldurmaAktif()
{
    this.buttonDoldur.Enabled = true; //Elle Eklendi
}

public void DoldurmaPasif()
{
    this.buttonDoldur.Enabled = false; //Elle Eklendi
}

public void BoşaltmaAktif()
{
    this.buttonBoşalt.Enabled = true; //Elle Eklendi
}

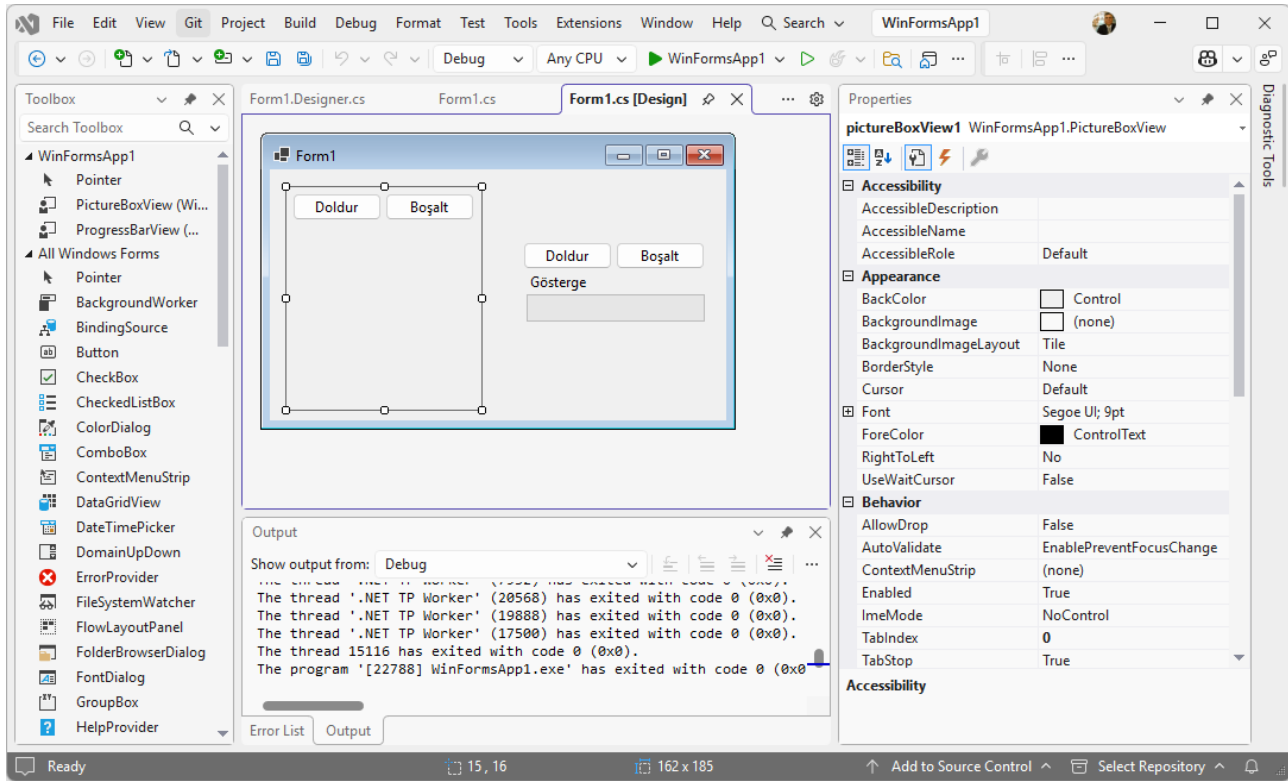
public void BoşaltmaPasif()
{
    this.buttonBoşalt.Enabled = false; //Elle Eklendi
}

public void Update(IModel pModel)
{
    //Elle Eklendi:
    SolidBrush whiteBrush = new SolidBrush(Color.White);
    SolidBrush redBrush = new SolidBrush(Color.Red);
    Graphics graphics = this.pictureBoxPie.CreateGraphics();
    graphics.FillRectangle(whiteBrush, 0, 0,
        this.pictureBoxPie.Width,
        this.pictureBoxPie.Height);
    graphics.FillPie(redBrush, 0, 0,
        this.pictureBoxPie.Width,
        this.pictureBoxPie.Height, 0.0F,
        pModel.DolulukOranı * 360.0F / pModel.EnÇok);
    Font font = new Font(FontFamily.GenericSerif, 12);
    Point point = new Point(this.pictureBoxPie.Width / 2,
        this.pictureBoxPie.Height / 2);
    SolidBrush blackBrush = new SolidBrush(Color.Black);
    graphics.DrawString(string.Format("%{0}",
        pModel.DolulukOranı), font, blackBrush, point);
}
}

```

Asıl Forma Kullanıcı Kontrollerini Ekleme ve Çalıştırma

Kullanıcı Kontrollerimizi tanımladıktan sonra uygulamanın asıl form olan `Form1.cs` formuna dönüp bu kontrolleri ekleyebiliriz. Bu forma kullanıcı kontrollerimizi **Toolbox** üzerinden sürükleyip bırakarak ekleyebiliriz.



Şekil 18.26: WinFormApp1 Uygulamasında Form1

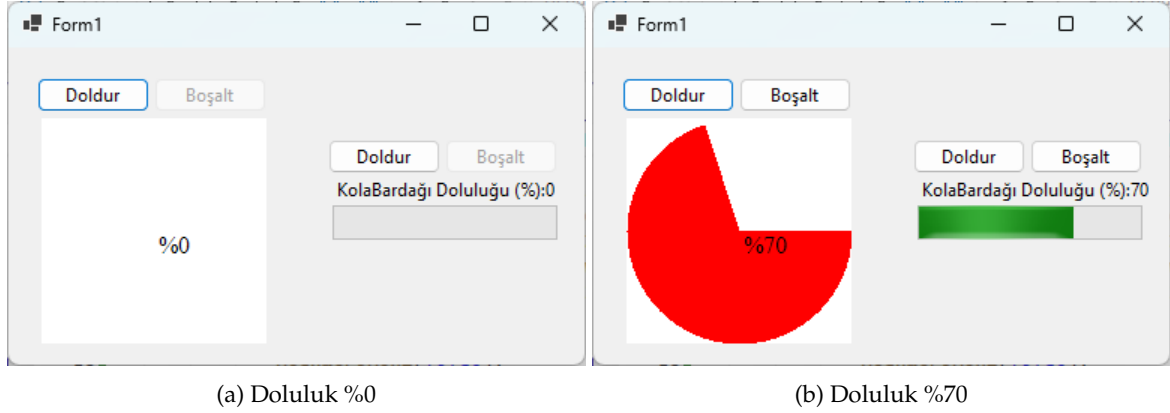
Formun kod kısmına geçerek Form1.cs dosyasında aşağıdaki değişiklikleri yaparak programı son haline getiririz.

```
namespace WinFormsApp1
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
            MVCBaşlat();
        }

        public void MVCBaşlat()
        {
            //Controller
            IController garson = new HızlıGarson();
            //Model
            IModel bardak = new somutBardak("KoLaBardağı");
            //Denetçinin Modeli
            garson.Model = bardak;
            //Her bir Gösterime Denetçi Ataması
            pieView1.Control = garson;
            progressBarView1.Control = garson;
            //Modele Gösterimlerin Kayıt Olması
            bardak.AddObserver(pieView1);
            bardak.AddObserver(progressBarView1);
        }
    }
}
```

Program çalıştırıldığında, MVC deseni ile birbirine gevşek bağlı (**loose couple**) denetleme ara yüzlerinin yazılımlara kolay eklendiğini görebiliriz. Ayrıca programda yapılacak değişiklik isteklerine MVC deseni daha hızlı cevap verecektir. Bunun yanı sıra hazırlanan ara yüzlerle birlikte, soyut sınıfları da başka projelere taşıyıp

yeniden kullanmak (**reusing**) oldukça kolay olacaktır. Böylece ileride olabilecek değişikliklere daha hızlı ayak uydurabilen daha esnek bir projeye sahip olmuş oluruz. Sonuç olarak MVC deseni, kullanıcı ara yüzü işlemleri ile iş mantığının birbirinden ayrılmasını sağlar. Ayrıca Model, Gösterim ve Denetçi bileşenleri ayrı katmanlar halinde Katmanlı Mimari başlığı altında anlatıldığı şekilde tasarlanabilir.



Şekil 18.27: WinFormApp1 Uygulamasının Çalışması

ASP.NET MVC ve Jakarta EE gibi günümüzde en çok kullanılan teknolojiler MVC ara yüzlerini web uygulamasına entegre edilmiş olarak sunmaktadır. ASP.NET MVC altyapısında gösterim bileşeni ASP.NET web sayfası olacak şekilde hazırlanır. Jakarta EE altyapısında ise sunum bileşeni JSP olacak şekilde hazırlanır. Model ve Denetçi bileşenleri ise genellikle nesneye dayalı sınıflar olarak tasarlanırlar.

18.6 Düşün ve Çöz

- ◊ **Düşün:** Tekil Nesne (**Singleton**) deseni çok iş parçacıklı (**multi-threaded**) ortamlarda neden tehlikeli olabilir? İş parçacığı-güvenli **Singleton** kodlaması (**implementation**) için "double-checked locking" ve **Lazy<T>** yaklaşımlarını karşılaştırınız.
- ◊ **Düşün:** Fabrika Yöntemi ile Soyut Fabrika deseni arasındaki fark nedir? Bir oyun motoru geliştirirken farklı platformlar (PC, Konsol, Mobil) için hangi deseni kullanırsınız?
- ◊ **Düşün:** Gözlemci deseni ile **event/delegate** mekanizması arasındaki ilişki nedir? Gözlemci deseni yanlış uygulandığında bellek sızıntısına nasıl yol açabilir?
- ◊ **Düşün:** Strateji deseni ve Durum deseni yapısal olarak birbirine benzer görünse de farklı niyetlere hizmet eder. Bu farkı bir varlık (örneğin bir sipariş) üzerinden açıklayınız.
- ◊ **Düşün:** Komut (**Command**) deseni geri alma (**undo**) işlevini nasıl mümkün kılar? Metin editöründeki **Ctrl+Z** mekanizmasını bu deseni nasıl modelleriniz?
- ◊ **Düşün:** Dekoratör deseni kalıtıma alternatif olarak neden tercih edilebilir? Bir kahve siparişi sistemi üzerinden dekoratörün esnekliğini gösteriniz.
- ◊ **Düşün:** MVC deseni ile MVP (**Model-View-Presenter**) ve MVVM (**Model-View-ViewModel**) arasındaki farklar nelerdir? WPF veya MAUI geliştirmesinde hangisi daha yaygın kullanılır?
- ◊ **Çöz:** Bir metin editörü için Komut deseni uygulayınız: Yazma, Silme ve Geri Alma işlemlerini destekleyen bir komut yığını (**command stack**) oluşturun.
- ◊ **Çöz:** Farklı veritabanı motorlarına (SQLite, MSSQL, PostgreSQL) bağlanabilen bir uygulama için Soyut Fabrika deseni kullanarak bağlantı ve sorgu nesnelerini üretin.
- ◊ **Çöz:** Bir bildirim sistemi için Gözlemci deseni uygulayınız: **HavaDurumu** sınıfı sıcaklık değiştiğinde kayıtlı tüm gözlemcileri (**SicaklikGostergesi**, **IklimlendirmeSistemi**) haberdar etsin.
- ◊ **Çöz:** Dekoratör deseni kullanarak bir metin biçimlendirme sistemi oluşturunuz: Kalın, İtalik ve Altı Çizili biçimleri herhangi bir kombinasyonda uygulanabilsin.
- ◊ **Çöz:** Strateji deseni kullanarak aşağıda verilen farklı sıralama algoritmalarını **KabarcıkSiralama**, **HızlıSiralama**, **BirlestirmeSiralama** arasında çalışma zamanında değiştirilebilen bir sıralayıcı tasarlayınız.